UNIVERSITÄT BONN

Chapter 5: Segmentation

Prof. Dr.-Ing. Thomas Schultz

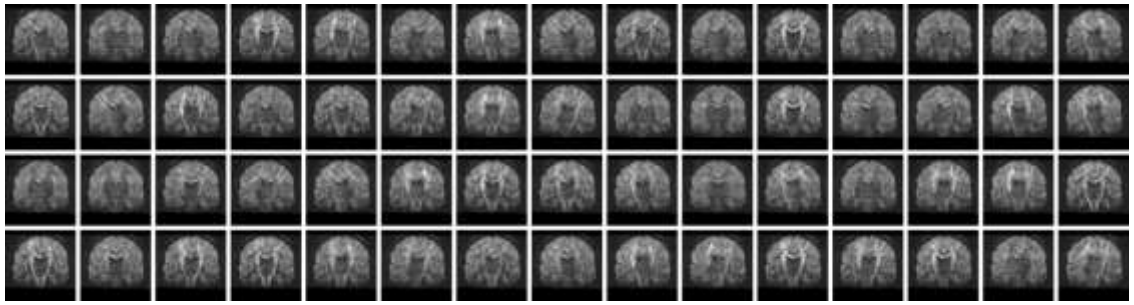
URL: http://cg.cs.uni-bonn.de/schultz/
E-Mail: schultz@cs.uni-bonn.de
Office: Friedrich-Hirzebruch-Allee 6, Room 2.117

November 24/27 and December 4, 2025

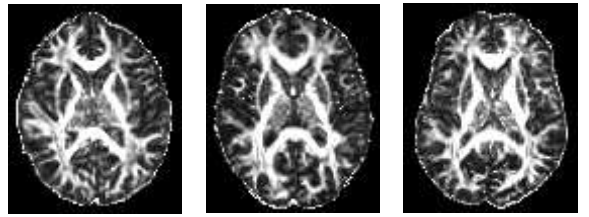
Image Acquisition and Analysis in Neuroscience

Acquisition

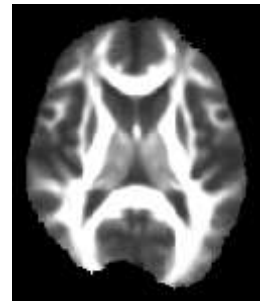
Reconstruction & Artifact Correction



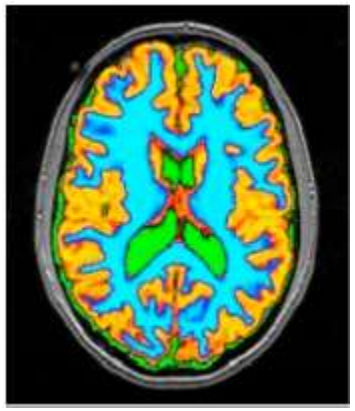
Registration



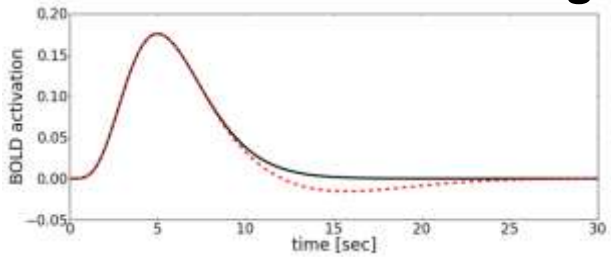
Atlas Construction



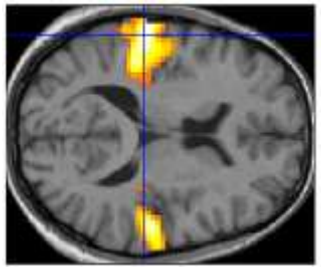
Segmentation



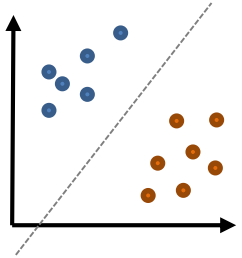
Mathematical Modeling



Hypothesis Testing



Machine Learning



5.1 Image Segmentation: Goals and Definitions

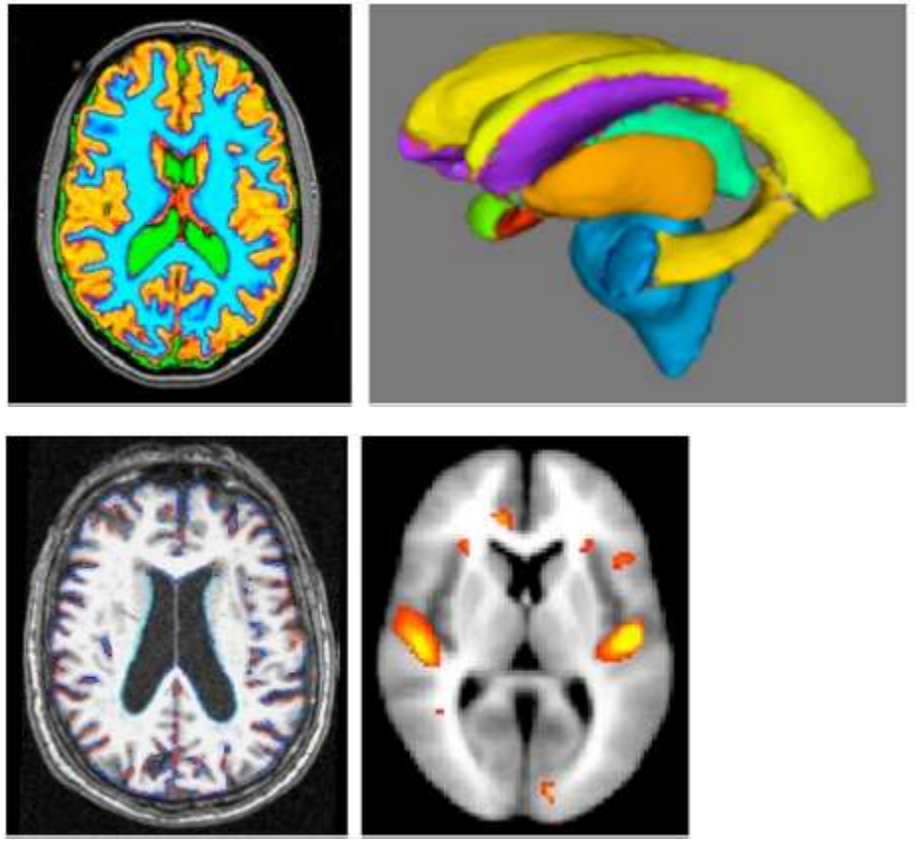
Segmentation, Clustering, Classification

Three closely related problems:

- **Segmentation** partitions images into connected regions that belong to the same object or material
- **Clustering** identifies groups of similar points in a (potentially abstract) data space
  - Often used for segmentation, examples below
- **Classification** assigns a voxel or pre-segmented object to a previously known category
  - *Example:* Detect material at each voxel

Segmentation in Neuroimaging

- **Uses of segmentation** in neuroimaging:
  - Extract the brain from MR scans
    - Should always be done before normalization and tissue type segmentation
    - Restrict domain of statistical analysis
  - Measure the volume of the brain or specific regions of interest (ROIs)
    - Longitudinal comparison: Study learning or aging
    - Between-group comparison: Study functional localization
  - Limit other analyses to specific ROIs



Images from Oxford FMRIB

Overview of This Chapter

We will focus on **three families of segmentation techniques,** which are implemented in widely used software packages:

- **Deformable models**
  - Regularize geometrical properties of surfaces, e.g., curvature
  - Frequently used for brain extraction
- **Markov Random Fields**
  - Regularize spatial smoothness of label maps
  - Frequently used for segmenting tissue types or specific brain areas
- **Convolutional Neural Networks**
  - Learn from many examples to achieve fast and robust results

Metrics for Evaluating Segmentations: Overlap

- The **overlap** between
  - Segmentation A and
  - Reference („Ground Truth“) B

  can be quantified with the
  - Dice-Score (DSC) or
  - IoU = Intersection over Union
    - Ratio between intersection and union of the two masks

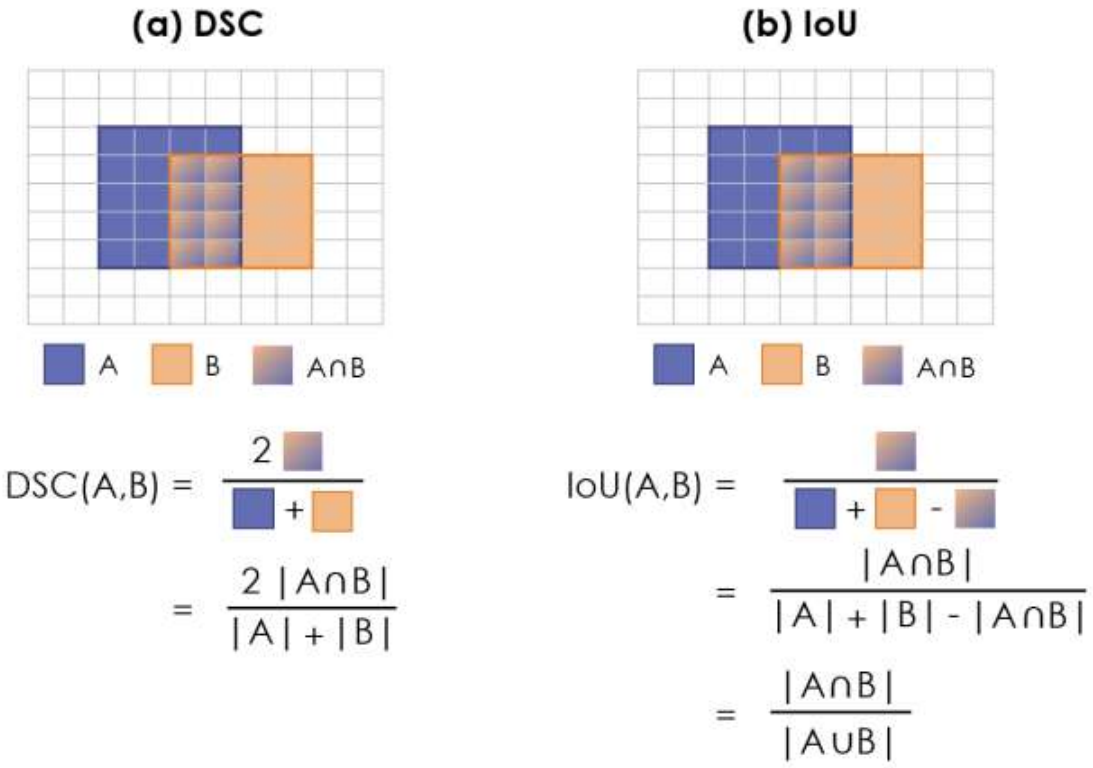


Image source: Reinke et al. 2021

Metrics for Evaluating Segmentations: Contour based

- The maximal difference between **contours** of
  - Segmentation A and
  - Reference („Ground Truth“) B

  can be quantified with the
  - Hausdorff distance (HD)
  - More robustly:
    HD with percentile

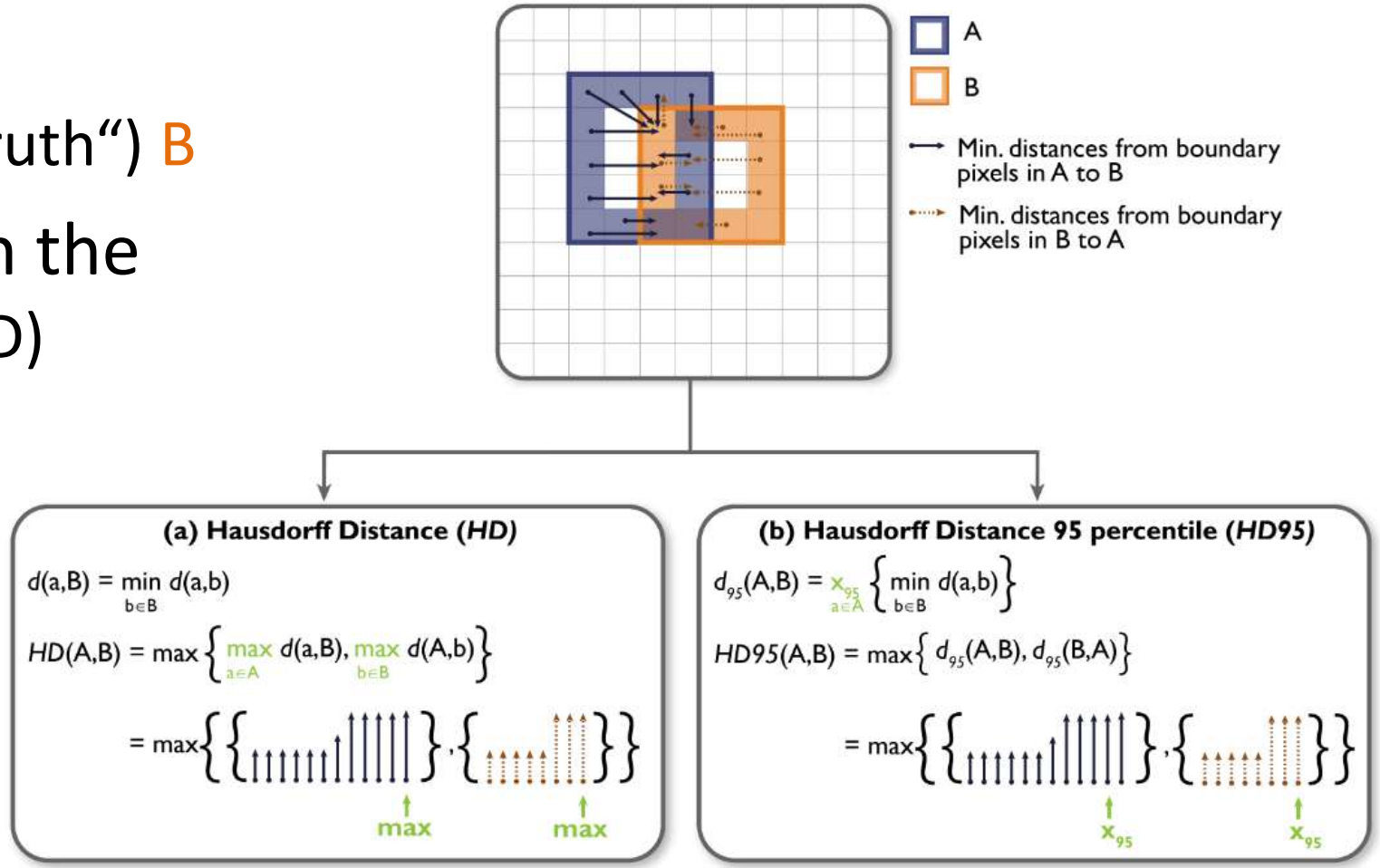


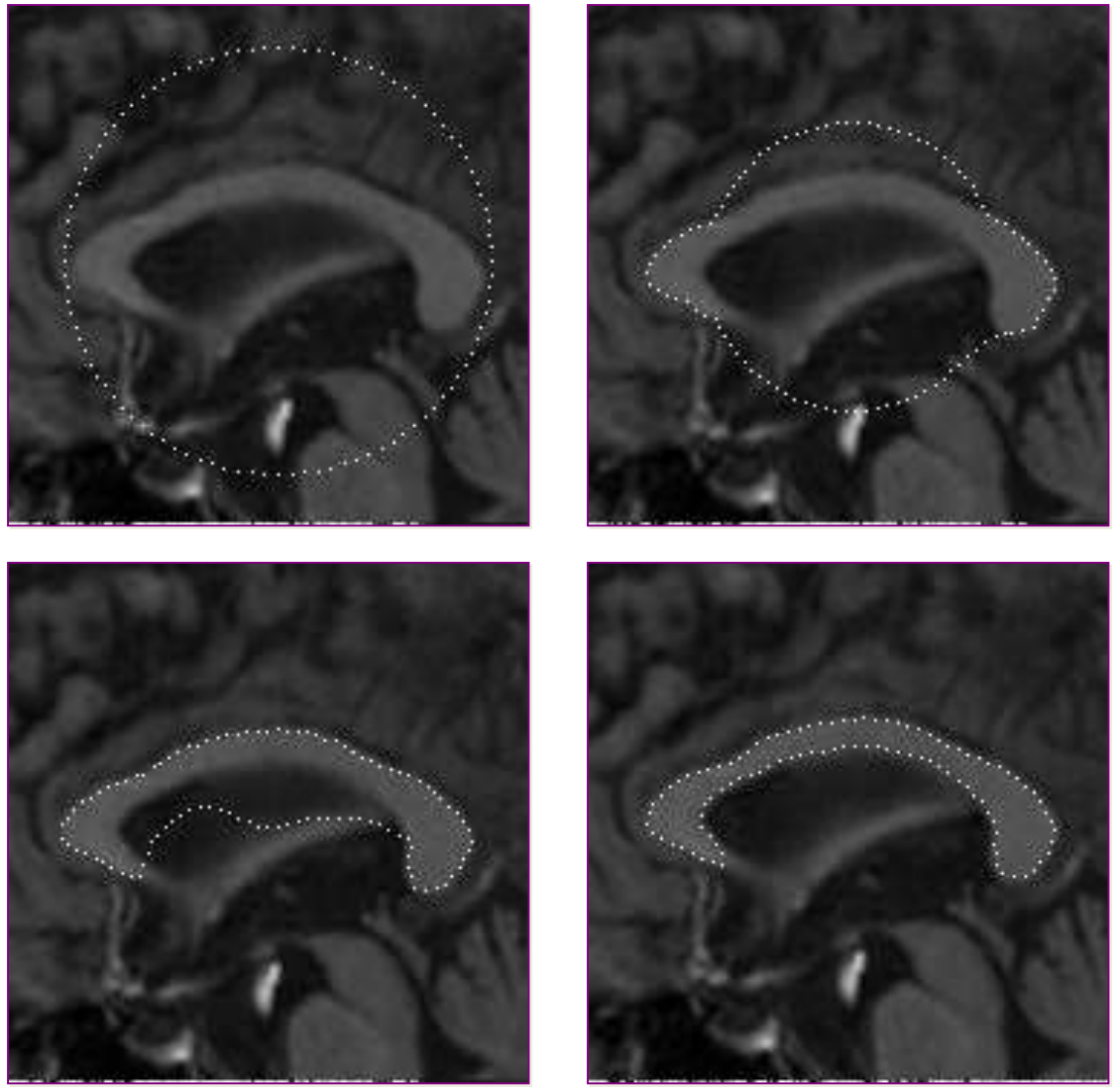
Image source: Reinke et al. 2021

5.2 Deformable Models

Intuition: Deformable Models

Deformable models are "active" contours that deform until they reach a steady state

- Contour is moved by forces that
  - *Either* are computed from a gradient descent of a weighted sum of
    - External energy (data term)
      - e.g., stop at edges, attempt to capture regions with specific intensities
    - Internal energy (smoothness term)
      - e.g., contour should be short / smooth
  - *Or* are specified ad-hoc to combine image information with regularity



Images taken from [Davatzikos et al. 1996]

Real-World Example: FSL's Brain Extraction Tool

Brain Extraction Tool (BET) from FSL, http://fsl.fmrib.ox.ac.uk/

- Widely used (>13,000 citations on google scholar in Nov 2025)
- Almost fully automated, two intuitive parameters:
  1. Make the segmentation larger or smaller
  2. Compensate for intensity gradients in the 3D image
- Uses a **deformable surface model**
  - Automated initialization to a small sphere that is (mostly) inside of the brain
  - Force terms attempt to push it to a location at which image intensity drops, while avoiding high curvature and preserving a uniform sampling

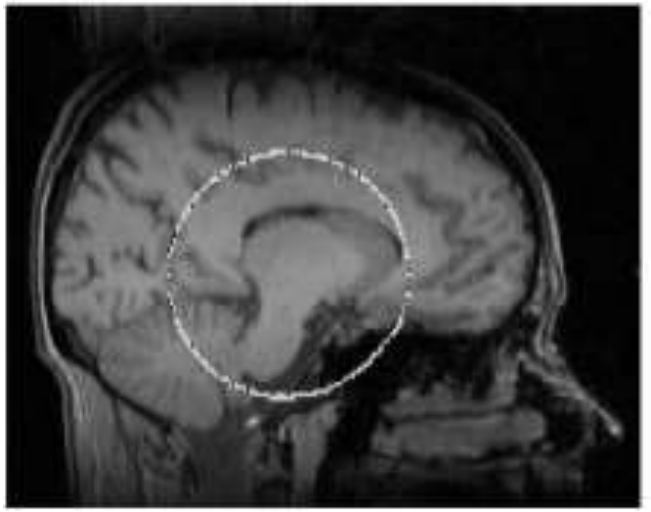


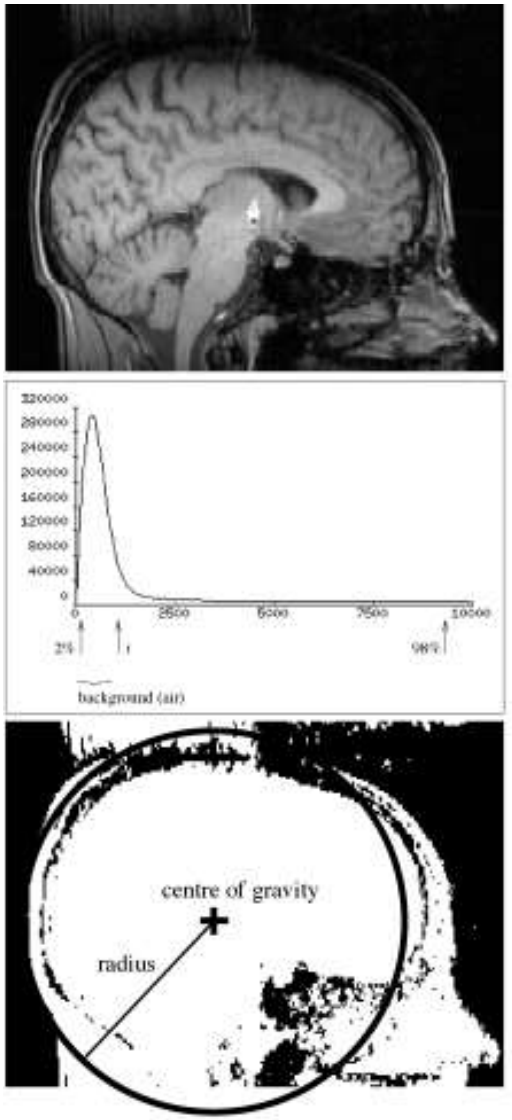
Image from [Smith 2002]

Initialization: Finding Thresholds

- Based on histogram, we find I_2 and I_{98}:
  - 2% of intensities are below I_2 or above I_{98}
- Preliminary global threshold

$$I_{\theta_g} = I_2 + 0.1(I_{98} - I_2)$$

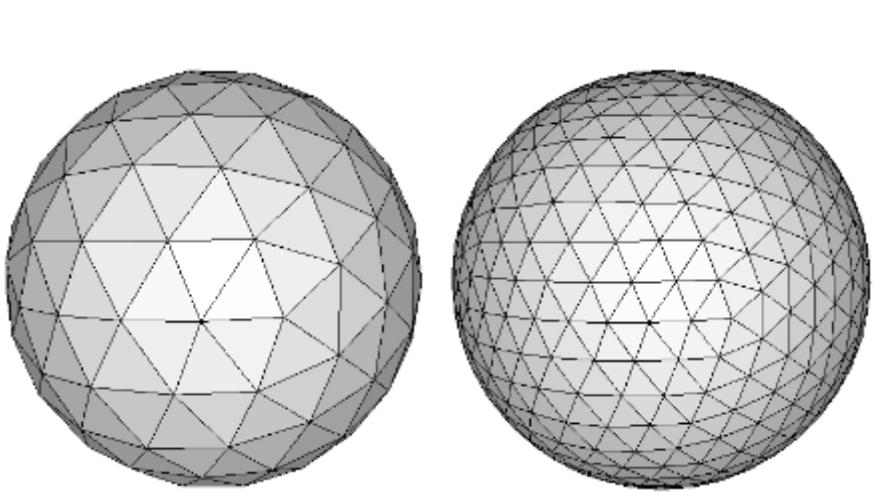
- Find center of gravity (COG) of above-threshold voxels
  - Create sphere around COG whose volume equals the total volume of above-threshold voxels
  - Find median intensity I_m within that sphere

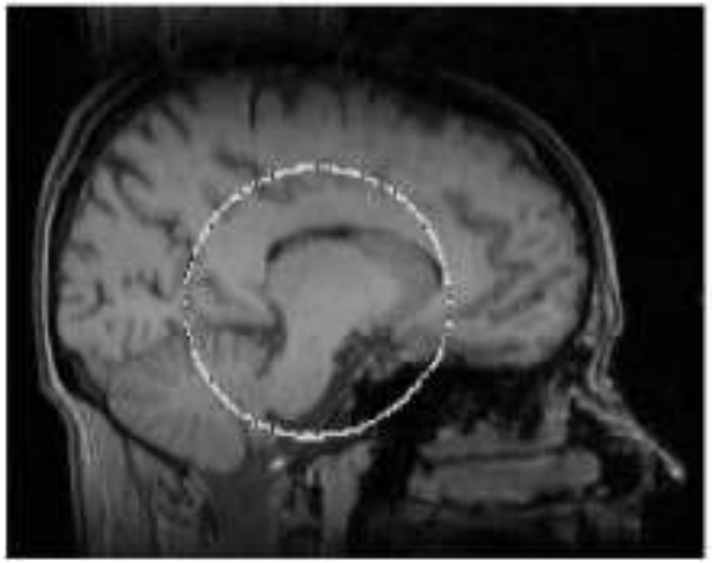


Images taken from [Smith 2002]

Initializing the Mesh

- Evolving surface represented by a triangular mesh
  - Initialized to tessellation of a sphere with half of the previously determined radius
    - *Reason:* Easier to stop pushing outwards once we leave the brain, especially when trying to capture deep sulci

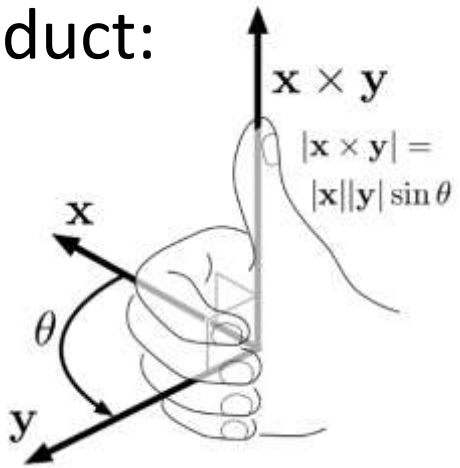


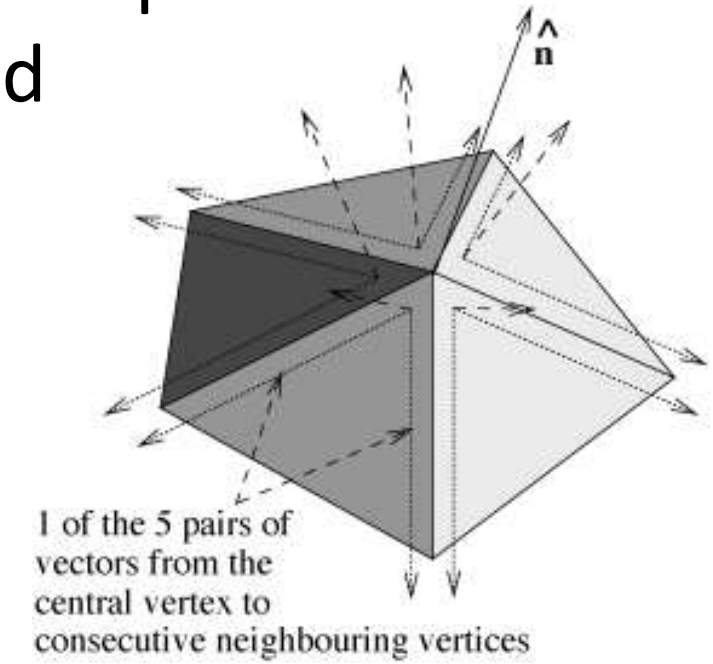


Middle image from [Smith 2002], right image from [Dale et al. 1999]

Moving Towards the Brain Surface

- **Goal:** Move outwards with force f_D in normal direction $\mathbf{n}$ while we are within the brain
- **Surface Normal n** estimated by adding the cross products of vectors to all pairs of neighboring vertices and normalizing the result
  - Amounts to area-weighted average of normals
  - **Reminder** about the cross product:

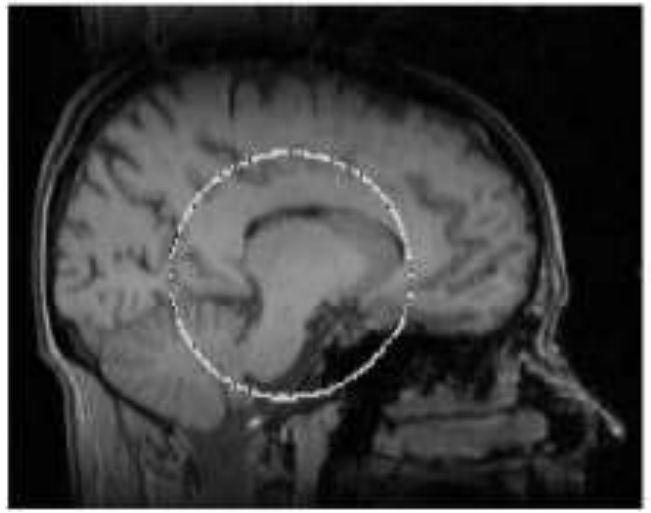




Triangle image taken from [Smith 2002]

Computing a Local Threshold I_{θ_l}

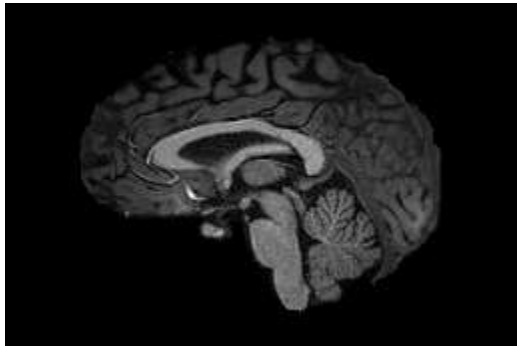
A **local threshold** I_{θ_l} defines the intensity at which we assume that we have left the brain



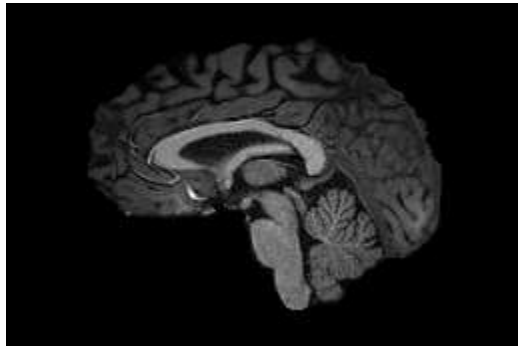
- Definition in terms of maximum intensity in a local neighborhood $I_{\max}$ accounts for inhomogeneous intensities:

$$I_{\theta_l} = I_2 + b(I_{\max} - I_2)$$

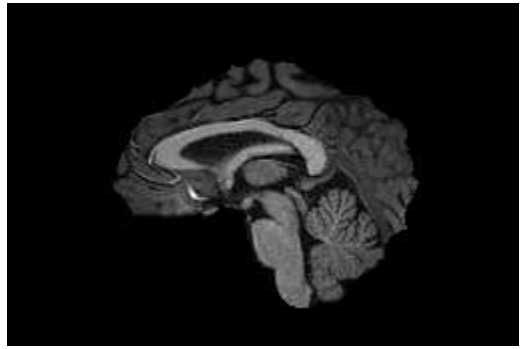
  - $b \epsilon [0,1]$ is the main parameter of the algorithm
  - Smaller *b* leads to more generous segmentations



b=0.25



b=0.5



b=0.75

Speed for Moving Towards Brain Surface

- Local minimum and maximum intensities $I_{\min}$ / $I_{\max}$ are found via an inward search (along -**n**) for a certain distance
  - If $I_{\min}$ is below I_{θ_l}, we already left the brain and need to backtrack
  - Otherwise, we should continue to push outwards, the faster the further $I_{\min}$ is still above I_{θ_l}
- **Speed** for moving along $\mathbf{n}$ is computed as

$$f_D = \frac{2\left(I_{\min} - I_{\theta_l}\right)}{I_{\max} - I_2}$$

  - Outward motion ($f_D > 0$) if $I_{\min} > I_{\theta_l}$
  - Inward motion ($f_D < 0$) if $I_{\min} < I_{\theta_l}$

Robust Estimates of I_{min} / I_{max}

- The estimates of I_{min} and I_{max} are defined as

$$I_{\min} = \max\left(I_2, \min\left(I_m, I(0), I(1), \ldots, I(d_1)\right)\right)$$

$$I_{\max} = \min\left(I_m, \max\left(I_{\theta_g}, I(0), I(1), \ldots, I(d_2)\right)\right)$$

  - Different spatial ranges for searching I_{min} and I_{max}
    - d_1=20mm vs. d_2=10mm ("empirical" choice)
- Upper/lower bounds from the global thresholds I_2, I_{θ_g}, and I_m make the algorithm more robust to outliers by limiting the maximum speed f_D

Regularization based on Mean Displacement

- Averaging vectors to all neighboring vertices produces a **mean displacement s**; split into:
  - Normal component (relates to curvature):

$$\mathbf{s_n} = (\mathbf{s} \cdot \mathbf{n})\mathbf{n}$$

  - Tangential component (relates to uniformity of sampling):

$$\mathbf{s_t} = \mathbf{s} - \mathbf{s_n}$$

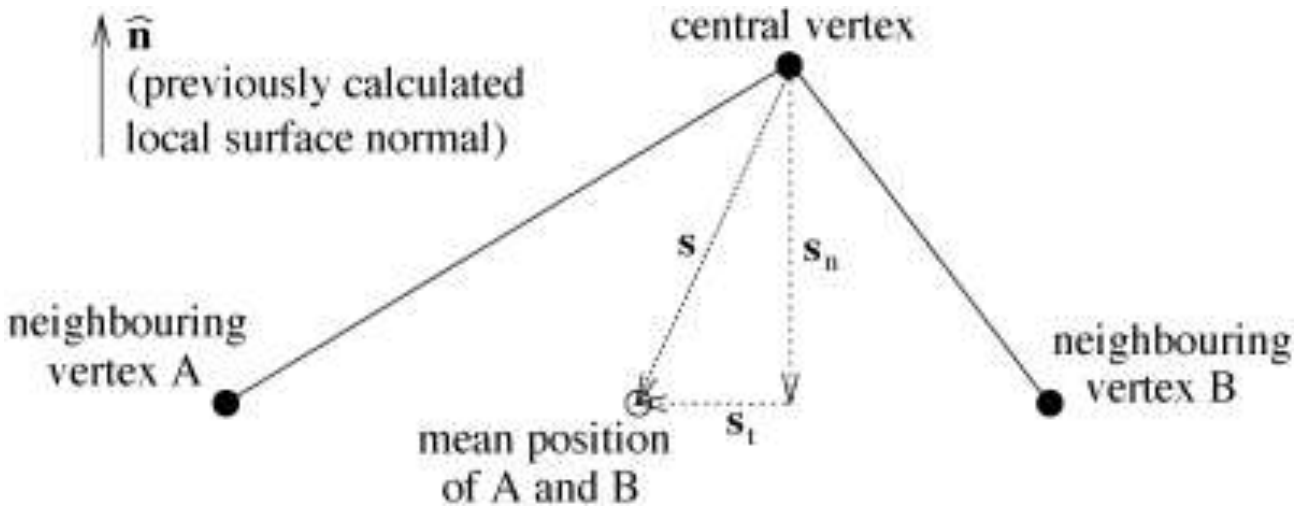


Image taken from [Smith 2002]

Vertex Spacing and Surface Smoothness

1. **Goal:** We would like to maintain a **uniform distribution of vertices** along the surface

   Rule: Move vertex by $\frac{1}{2}\mathbf{s_t}$ (factor $\frac{1}{2}$: stability)

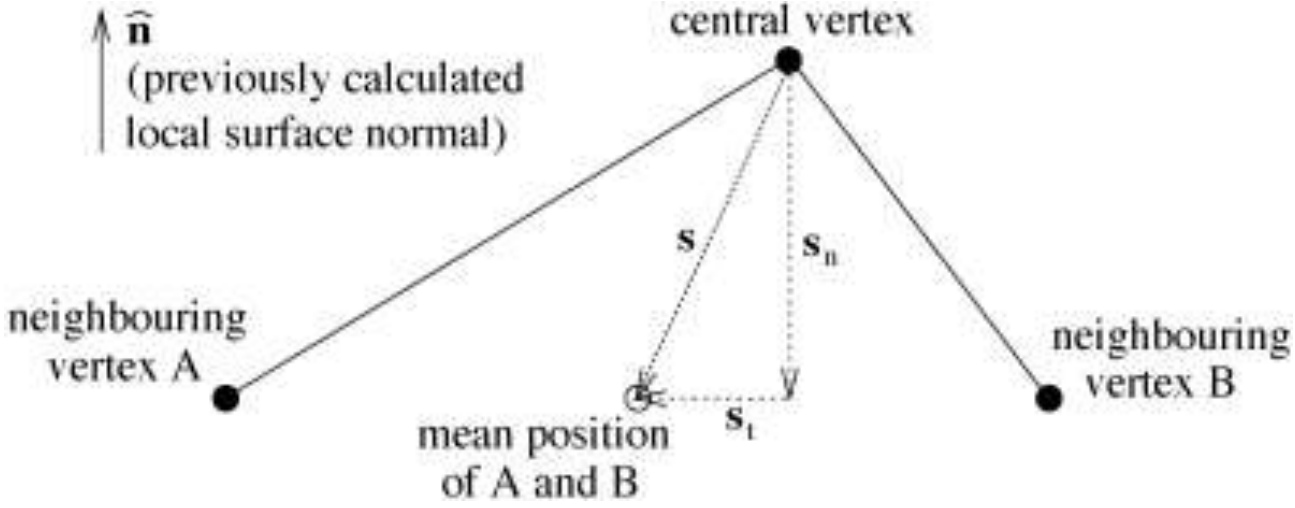
2. **Goal:** We would like to maintain a **smooth surface**

   Idea: Move vertex by $f_S \mathbf{s_n}$

   Improvement: Make f_S curvature adaptive

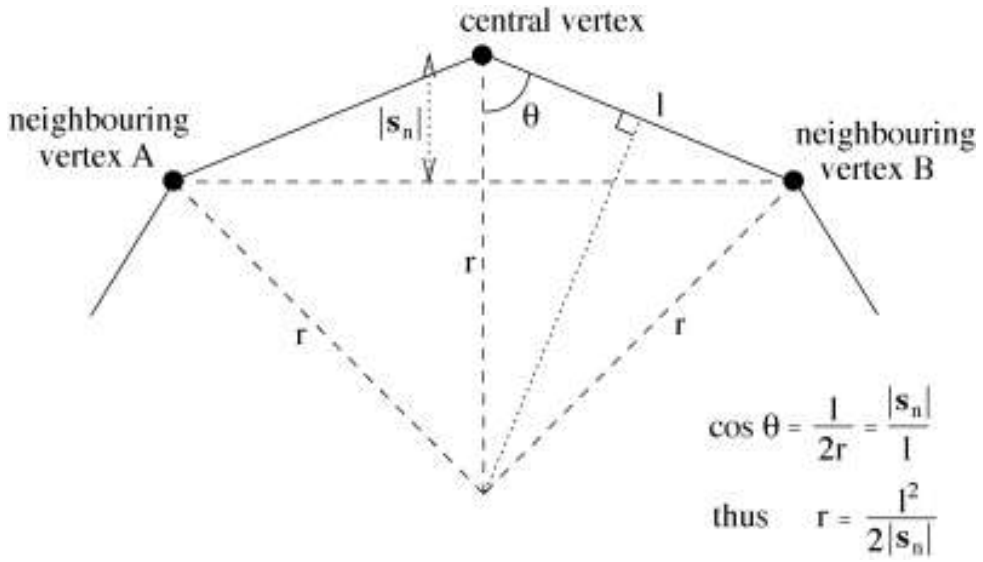
   – eliminate excessive curvature

   – permit moderate curvature, required to model the brain



Computing Local Curvature

- **Radius of curvature** *r:* Radius of a sphere that has the same curvature $\kappa = 1/r$ as the local surface
- To derive estimate of *r,* assume all neighbors have the same tangential and normal distance:

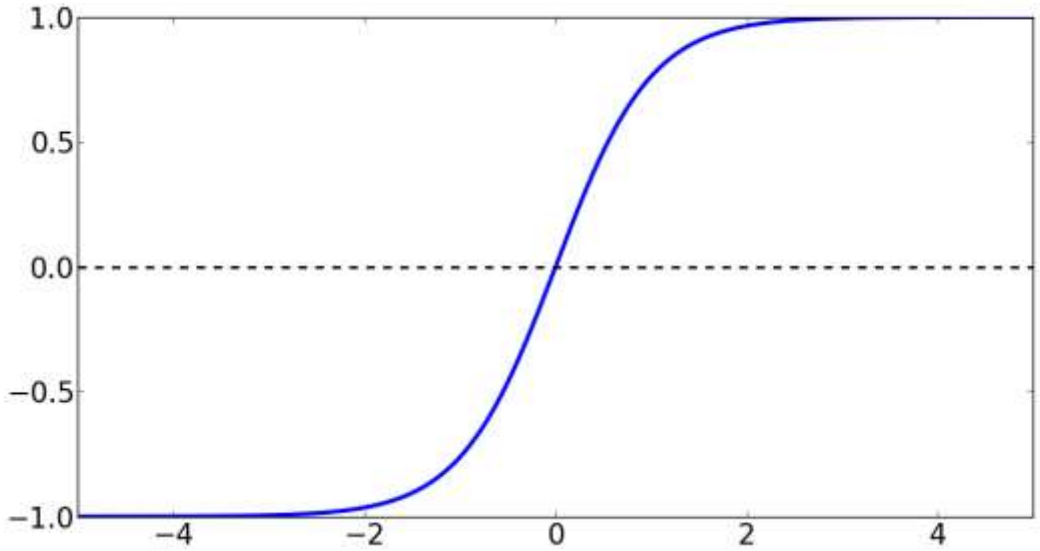


- In practice, simply set *l* to average distance between neighbors across the whole surface

Image taken from [Smith 2002]

Computing f_S from Local Curvature

- f_S should be
  - Close to zero for small curvature $\kappa < 1/r_{\text{max}}$
  - Close to one for large curvature $\kappa > 1/r_{\text{min}}$
- Make use of $\tanh(x) = \frac{1-e^{-2x}}{1+e^{-2x}}$
  - For $x < -3$ or $x > 3$, close to -1 or 1, respectively



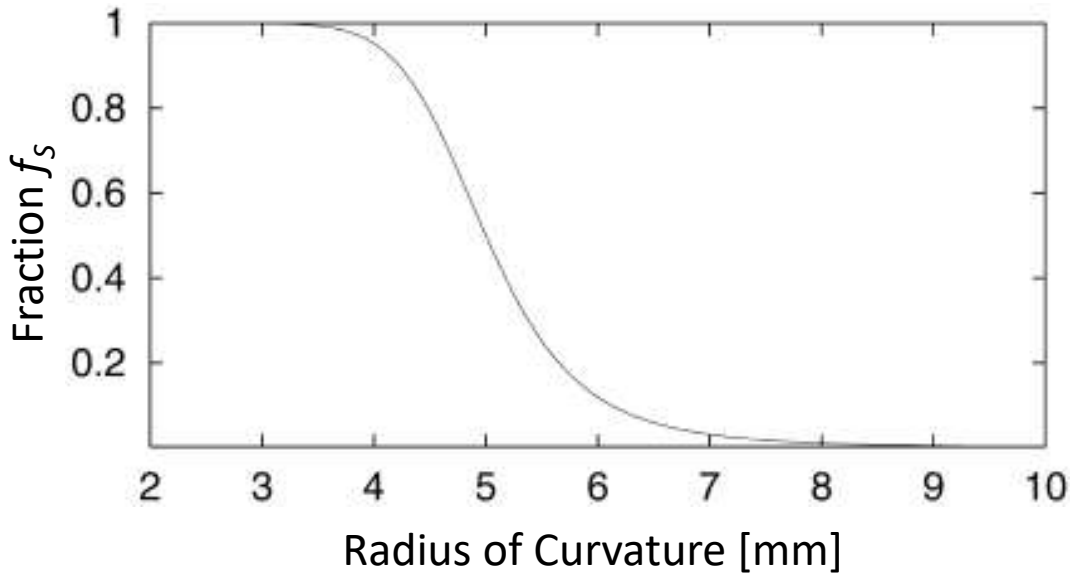
f_S from Local Curvature: Final Equation

- It's easy to convince yourself that defining f_S as follows leads to the desired behavior:

$$f_S = \frac{1 + \tanh\left(\left(\frac{1}{r} - E\right)F\right)}{2}$$

$$\mathrm{E} = (1/\mathrm{r}_{\min} + 1/\mathrm{r}_{\max})/2$$

$$\mathrm{F} = 6/(1/\mathrm{r}_{\min} - 1/\mathrm{r}_{\max})$$



Plot for
r_{min}=3.33 mm
r_{max}=10 mm

Image taken from [Smith 2002]

BET: Example Evolution

- **Surface evolution** after putting together vertex spacing, surface smoothness, and data terms

$$\mathbf{u} = \frac{1}{2}\mathbf{s_t} + f_S\mathbf{s_n} + 0.05 l f_D \mathbf{n}$$

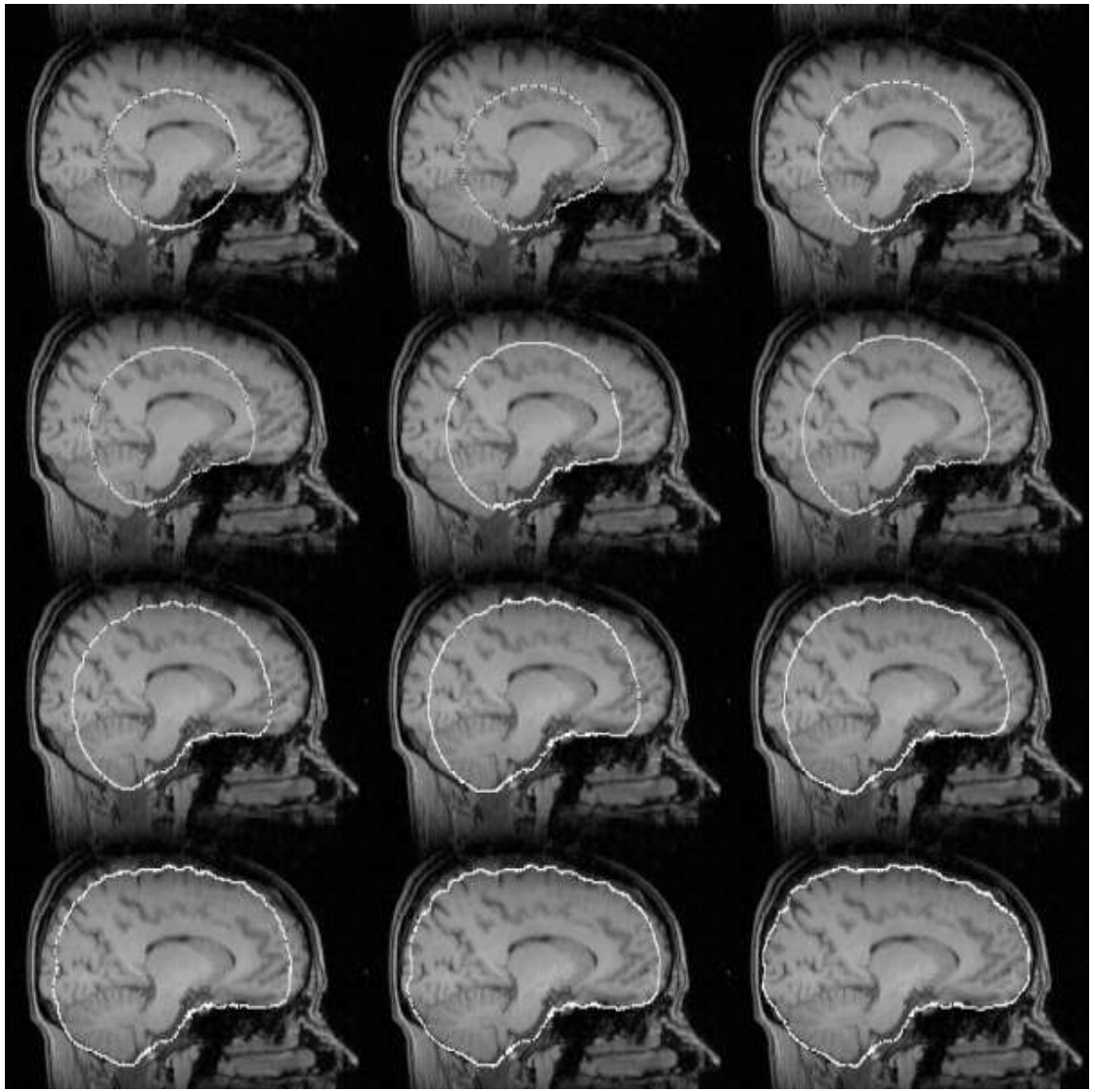


Image taken from [Smith 2002]

Lessons From This Real-World Example

Lesson from BET's approach:

- **User-defined parameters** should be *few*, *intuitive*, and the result should be relatively *robust* to them
- Hard-code reasonable defaults or robustly set parameters **automatically** whenever possible
  - Discard outliers (e.g. last 2% of the distribution)
  - Use median rather than mean, enforce bounds
  - Define quantities that are invariant to global contrast, image voxel size, mesh density etc.

Summary: Deformable Contours

- Deformable contours represent the **boundary** of the segmented object
  - Iteratively deform an initial shape (“active contour”)
  - Typically combine a data term with a regularizer (e.g., smoothness)
  - Driven by ad-hoc forces (*example*: BET) or derived from an energy functional via calculus of variations (outside our scope)
- Remaining **challenges**:
  - Accounting for topological changes
  - Adjusting sampling density / distribution

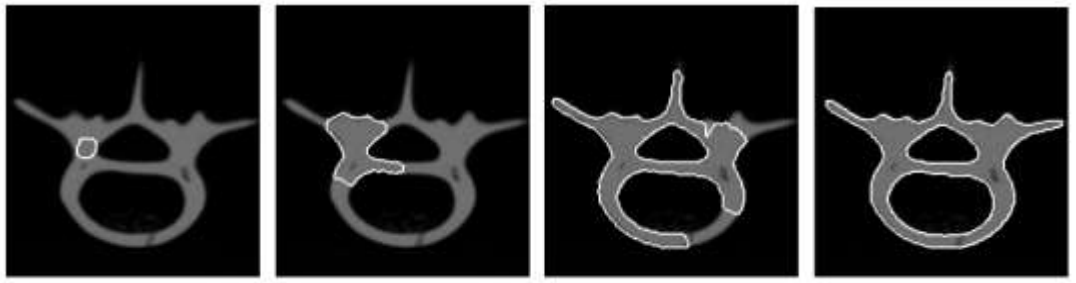


Image: [McInerney/Terzopoulos 1999]

5.3 Segmentation via Clustering

Clustering: Introduction

- Clustering is used not only for image segmentation, but more general data analysis
  - Look for meaningful structures in the data, based on similarity alone: Group similar data together, keep dissimilar data apart
  - "Unsupervised classification"

- We will cover two widely used methods that serve as a foundation for segmentation:
  1. *K*-means / *K*-medoids
  2. Gaussian Mixture Models

K-means for Image Segmentation

Strategy: Represent each pixel as a point in a feature space, apply *K*-means to the result

- Simple example: RGB color channels
- Can be combined with XY(Z) image position

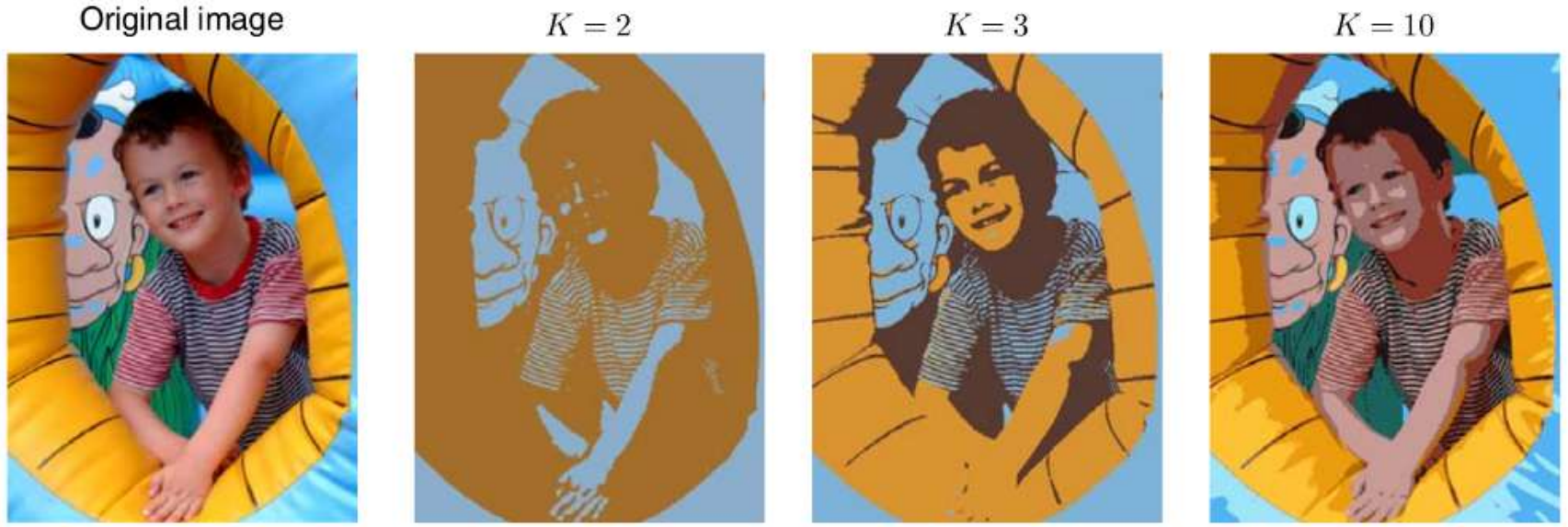
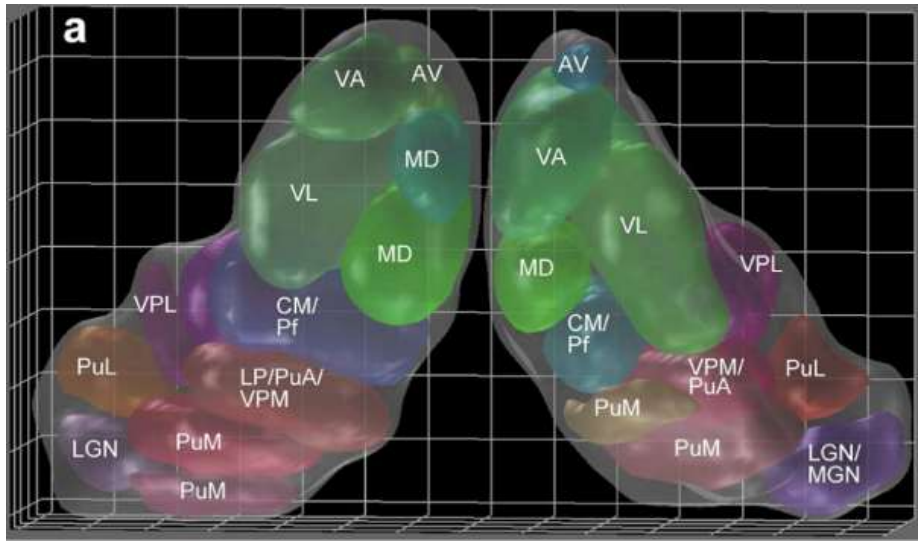
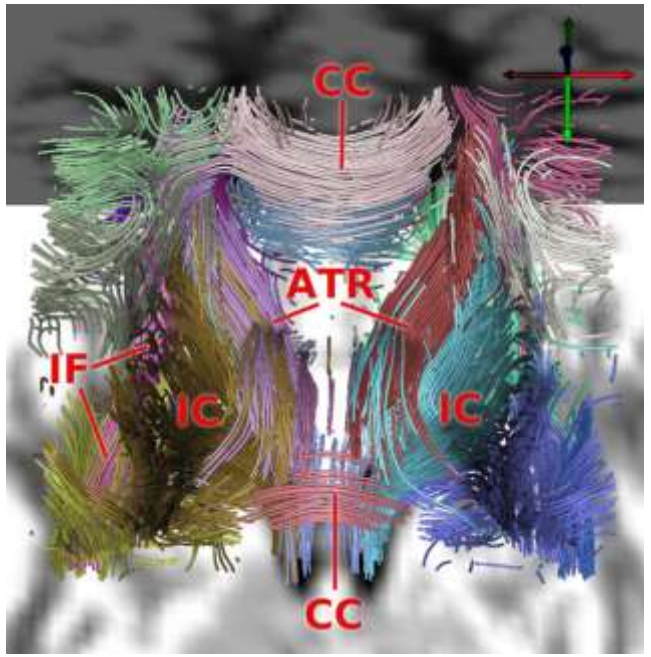


Image from [Bishop 2006]

Clustering in Neuroimaging



Wiegell et al. 2003: *Segmentation of thalamic nuclei*
Clustering as an image segmentation technique



Schultz et al. 2007: *Grouping of reconstructed nerve fiber bundles*
Clustering can be applied to objects that are not elements of a vector space

K-means Distortion Term

- **Goal:** Given data points $\mathbf{v}_i$, find *K* cluster centers $\boldsymbol{\mu}_j$ and an assignment *γ(i)* from each data index *i* to a cluster index *j* so that representing each data point by its cluster center leads to minimal distortion *D*:

$$D = \sum_i \left\| \mathbf{v}_i - \boldsymbol{\mu}_{\gamma(i)} \right\|^2$$

- No exact and efficient algorithm for this problem exists, since optimizing *D* is NP-hard already for *k*=2
  [Dasgupta 2008] [Kanade et al. 2008] [Aloise et al. 2009]

K-means Algorithm

- The *K*-means algorithm optimizes *D* iteratively by alternating updates to assignments and cluster centers:
  1. Initialize cluster centers $\boldsymbol{\mu}_1,\ldots,\boldsymbol{\mu}_k$
     - *Common method:* pick *K* random points, optionally depending on squared distance to previous ones
  2. Update $\gamma(i)$: For each data point, compute the cluster center it is closest to and assign the data point to this cluster
  3. Update $\boldsymbol{\mu}_j$: set cluster centers to mean of data points in cluster
  4. Continue at step 2, until there are no more re-assignments

Example: *K*-means

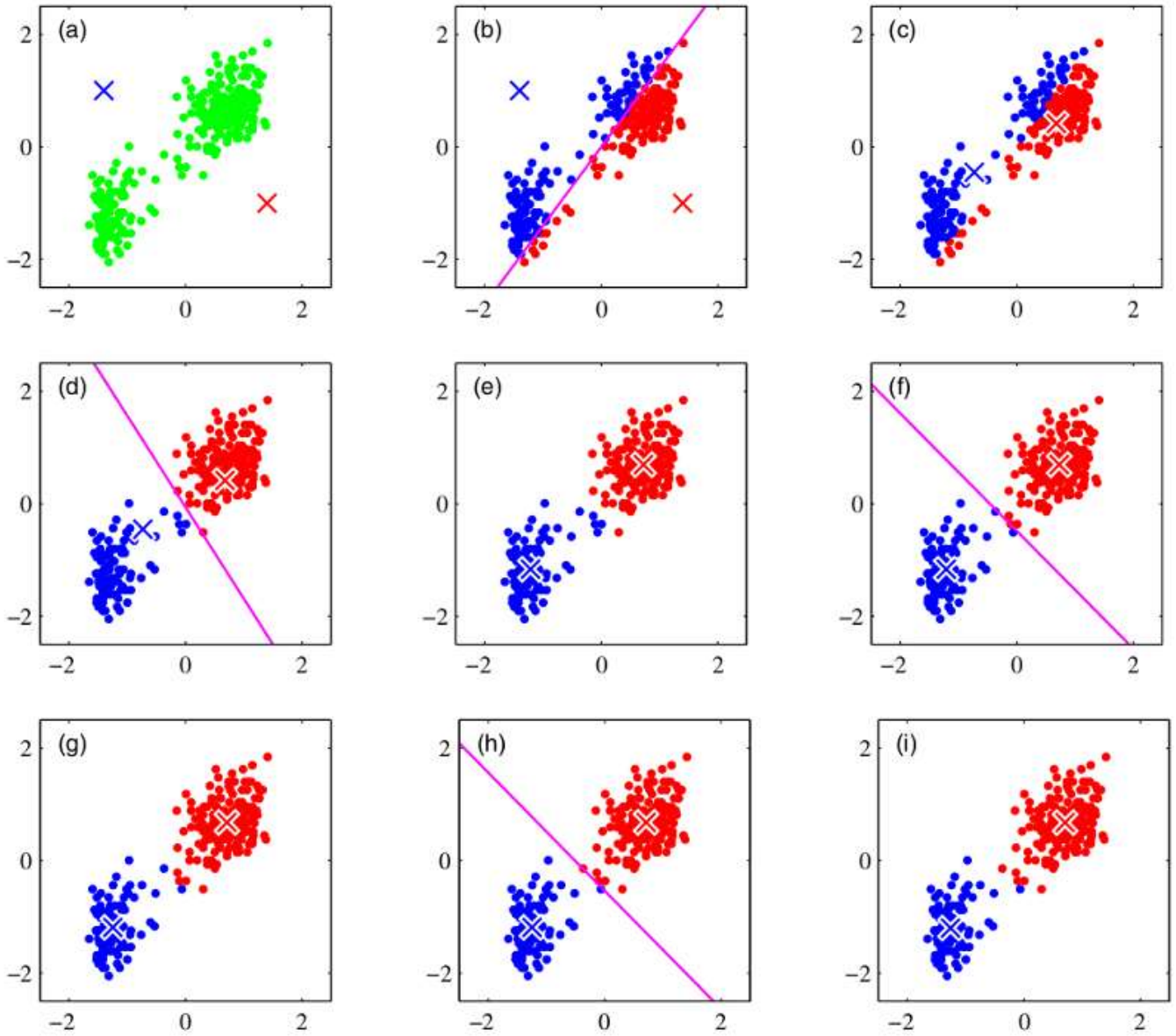
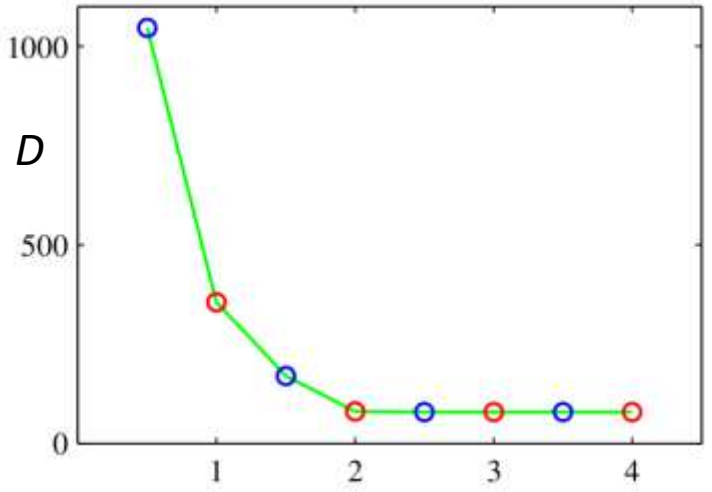


Illustration from [Bishop 2006]

Formal Justification of *K*-means

- *K*-means alternates between optimizing *D* with respect to assignments or cluster positions, which can be done in closed form:



$$D = \sum_i \left\| \mathbf{v}_i - \boldsymbol{\mu}_{\gamma(i)} \right\|^2$$

  - When $\boldsymbol{\mu}_j$ are fixed, all $\gamma(i)$ are independent
  - When cluster members C_j are fixed, setting the derivative of *D* w.r.t. $\boldsymbol{\mu}_j$ to zero gives:

$$2 \sum_{i \in C_j} \left(\mathbf{v}_i - \boldsymbol{\mu}_j \right) = 0 \quad \Leftrightarrow \quad \boldsymbol{\mu}_j = \frac{\sum_{i \in C_j} \mathbf{v}_i}{\left| C_j \right|}$$

- **Note:** *K*-means does *not* guarantee convergence to the global optimum!

Illustration from [Bishop 2006]

Problems with *K*-means Clustering

- With random initialization, you may get sub-optimal clusters

  - Can try different initializations, possibly "smart" ones that take care to spread out the initial cluster centers (e.g., *K*-means++)
  - Keep the result with the lowest distortion measure
- "Collapsing" (empty) clusters
  - Can split one of the other clusters
- Implicit assumption: Clusters are spherical and have same size
- How to pick the number of clusters?

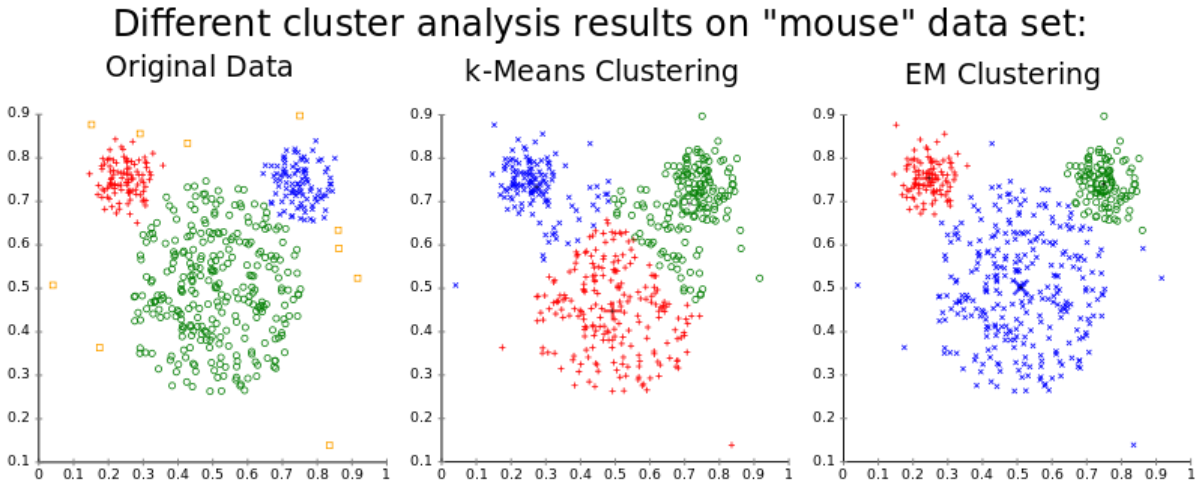
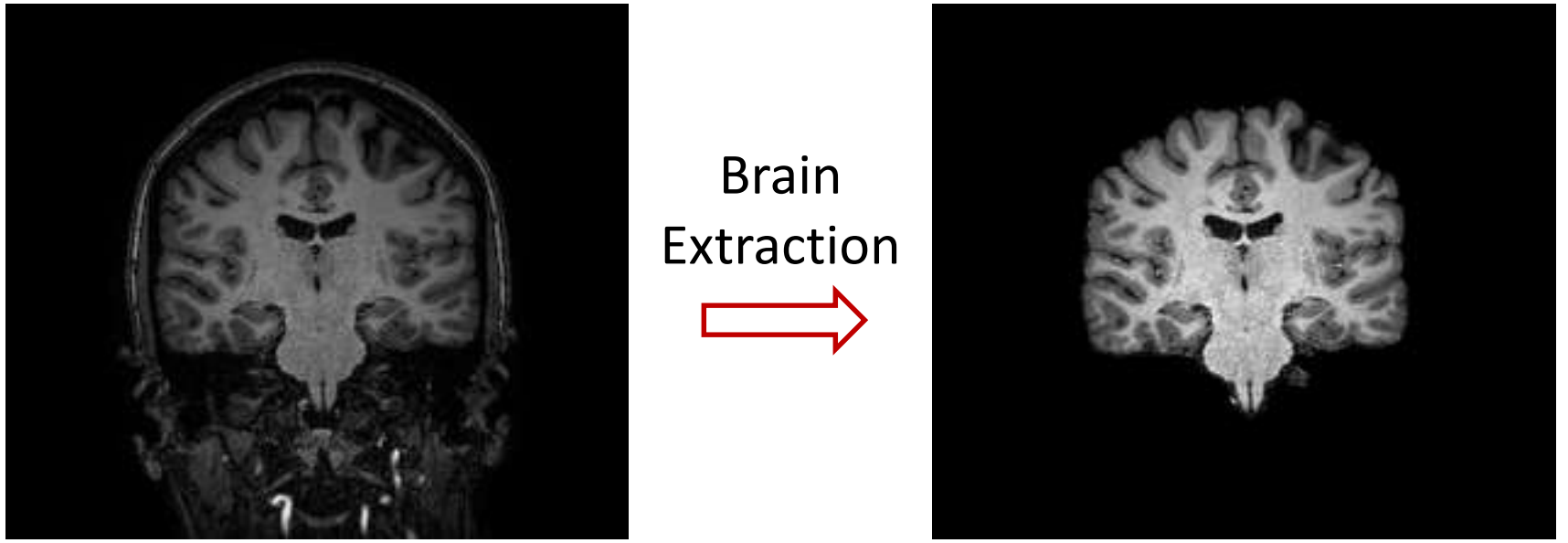


Image Source: Agor153/Chire @ Wikipedia

K-means for Tissue Segmentation

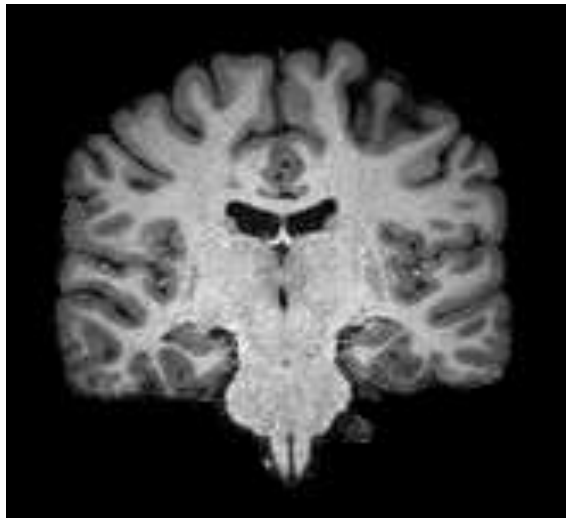
- After brain extraction, we would like to classify gray matter vs. white matter vs. CSF:

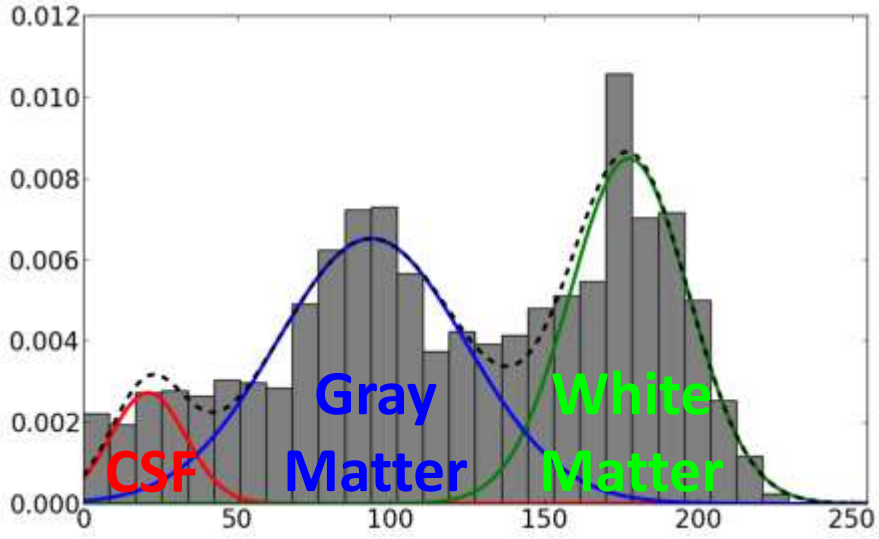


- Could try *k*-means, but:
  - Would like to model partial voluming / uncertainty
  - Would like to allow for different variances per class

GMMs for Segmentation

- CSF, gray matter, and white matter can be distinguished in the histogram:





- Gaussian Mixture Models (GMMs) assume:
  - Each material has normally distributed MR intensities
  - We are observing a mixture of them

Motivation: Gaussian Mixture Models

- **Task**: Given samples from *K* Gaussians, recover
  - their means, covariances, and mixing weights
  - probabilistic sample assignments

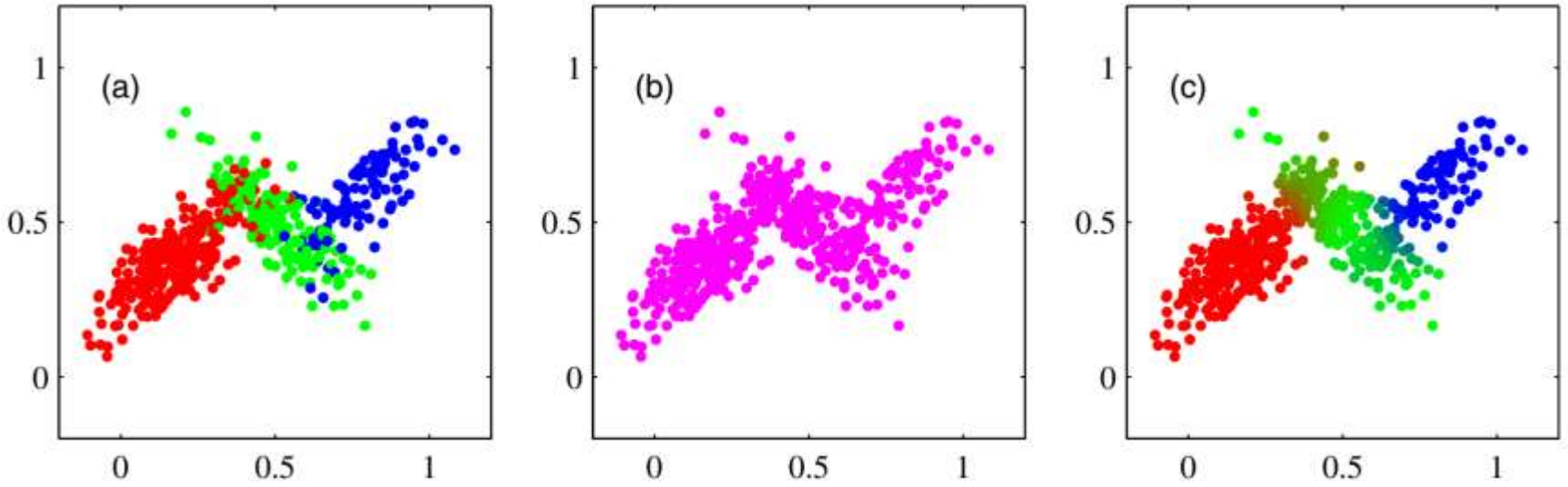


Illustration from [Bishop 2006]

Gaussian Mixture Models

- **Definition:** A Gaussian Mixture Model is given by mixing *K* Gaussian distributions:

$$p(x) = \sum_{k=1}^{K} \pi_k N(x|\mu_k, \sigma_k^2)$$

  - where $N\left(x\middle|\mu_k, \sigma_k^2\right) = \frac{1}{\sqrt{2\pi}\sigma_k} \mathrm{e}^{-\frac{(x-\mu_k)^2}{2\sigma_k^2}}$
    - Derivation will use 1D Gaussians for simplicity
  - π_k are the mixing coefficients
    - Normalization constraint: $\sum_{k=1}^{K} \pi_k = 1$

Membership Vectors and Mixing Coefficients

- For each data point $i \in \{1,2,\ldots,n\}$, introduce a hidden **membership vector** $\mathbf{z}_i \in \{0,1\}^K$ that specifies the cluster k to which i belongs.
  - For each i, $\sum_{k=1}^{K} z_{ik} = 1$
- Results in **mixing coefficient** π_k of cluster k

$$\pi_k = \frac{1}{n}\sum_{i=1}^{n} z_{ik}$$

  - Naturally fulfill the normalization constraint $\sum_{k=1}^{K} \pi_k = 1$

Joint and Marginal Probabilities

- The joint probability distribution of two random variables provides the probability that both variables simultaneously take on specific values
  - *Example:* Joint distribution of the place and type of consumed beer

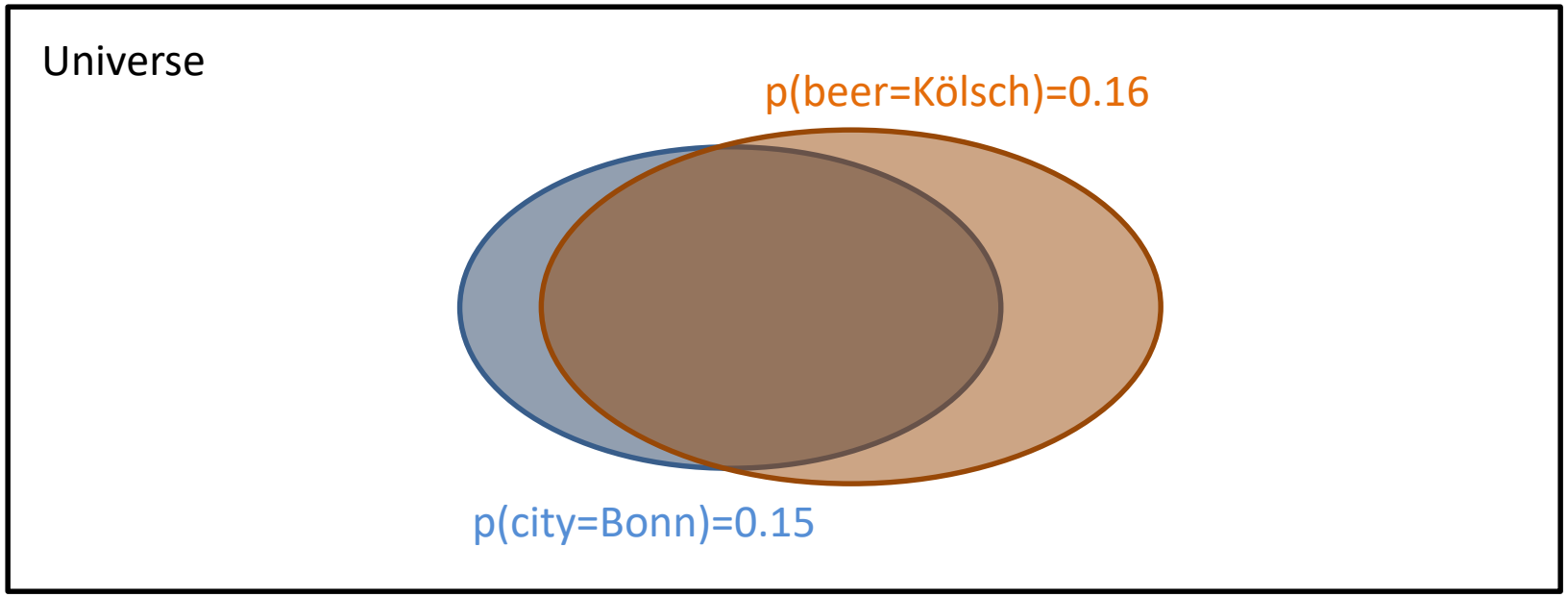
	Bonn	**Hamburg**	
Kölsch	0.12	0.04	0.16
Pils	0.03	0.81	0.84
	0.15	0.85	

- The marginal probability is the probability of one variable taking on a specific value regardless of the other one
  - Computed by summing over all values of the other variable („marginalization“)

Conditional Probabilities

- The **conditional probability** p(B|A) of B (e.g., beer=Kölsch) given A (e.g., city=Bonn) is obtained by re-normalizing their joint probability p(A∩B) w.r.t. p(A):

$$p(B|A) = \frac{p(A \cap B)}{p(A)}$$



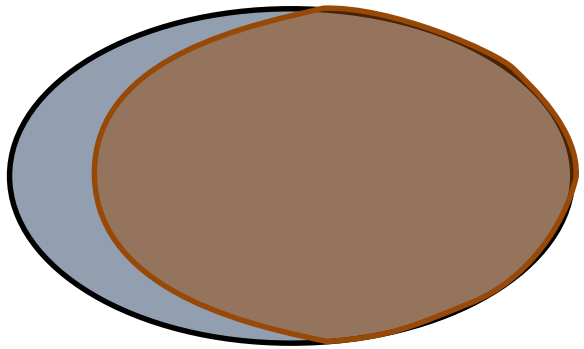
Conditional Probabilities

- The **conditional probability** p(B|A) of B (e.g., beer=Kölsch) given A (e.g., city=Bonn) is obtained by re-normalizing their joint probability p(A∩B) w.r.t. p(A):

$$p(B|A) = \frac{p(A \cap B)}{p(A)}$$

  - „Restrict the universe“ to A

p(beer=Kölsch|city=Bonn)=0.8



“Local Universe”: city=Bonn

Responsibilities

- In GMM-based clustering, we estimate class membership *probabilistically*. For this, we define a **conditional probability of point *i* belonging to cluster *k*** given the data

$$\rho_{ik} \coloneqq p(z_{ik} = 1 | x_i)$$

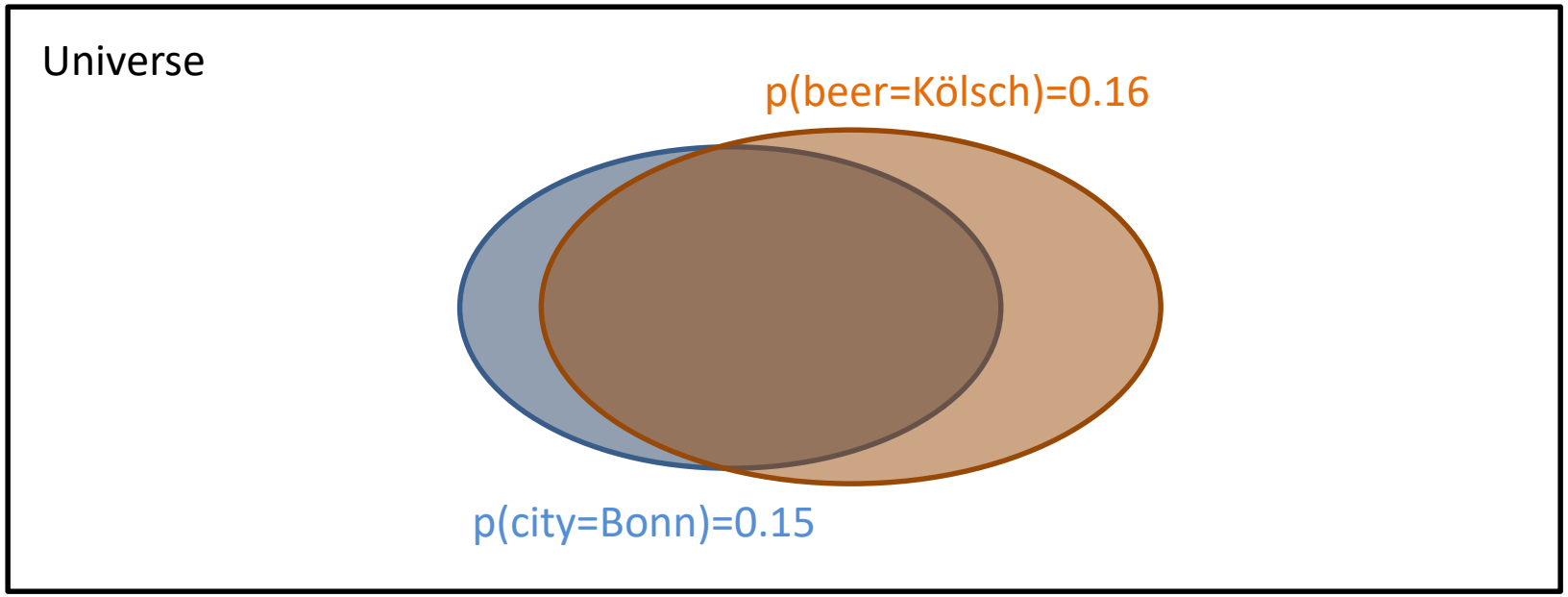
  - "Responsibility" of cluster *k* for data *i*
- **Expectation-Maximization** algorithm alternates two steps:
  - **E-Step:** Given π, μ, σ, estimate ρ
  - **M-Step:** Given ρ, optimize π, μ, σ to fit data

Bayes' Rule

- **Bayes' rule** computes the conditional probability p(A|B) from p(B|A), p(A), and p(B):

$$p(A|B) = \frac{p(B|A)\,p(A)}{p(B)}$$

  - Follows from: $p(A \cap B) = p(B|A)p(A) = p(A|B)p(B)$

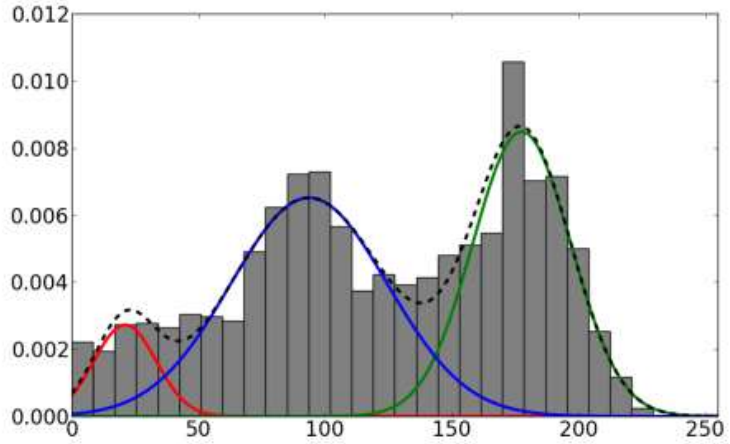


E-Step: Computing Responsibilities

- Given the parameters of a GMM, ρ_{ik} can be computed using Bayes' rule:

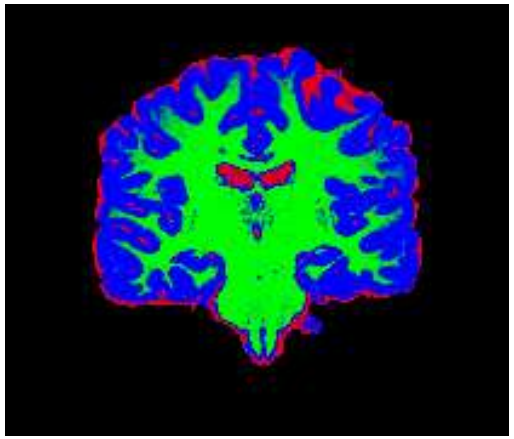
$$\rho_{ik} = p(z_{ik} = 1|x_i) = \frac{p(x_i|z_{ik} = 1)\, p(z_{ik} = 1)}{p(x_i)}$$

$$= \frac{N(x_i|\mu_k, \sigma_k^2)\pi_k}{\sum_{l=1}^{K} \pi_l N(x_i|\mu_l, \sigma_l^2)}$$

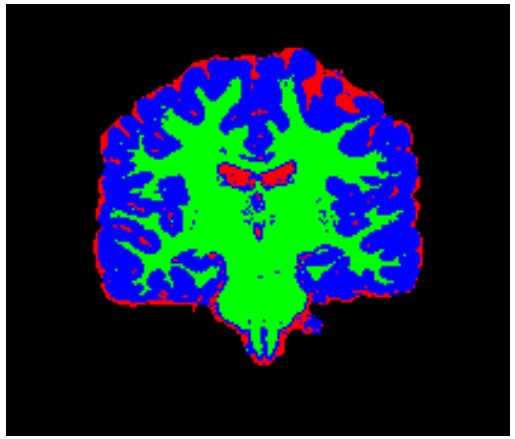


Segmentation Using GMMs

- Given a GMM, the ρ_{ik} provide the desired probability of a voxel containing CSF/GM/WM
  - If hard classification is desired, we can assign voxel *i* to material *k* with maximum ρ_{ik}



Probabilities



Hard Assignments

- **But:** Still need to estimate parameters π, μ, σ

M-Step: Fitting GMM to Data

- We would like the GMM to explain our data as well as possible. Therefore, we will **maximize the likelihood** of our data given the GMM. If we assume independent samples x_i:

$$p(X|\pi, \mu, \sigma) = \prod_{i=1}^{n} \sum_{k=1}^{K} \pi_k N(x_i|\mu_k, \sigma_k^2) \rightarrow \max$$

- For ease of computation, we take the log:

$$\ln p(X|\pi, \mu, \sigma) = \sum_{i=1}^{n} \ln \sum_{k=1}^{K} \pi_k N(x_i|\mu_k, \sigma_k^2)$$

Optimizing μ_k

Recall: $\ln p(X|\pi,\mu,\sigma) = \sum_{i=1}^{n} \ln \sum_{l=1}^{K} \pi_l N(x_i|\mu_l,\sigma_l^2)$

$$N\left(x\middle|\mu_l,\sigma_l^2\right) = \frac{1}{\sqrt{2\pi}\sigma_l} \exp -\frac{(x-\mu_l)^2}{2\sigma_l^2}$$

- In order to optimize μ_k while keeping all other parameters fixed, set derivative w.r.t. μ_k to 0:

$$\frac{\partial \ln p}{\partial \mu_k} = \sum_{i=1}^{n} \underbrace{\frac{\pi_k N(x_i|\mu_k,\sigma_k^2)}{\sum_{l=1}^{K} \pi_l N(x_i|\mu_l,\sigma_l^2)}}_{\rho_{ik}} \frac{2(x_i-\mu_k)}{2\sigma_k^2} = 0$$

- This results in the following update rule:

$$\mu_k = \frac{\sum_{i=1}^{n} \rho_{ik} x_i}{N_k} \quad \text{with} \quad N_k := \sum_{i=1}^{n} \rho_{ik}$$

"effective number of points in cluster k"

Optimizing σ_k

- σ_k can be optimized in a very similar way (left as an exercise)
- Results in the following update rule:

$$\sigma_k^2 = \frac{\sum_{i=1}^{n} \rho_{ik}(x_i - \mu_k)^2}{N_k}$$

Method of Lagrange Multipliers

- While optimizing π_k, we need to maintain the normalization constraint: $\sum_{l=1}^{K} \pi_l = 1$
- This can be achieved using a Lagrange multiplier:
  - A necessary condition for a local maximum of *f(***x***)* subject to an equality constraint *g(***x***)=c* is given by solving the system:

$$\nabla f(\mathbf{x}) = \lambda \nabla g(\mathbf{x}) \text{ and } g(\mathbf{x}) = c$$

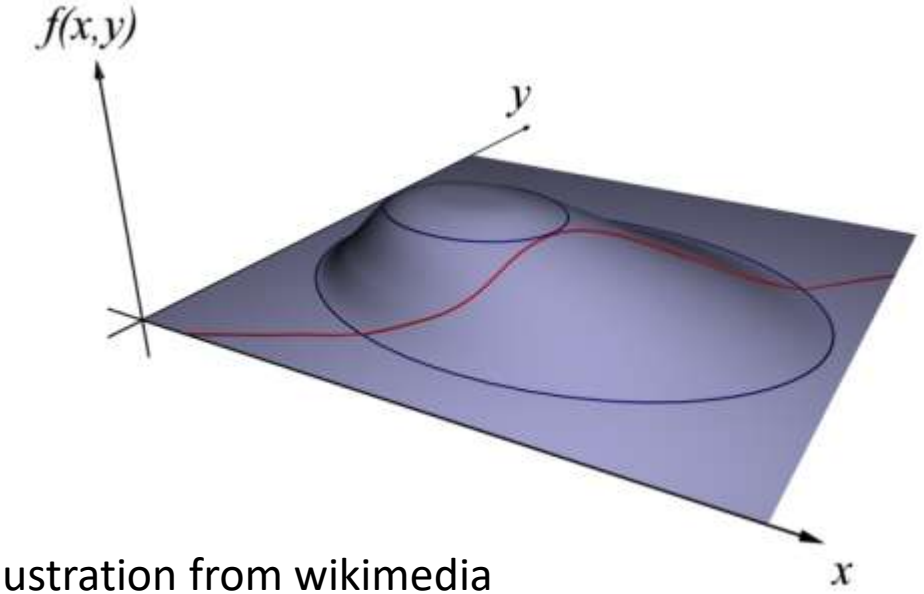
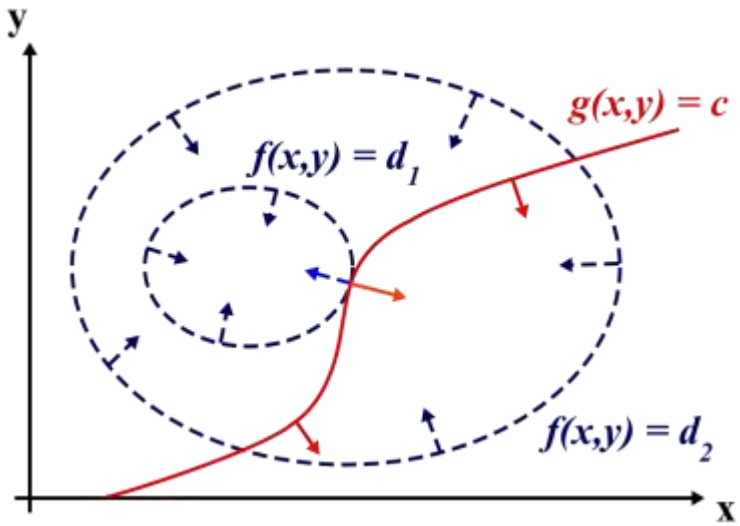


Illustration from wikimedia



Optimizing π_k

Recall: $\ln p(X|\pi,\mu,\sigma) = \sum_{i=1}^{n} \ln \sum_{l=1}^{K} \pi_l N(x_i|\mu_l,\sigma_l^2)$

- Computing $\frac{\partial \ln p}{\partial \pi_k} = \lambda \frac{\partial \sum_{l=1}^{K} \pi_l}{\partial \pi_k}$ gives us

$$\sum_{i=1}^{n} \frac{N(x_i|\mu_k,\sigma_k^2)}{\sum_{l=1}^{K} \pi_l N(x_i|\mu_l,\sigma_l^2)} = \lambda \quad (*)$$

- Multiplying both sides of (*) by π_k and summing over *k* gives $\lambda = n$ (since $\sum_{k=1}^{K} \pi_k = 1$)
- Multiplying (*) by π_k and substituting $\lambda = n$ gives us the update rule $\pi_k = \frac{N_k}{n}$

Overview of the EM Algorithm

1. **Initialize** K means μ_k, variances σ_k^2, and mixing coefficients π_k
2. **E-step:** Update responsibilities

$$\rho_{ik} = \frac{\pi_k N(x_i|\mu_k, \sigma_k^2)}{\sum_{l=1}^{K} \pi_l N(x_i|\mu_l, \sigma_l^2)}$$

3. **M-step:** Update GMM parameters

$$\mu_k = \frac{\sum_{i=1}^{n} \rho_{ik} x_i}{N_k}; \sigma_k^2 = \frac{\sum_{i=1}^{n} \rho_{ik}(x_i - \mu_k)^2}{N_k}; \pi_k = \frac{N_k}{n}$$

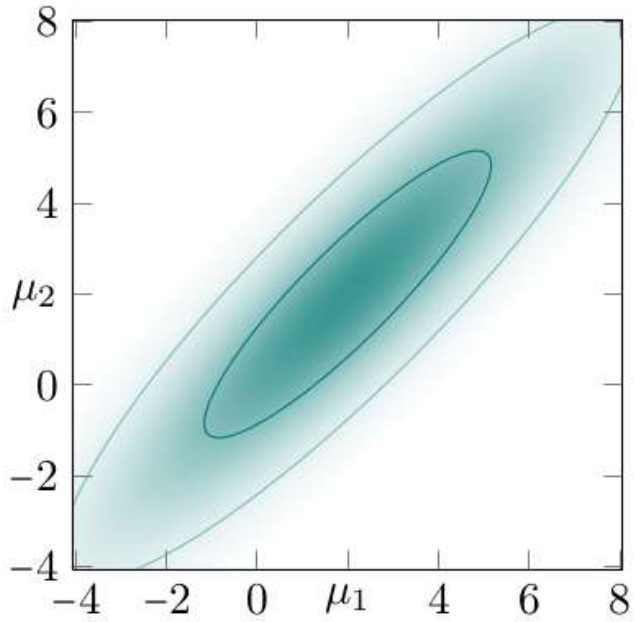
4. **Iterate** E-step and M-step until log likelihood or model parameters have stabilized

Multivariate Normal Distributions

- The 1D Gaussian distribution that we've worked with so far has a natural generalization to *p*-dimensional vector spaces:

$$N(\mathbf{x}|\boldsymbol{\mu},\boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{\frac{p}{2}}|\Sigma|^{\frac{1}{2}}} \mathrm{e}^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^{\mathrm{T}}\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})}$$

  - Mean vector $\boldsymbol{\mu}$
  - Variance-covariance matrix $\boldsymbol{\Sigma}$
    - symmetric positive definite
  - *Examples*: Color images, multiple contrasts in MRI



EM With Vector Input

- The basic EM algorithm remains exactly the same with vector input. The update rules for $\boldsymbol{\mu}_k$ and $\boldsymbol{\Sigma}_k$ in the M-step are:

$$\boldsymbol{\mu}_k = \frac{\sum_{i=1}^{n} \rho_{ik} \mathbf{x}_i}{N_k}$$

$$\boldsymbol{\Sigma}_k = \frac{\sum_{i=1}^{n} \rho_{ik} (\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^{\mathrm{T}}}{N_k}$$

Example: Expectation-Maximization

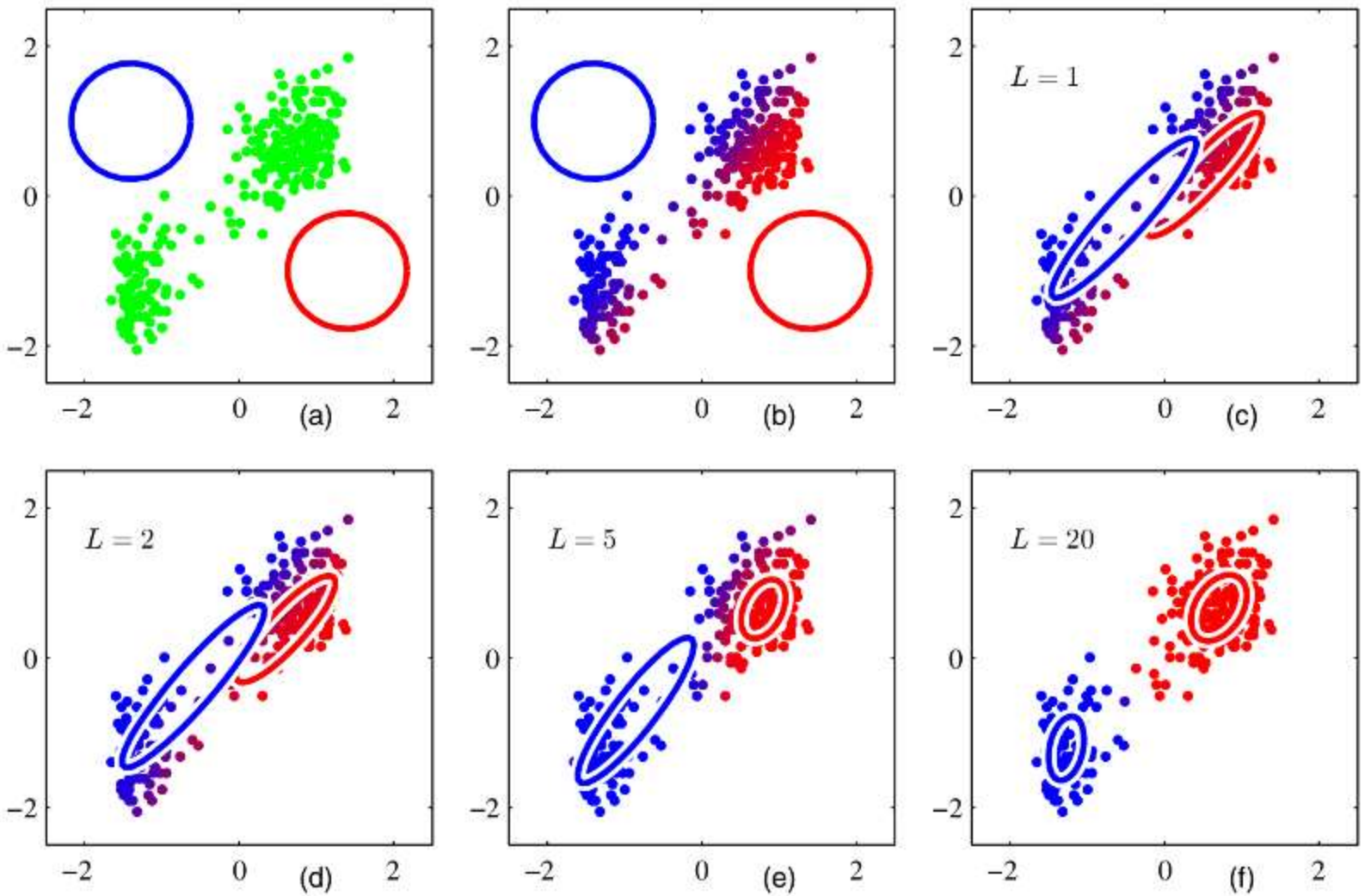


Illustration from [Bishop 2006]

EM vs. *K* means

- **Structural similarity** to *K* means:
  - **E-step:** Cluster assignment
  - **M-step:** Cluster center update
  - Can be made formal:
    - *K* means obtained by setting $\boldsymbol{\Sigma}_k \coloneqq \epsilon \mathbf{I}$ and letting $\epsilon \to 0$ (see Bishop 2006 for details)
- Convergence much slower than *K* means
  - Initializing cluster centers using *K* means often leads to faster convergence

Singularities in the EM Algorithm

- EM converges to a local optimum
  - Like *K* means, it relies on suitable initialization
- Components can collapse to single points
  - Similar singularity as in *K* means
  - *Heuristic:* Detect degenerate clusters and re-initialize their center, e.g., by splitting the largest cluster

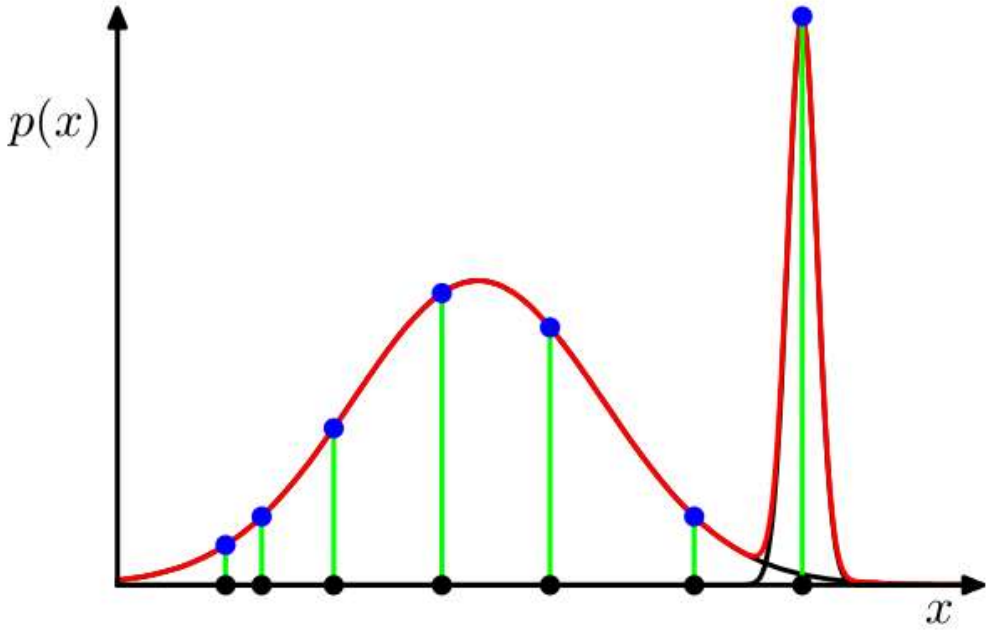


Illustration from [Bishop 2006]

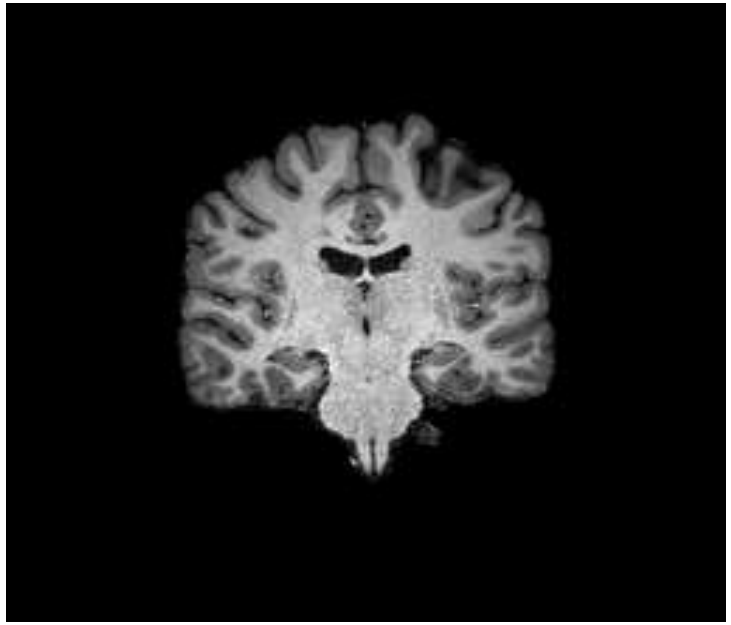
Summary: Gaussian Mixture Models

- **Gaussian Mixture Models** (GMMs) assume that each cluster corresponds to a normal distribution
  - In multivariate case: Ellipsoid cluster shapes
- Probabilistic **cluster membership** ("responsibilities") computed using Bayes' rule
- **Expectation-Maximization (EM) Algorithm** used to fit GMMs
  - Similarity to *K*-means
  - Convergence to local optima
  - Potential singularities

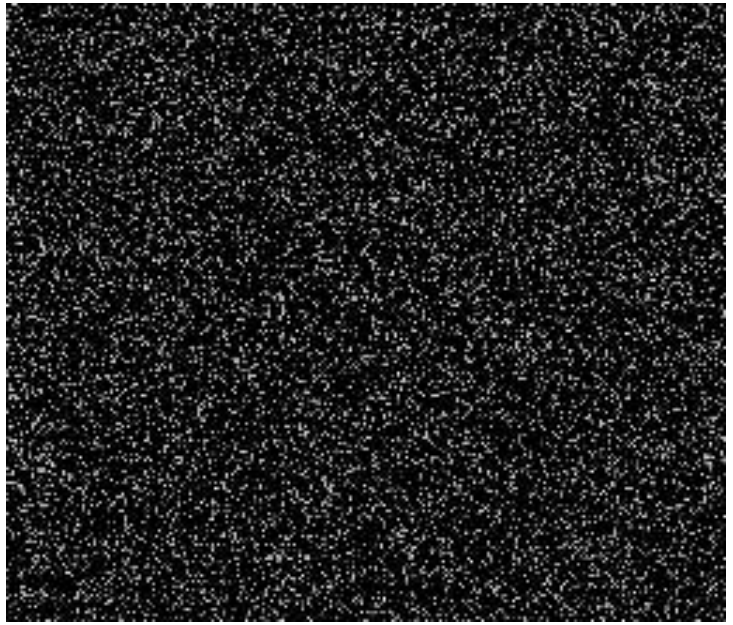
5.4 Markov Random Fields

MRFs: Motivation

- Gaussian Mixture Models only account for the local intensity. Therefore, the same GMM will be estimated for the following two images:



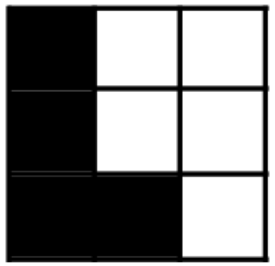
Brain Slice



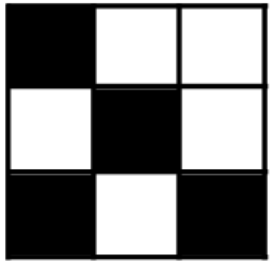
Permuted Brain Slice

Spatial Structure in Images

- **Segmentation vs. Clustering:**
  - Dedicated segmentation algorithms exploit the **spatial structure of the image**
    - *Example of spatial correlation:* It is much more likely that a voxel contains gray matter if all its neighbors contain gray matter
  - Modeling spatial correlations makes the method much more stable for **noisy images**



Likely configuration
High probability

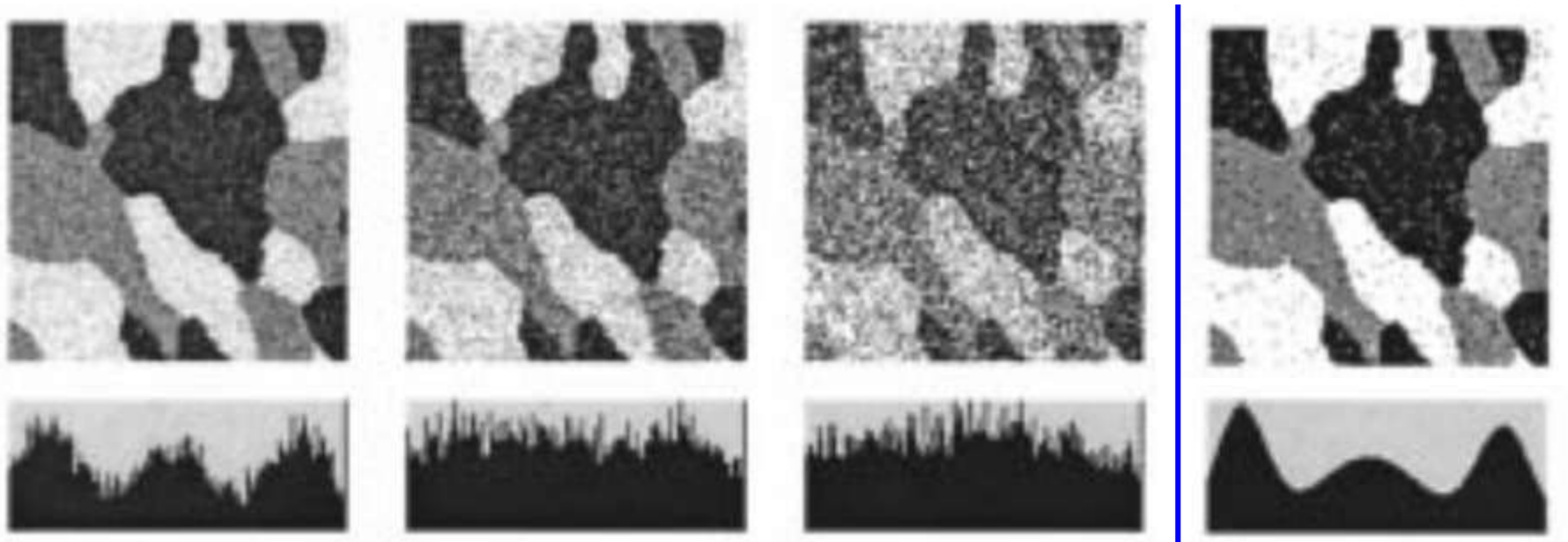


Unlikely configuration
Low probability

Images from Oxford brain fMRI lab

Image Noise

- Since Gaussian Mixture Models do not enforce any regularity in their classification results, they are highly susceptible to noise:



(low noise) (med. noise) (high noise)

Simulated Noisy Data

GMM Result
(low noise)

Images from Zhang et al. 2001

Prior Cluster Likelihood Revisited

- Remember conditional probability of voxel *i* belonging to cluster *k*:

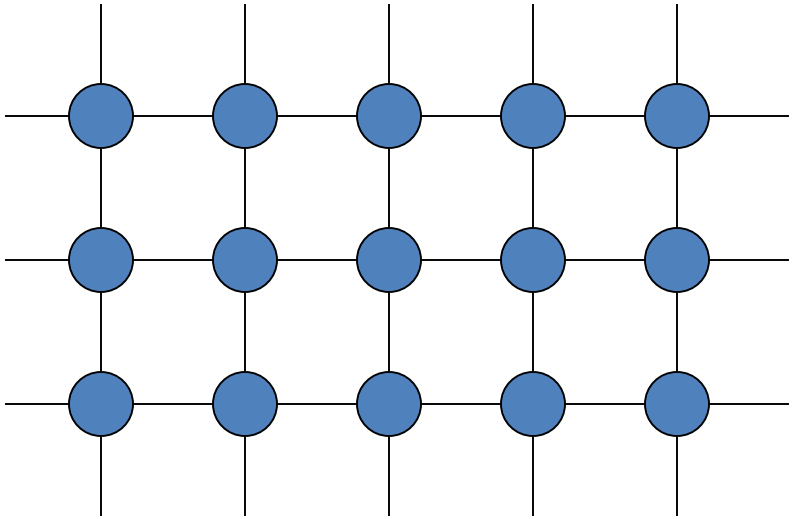
$$p(z_{ik} = 1|x_i) = \frac{p(x_i|z_{ik} = 1)\, p(z_{ik} = 1)}{p(x_i)}$$

- In the Gaussian Mixture Model, $p(z_{ik} = 1) = \pi_k$, i.e., independent from label of the neighbors.
- **Challenge:** We wish to model the spatial dependencies in MR images to better deal with noise. However, introducing long-range dependencies makes the model intractable.

Markov Random Fields

- **Common solution:** Define a neighborhood graph from the pixel grid. Each pixel depends on all others only through its immediate neighbors.
  - Generalizes the Markov chain introduced by Andrey Markov (1856-1922)
  - z_i are no longer independent; we need to consider the *joint distribution* of z_S (*S*=set of all voxels)
  - **But:** z_i independent from all others given its immediate neighbors $NB(i)$:

$$p\left(z_i \middle| z_{S-\{i\}}\right) = p(z_i | z_{NB(i)})$$

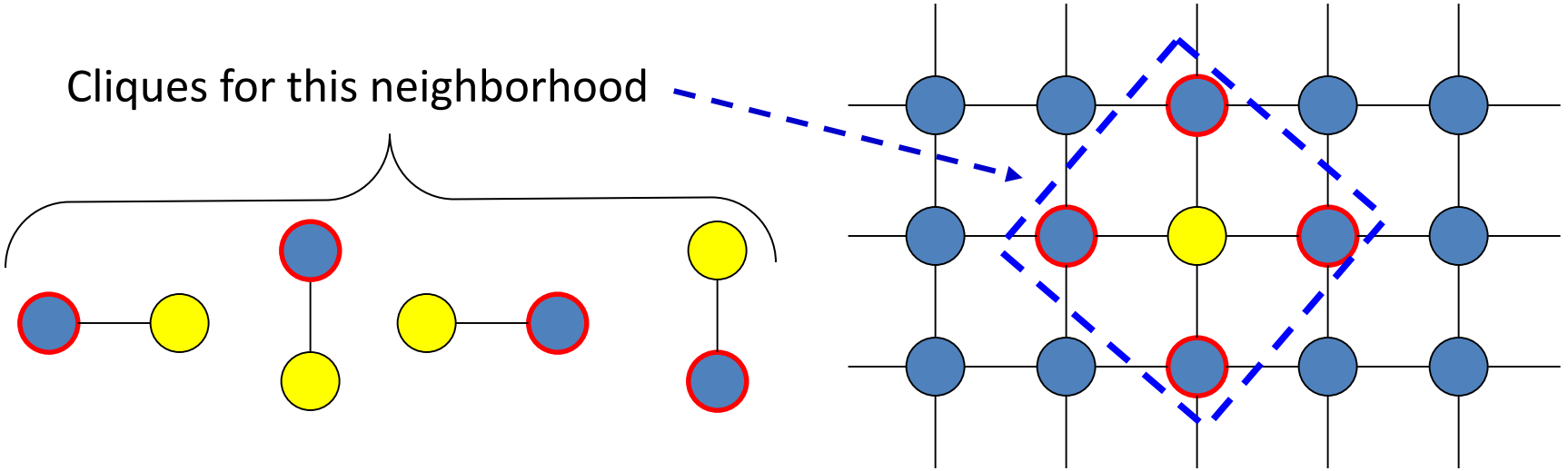


Hammersley-Clifford Theorem

- **Theorem:** If the density of z_S is positive, it factorizes over the cliques *C* of a Markov Random Field:

$$p(z_s) \propto \prod_C p(C) = e^{-\sum_C V_c(z_C)}$$

  - V_C are called "clique potentials"



Generalized Potts Model

- The **Generalized Potts Model** is a widely used potential for cliques consisting of pairs of voxels:

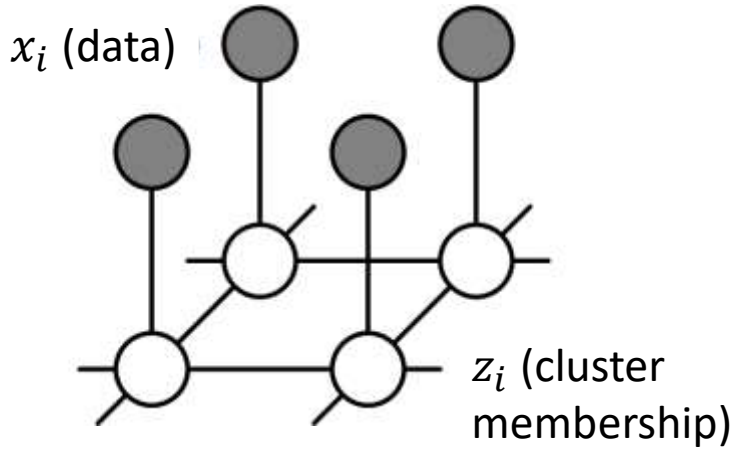
$$V_{ij}(z_i, z_j) = u_{ij}\delta(z_i \neq z_j)$$

u_{ij} = Penalty for discontinuity between voxels *i* and *j;* in the simplest case, the same number β for all pairs

$\delta(z_i \neq z_j) = 1$ if $z_i \neq z_j$, 0 else

Including the Data in the MRF

- So far, we've used MRFs to model the marginal of cluster membership z
- Let's add our data



$$p(z_S|x_S) = \frac{p(x_S|z_S)\,p(z_S)}{p(x_S)}$$

- Leads to **unary potentials** V_i:

$$p(z_s|x) \propto e^{-\sum_i V_i(x_i,z_i) - \sum_i \sum_{j \in NB(i)} V_{ij}(z_i,z_j)}$$

Unary Potentials

- We model the data likelihood with the same Gaussian Model as before:

$$p(x_i|z_i) = \frac{1}{\sqrt{2\pi}\sigma_{z_i}} \mathrm{e}^{-\frac{\left(x_i-\mu_{z_i}\right)^2}{2\sigma_{z_i}^2}}$$

- Re-writing it as a unary potential V_i such that

$$p(x_i|z_i) = e^{-V_i(x_i,z_i)}$$

involves the negative natural logarithm, i.e.,

$$V_i(x_i, z_i) = \ln\left(\sqrt{2\pi}\sigma_{z_i}\right) + \frac{\left(x_i - \mu_{z_i}\right)^2}{2\sigma_{z_i}^2}$$

The HMRF-EM Algorithm

- Modification of the EM Algorithm for Hidden Markov Random Fields:
  - **E-step:**
    1. Compute MRF-MAP estimate (hard labels)
    2. Calculate posterior distribution of labels:

$$\rho_{ik} = p(z_{ik} = 1|x_i) = \frac{p(x_i|z_{ik} = 1)\, p(z_{ik} = 1|N(i))}{p(x_i)}$$

  - **M-step:**
    - Use existing update rules to estimate μ_k and σ_k^2
    - Explicit mixing weights π_k no longer used

MAP Estimation

- The modified EM algorithm requires the label set z_S with **maximum a posteriori probability** (MAP)
  - Compute the most likely (hard) label assignment
  - Consider the *joint* distribution of labels for all voxels
  - This is done by minimizing the **energy**, i.e., the sum of all unary and pairwise potentials:

$$\underbrace{\sum_i V_i(x_i, z_i)}_{\text{Data Term}} + \underbrace{\sum_i \sum_{j \in NB(i)} V_{ij}(z_i, z_j)}_{\text{Smoothness Term}}$$

External Energy | Internal Energy

Iterated Conditional Modes (ICMs)

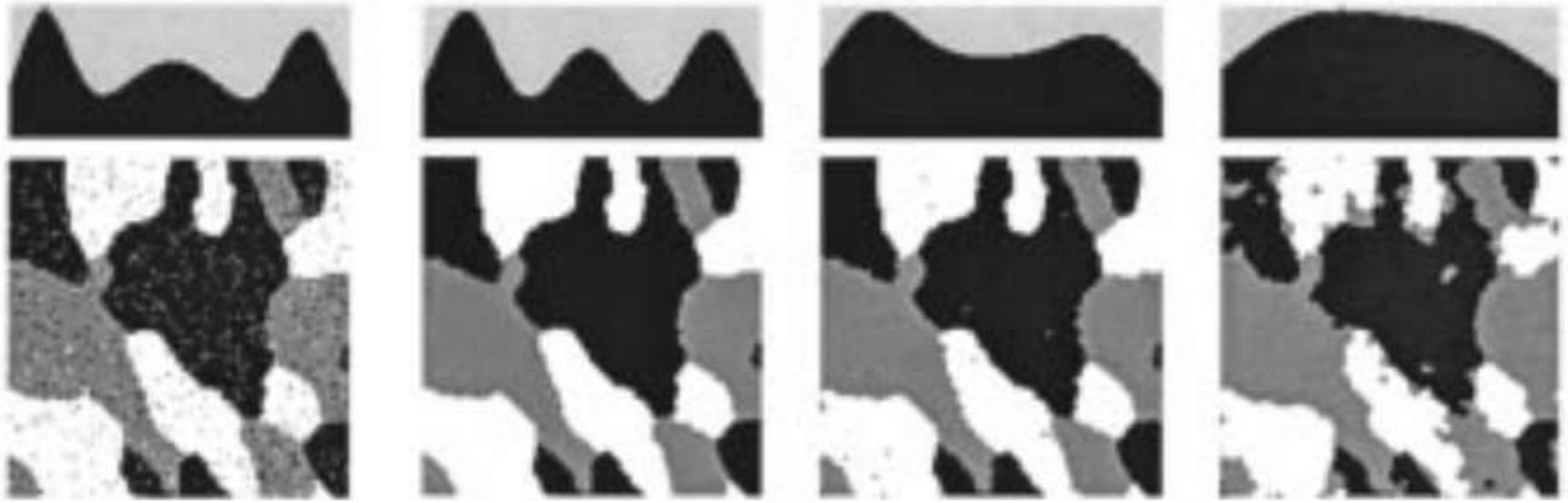
- **Simple approach for MAP estimation:** Iterated Conditional Modes
  1. Start with a reasonable initialization (e.g., without pairwise potentials)
  2. Iterate over all voxels:
     - Condition on all the neighbors (treat them as fixed)
     - Pick the label that minimizes the energy
  3. Repeat step 2 until convergence
- Converges to a local minimum

Iteration 0 Iteration 2 Iteration 4 Iteration 5 Iteration 6

Image from Fischl et al., Neuron 2002

Results of HMRF-EM

- Binary potentials greatly reduce misclassification on simulated example:

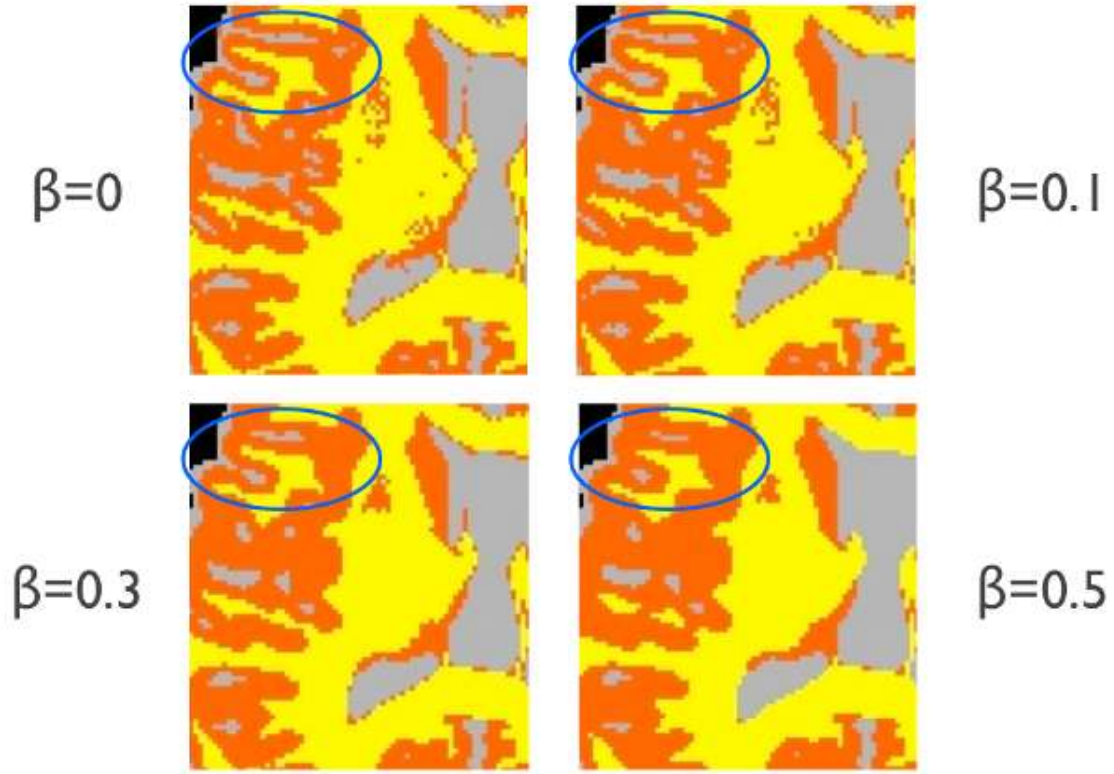


MCR: 5.82%	MCR: 0.12%	MCR: 1.04%	MCR: 8.73%
GMM Result (low noise)	HMRF-EM (low noise)	HMRF-EM (med. noise)	HMRF-EM (high noise)

Images from Zhang et al. 2001

Data vs. Smoothness Tradeoff

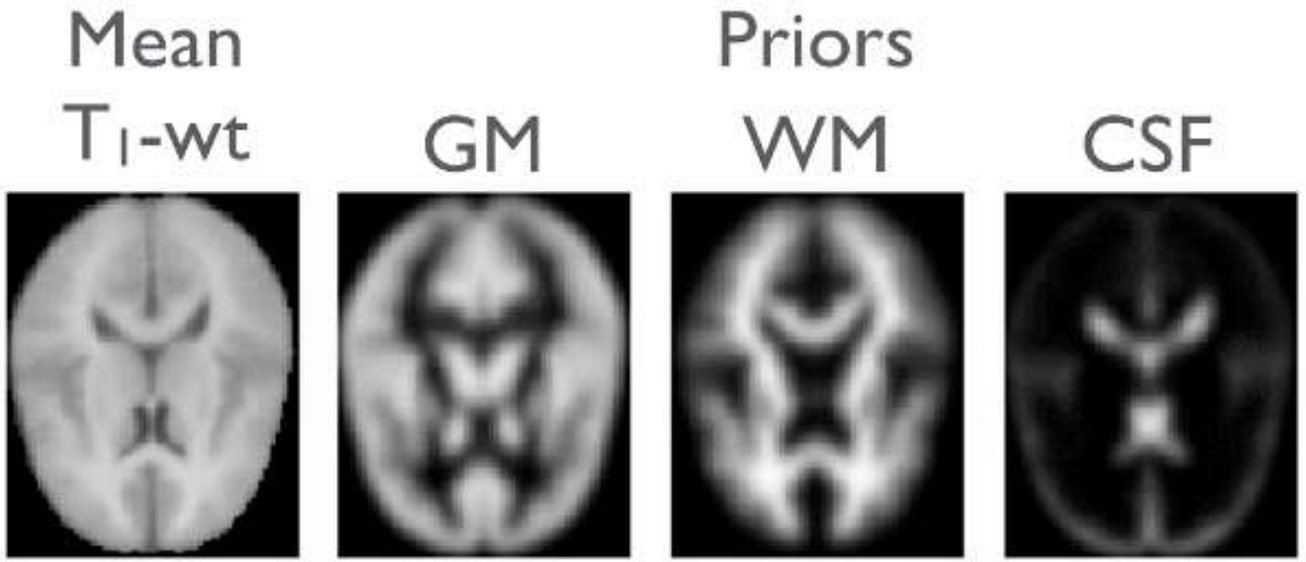
- The result of MRF segmentation depends on the relative weight of the smoothness and data terms
  - In generalized Potts model $V_{ij}(z_i, z_j) = u_{ij}\delta(z_i \neq z_j)$, defined by u_{ij}
  - Results with $u_{ij} = \beta$ for all *i,j:*
  - Detail view:



Images from Oxford brain fMRI lab

Priors from Atlas

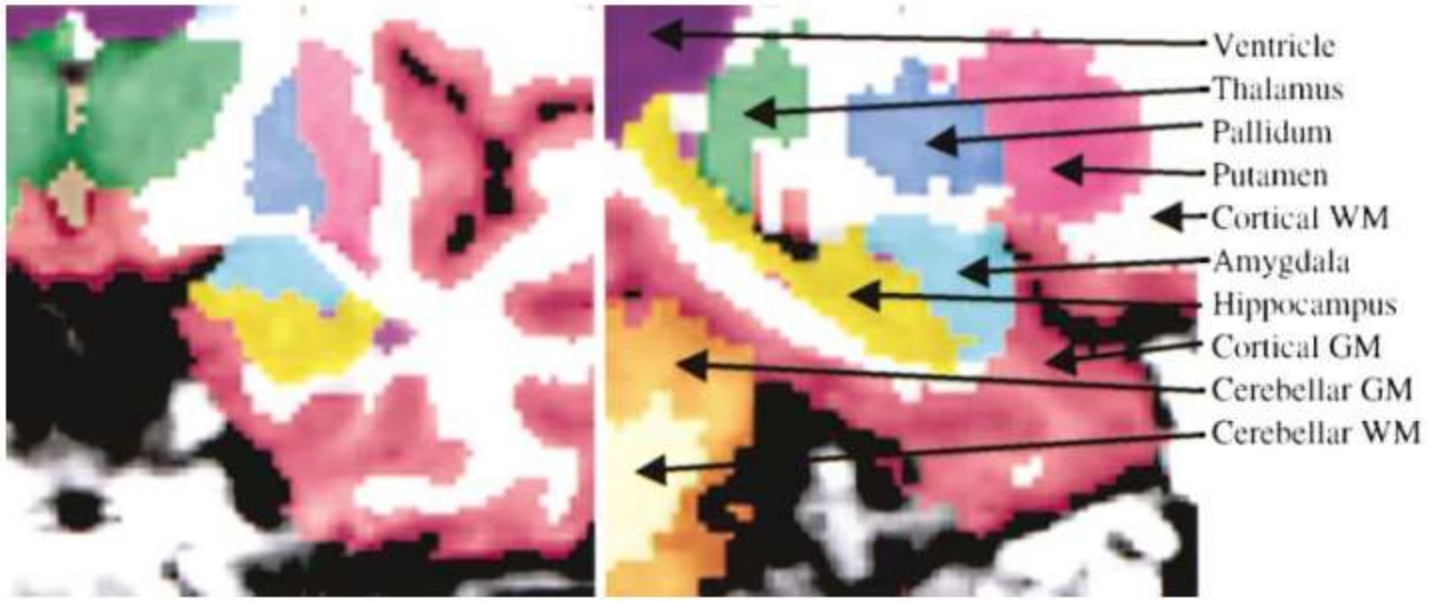
- In addition to smoothness, we can use tissue class probabilities from an atlas as a prior
  - Can help in difficult cases (e.g., strong bias field)
  - Requires registration to the atlas
  - Cave: Misregistration can skew the results



Images from Oxford brain fMRI lab

Real-World Example: Freesurfer

- **Freesurfer** (https://surfer.nmr.mgh.harvard.edu/)
  - Software suite for brain MR image processing and analysis
  - Includes two MRF based algorithms for classifying ≈80 cortical and ≈37 subcortical brain regions
  - Widely used: >5.000/10.000 citations (google scholar, Nov 2025)



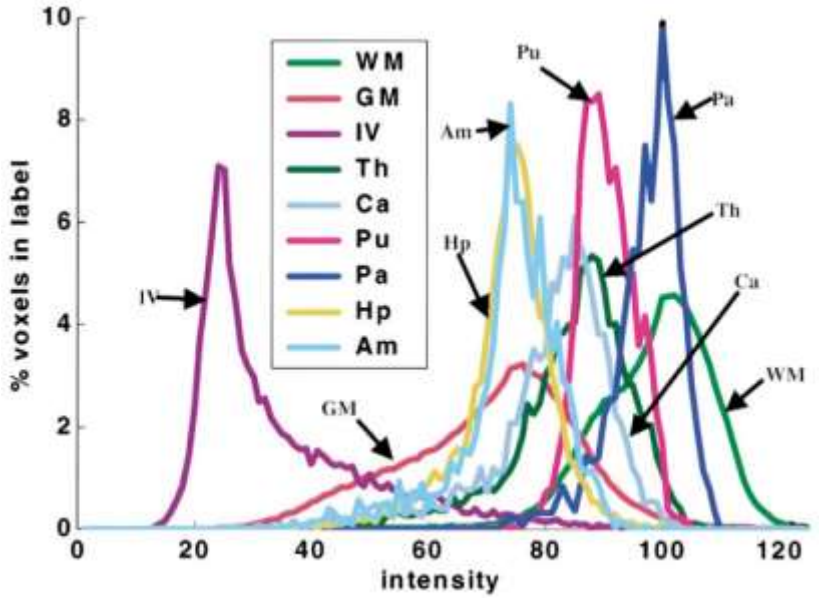
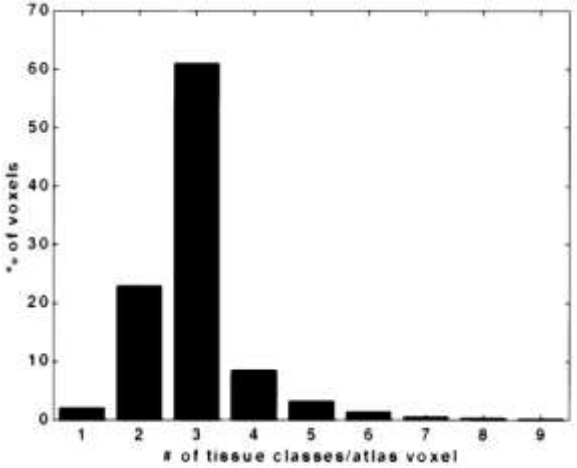


Image from Fischl et al., Neuron 2002

Two Key Ideas in Freesurfer

- Since brain structures cannot be distinguished based on their signal intensities alone, MRFs in Freesurfer are
  - **Nonstationary**: Linear transformation to an atlas (at 4mm resolution) strongly reduces the set of possible labels to an average ≈3
    - Intensity-based registration, restricted to a subset of least ambiguous atlas voxels
  - **Anisotropic**: Pairwise potentials capture directional relationship between labels
    - *Example*: "Amygdala is located anterior and superior to the hippocampus"
    - Accounts for six neighbors in cardinal directions
    - Learned along with the atlas



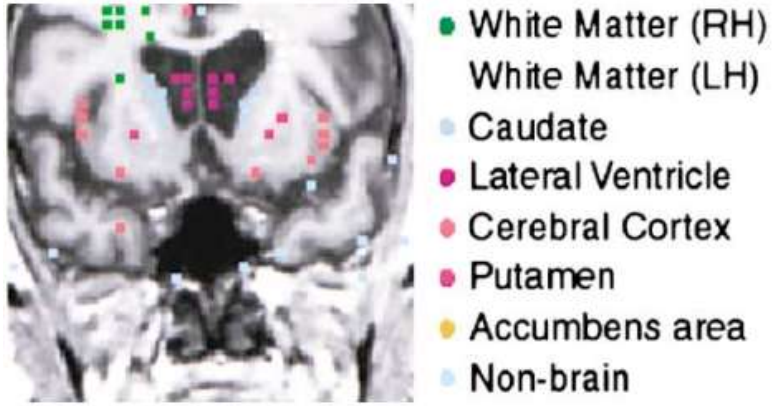


Image from Fischl et al., Neuron 2002

Summary: Markov Random Fields

- Intensity-based segmentation of noisy images can be achieved by combining GMMs with **Markov Random Fields (MRFs)** that model dependencies between pixels
  - MRFs limit dependence to immediate neighbors
  - This allows for factorization using clique potentials
- **Maximum a posteriori (MAP)** estimation is often addressed using heuristic approaches
  - Iterated Conditional Modes (ICM) is a simple option
- Additional **prior knowledge** from an atlas can be included in the unary and pairwise potentials
  - Permits automated classification of specific brain regions

5.5 Graph Cuts for Markov Random Fields

Flow Networks

- **Intuition:** Network of pipelines which can carry a maximum amount of water, roads that can carry a maximum amount of traffic etc.
- **Definition:** A *flow network* $G = (V, E)$ is a directed graph with
  - distinguished source and target nodes s, t
  - edge capacities
    $c(u, v) \geq 0$

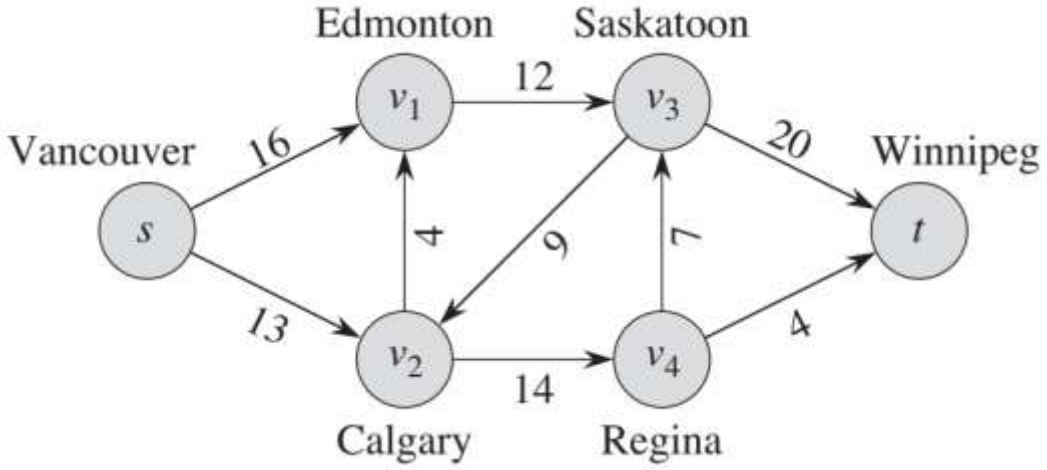


Illustration from [Cormen 2009]

Max-Flow Problem

- **Definition:** A *flow* in G is a real-valued function $f: V \times V \rightarrow \mathbb{R}$ that satisfies
  - **capacity constraint:** $0 \leq f(u,v) \leq c(u,v)\ \forall u,v$
  - **flow conservation:**

$$\sum_{v \in V} f(v,u) = \sum_{v \in V} f(u,v)\ \ \forall u \in V - \{s,t\}$$

- The **max-flow problem** asks for the flow with maximum value

$$|f| = \sum_{v \in V} f(s,v) - \sum_{v \in V} f(v,s)$$

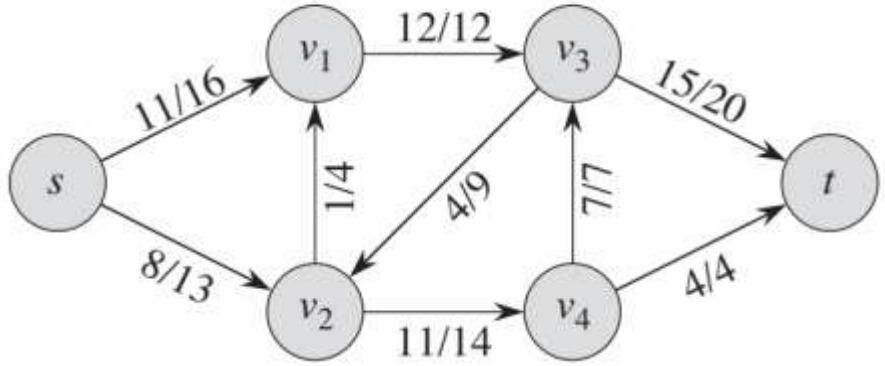


Illustration from [Cormen 2009]

Eliminating Antiparallel Edges

- To simplify algorithms, flow networks disallow **antiparallel edges:**

$$(u, v) \in E \Rightarrow (v, u) \notin E$$

- We will use antiparallel edges later on. However, we can always define an equivalent flow network that eliminates them, by inserting additional vertices:

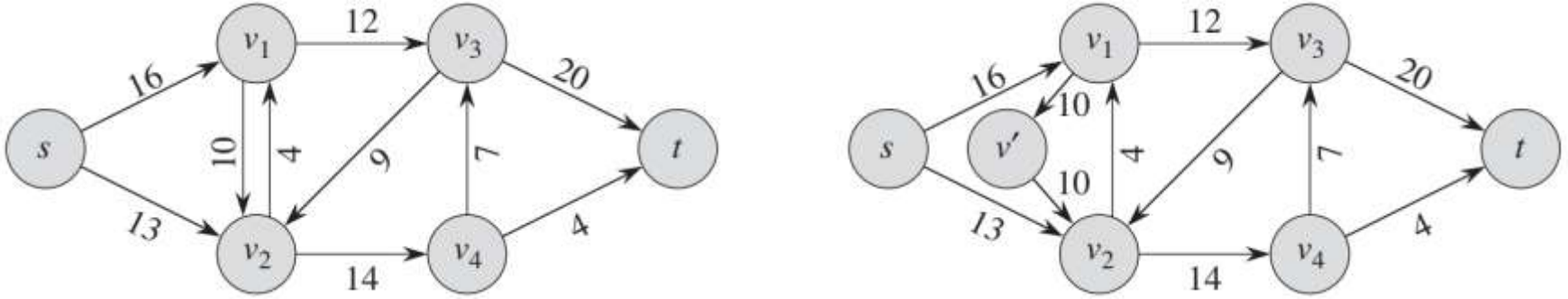


Illustration from [Cormen 2009]

Cuts of Flow Networks

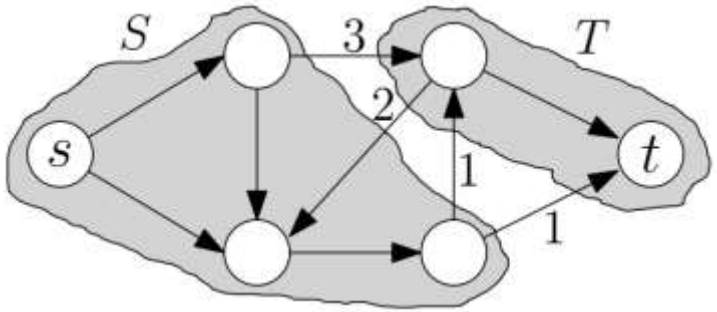
- **Definition 1**: A *cut* (S, T) of a flow network $G = (V, E)$ is a partition of V into S and $T = V - S$ such that $s \in S$ and $t \in T$
- **Definition 2:** The *net flow* $f(S, T)$ across the cut (S, T) is

$$f(S,T) = \sum_{u \in S} \sum_{v \in T} f(u,v) - \sum_{u \in S} \sum_{v \in T} f(v,u)$$

- **Definition 3:** The *capacity* $c(S, T)$ of a cut is

$$c(S,T) = \sum_{u \in S} \sum_{v \in T} c(u,v)$$

Minimum Cut



Quiz: Capacity of this cut?

- **Lemma** (without proof): For any cut (S, T) and flow f, it holds that
$$|f| = f(S, T) \leq c(S, T)$$
- **Definition:** A *minimum cut* of a network is a cut whose capacity is minimum over all cuts.
- **Theorem** (without proof): The minimum cut equals the maximum flow value

Min-Cut / Max-Flow Algorithms

- Efficient (i.e., polynomial time) algorithms for solving the min-cut / max-flow problem are available
  - *Widely known example:* Ford-Fulkerson method / Edmonds-Karp algorithm, $O(|V||E|^2)$ running time

MAP Estimation With Graph Cuts

- **Reminder:** In MAP estimation, we are looking for labels z_i that minimize

$$\sum_i V_i(x_i, z_i) + \sum_i \sum_{j \in N(i)} V_{ij}(z_i, z_j)$$

  - Already know about ICM, but no guarantee of finding optimal solution
- **New:** Under certain conditions, we can design a flow network for which
  - each cut corresponds to a potential labeling and each labeling to a cut
  - the corresponding capacity equals the equation above
  - **MAP Estimation becomes a min-cut problem**

Graph Structure

- We will assume two labels, positive unary terms, and "zero-diagonal" pairwise terms:
  - $V_{ij}(0,0) = V_{ij}(1,1) = 0$
  - $V_{ij}(0,1) > 0,\ V_{ij}(1,0) > 0$
- **General idea:**
  - Connect each pixel to source **and** sink
  - Min-cut has to detach it from either *s* or *t*
    - Detach from *s:* $z_i = 0$
    - Detach from *t:* $z_i = 1$

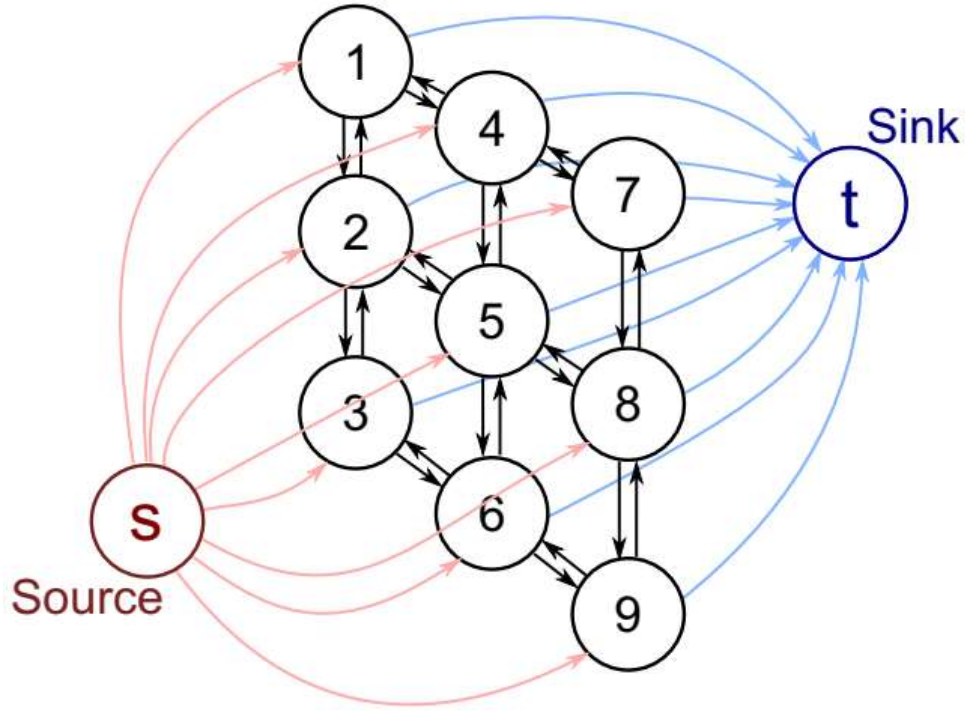


Illustration from [Prince 2012]

Capacities

- To ensure correct value of cut:
  - **Unary costs** are capacities of edges from *s* / to *t*
  - **Pairwise costs** are capacities of edges between pixels as shown

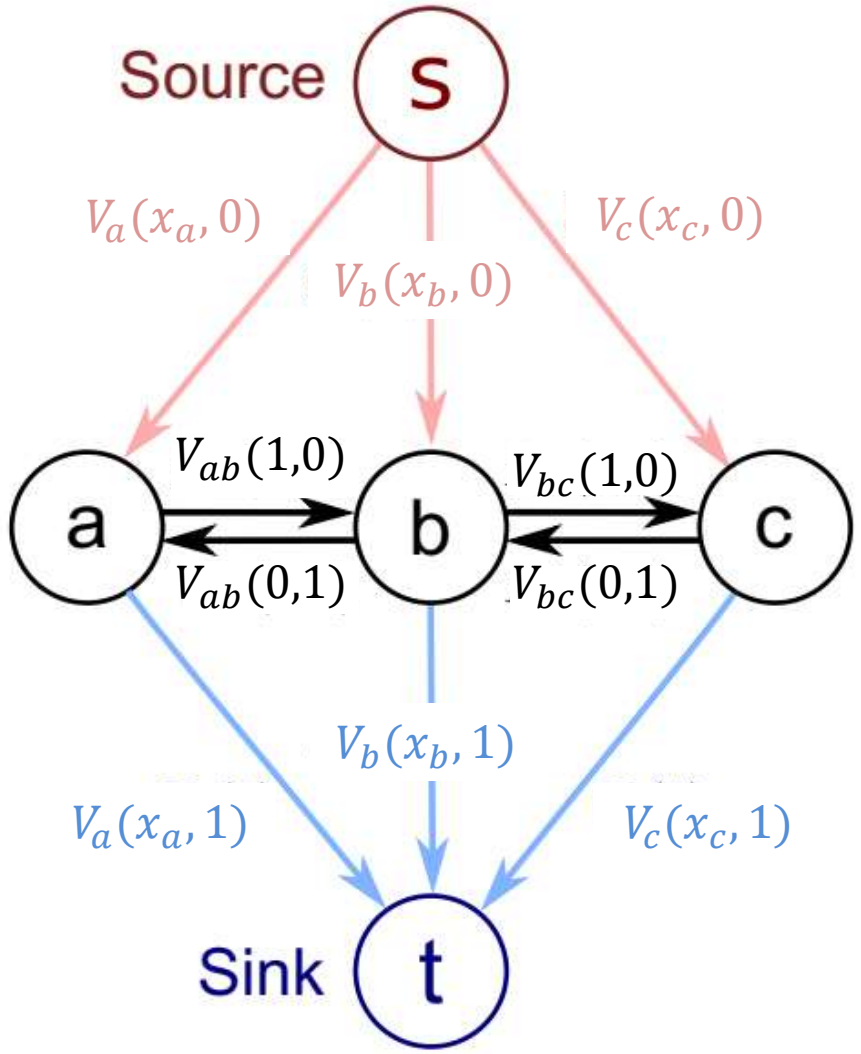
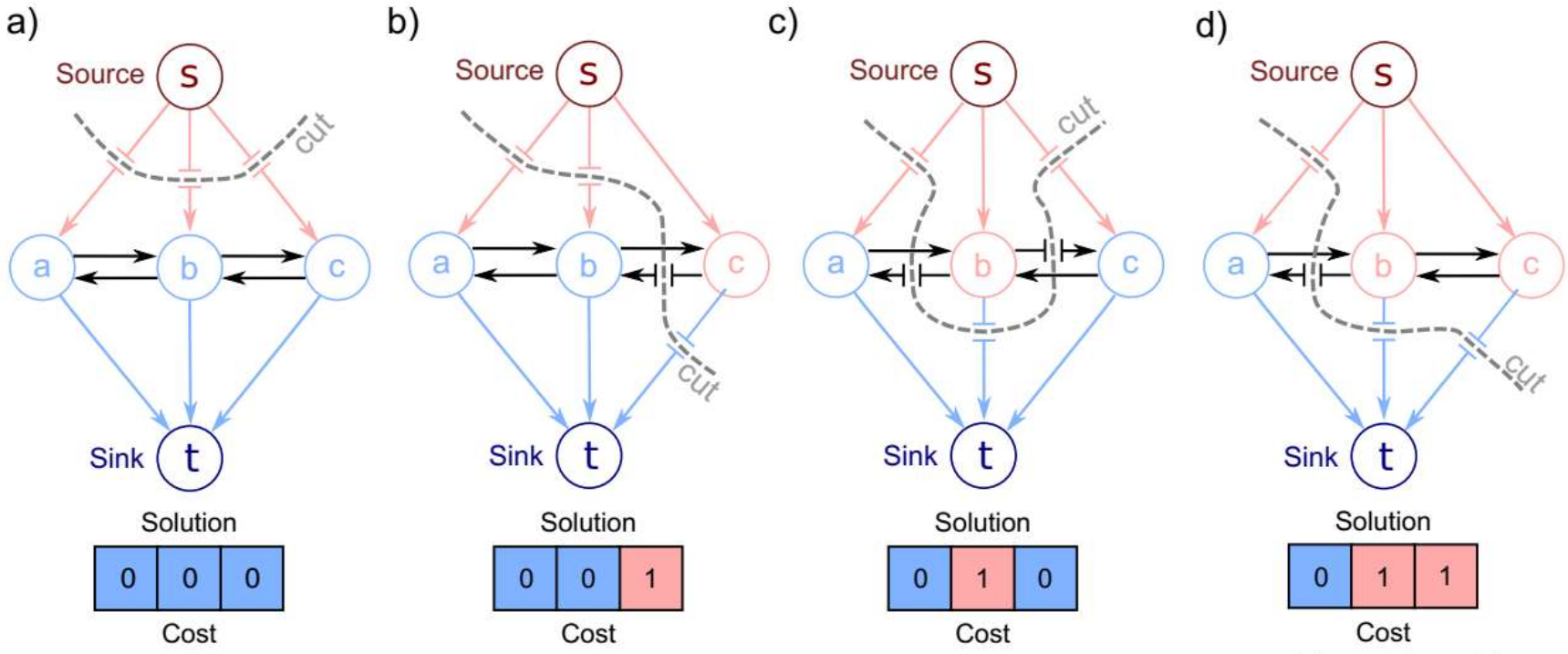


Illustration adapted from [Prince 2012]

A Few Examples



$V_a(x_a, 0)$
$+ V_b(x_b, 0)$
$+ V_c(x_c, 0)$

$V_a(x_a, 0)$
$+ V_b(x_b, 0)$
$+ V_c(x_c, 1)$
$+ V_{bc}(0,1)$

$V_a(x_a, 0)$
$+ V_b(x_b, 1)$
$+ V_c(x_c, 0)$
$+ V_{ab}(0,1)$
$+ V_{bc}(1,0)$

$V_a(x_a, 0)$
$+ V_b(x_b, 1)$
$+ V_c(x_c, 1)$
$+ V_{ab}(0,1)$

Illustration adapted from [Prince 2012]

Graph Cuts With Multiple Labels

- **Multi-label case** can be reduced to two-label case using the idea of iterative *alpha expansion:*
  - In each iteration, objective is minimized with respect to expanding one of the labels (“alpha”)
  - Alternate between labels, terminate once no more expansions are possible
  - *Note:* Optimality no longer guaranteed

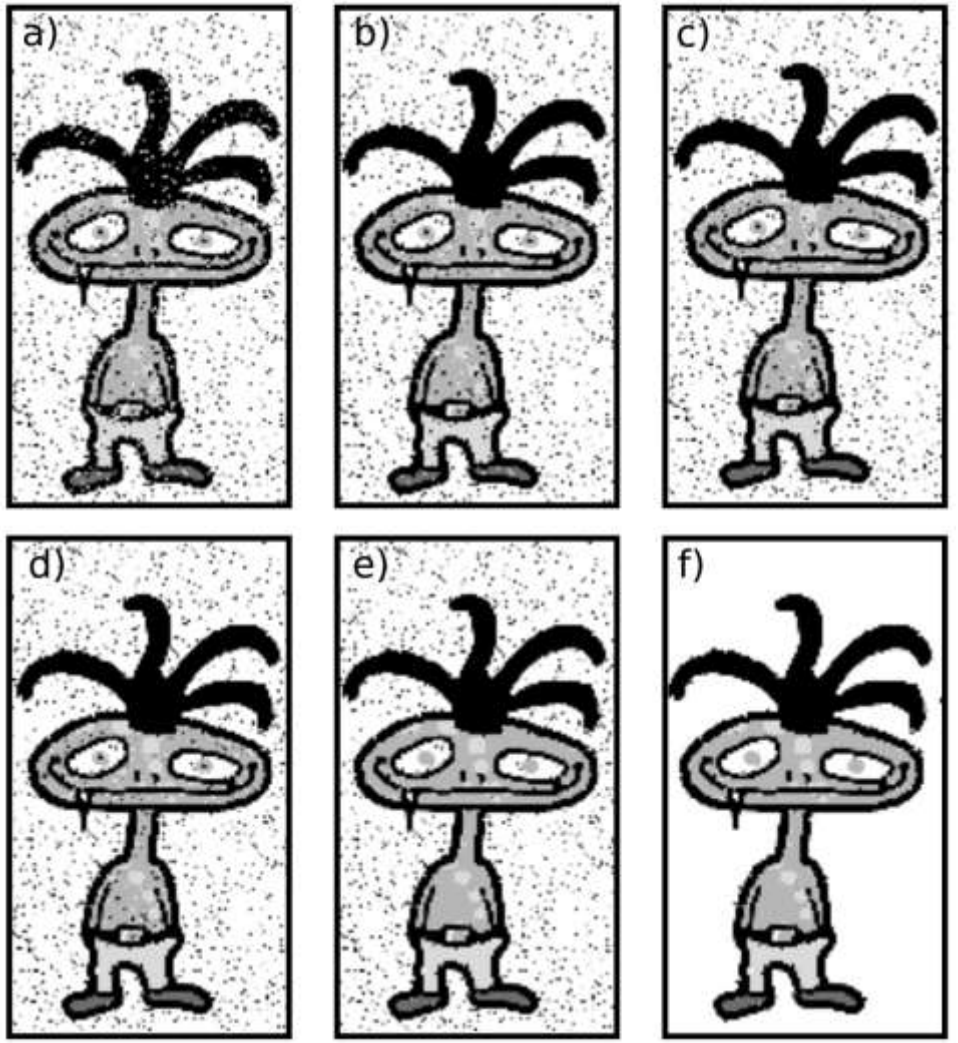
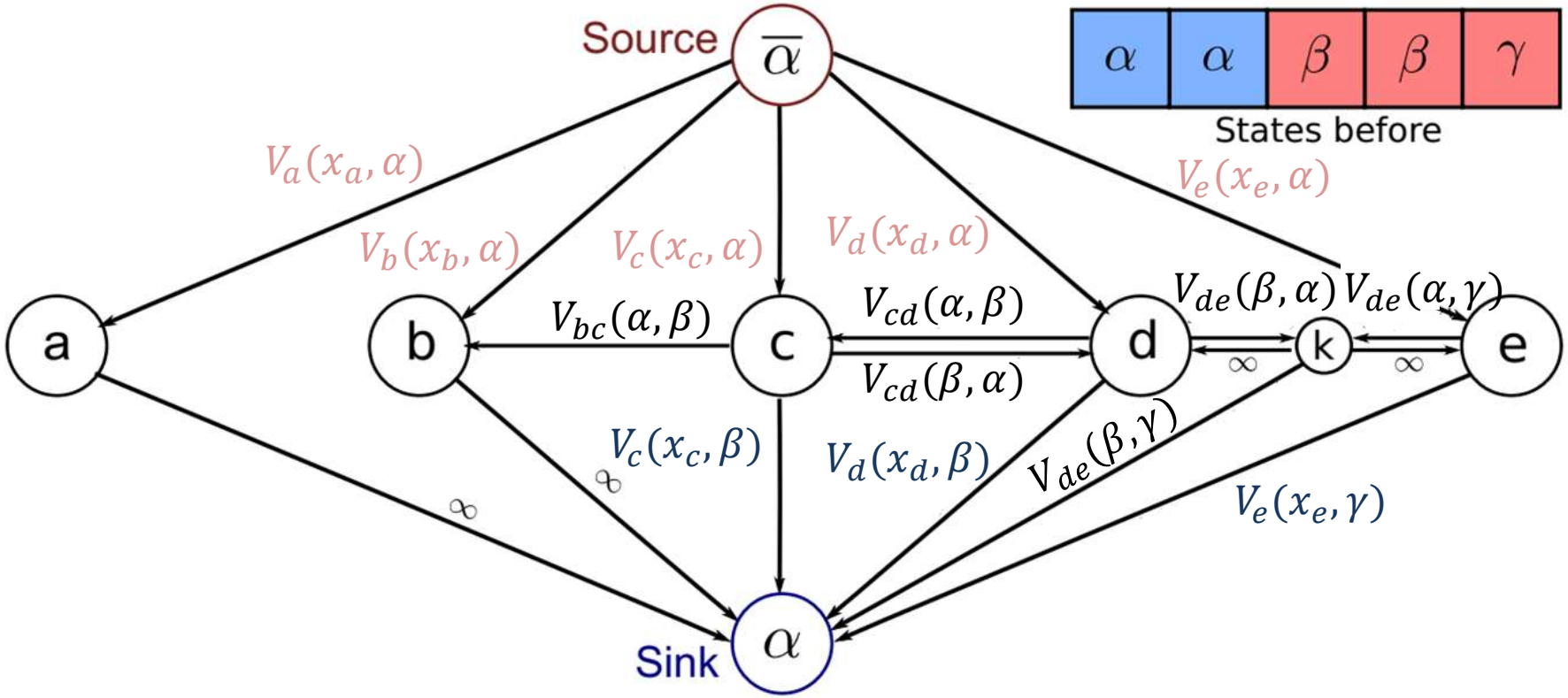


Illustration from [Prince 2012]

Assumptions of Alpha Expansion

- For alpha expansion to work, pairwise potentials have to be metric, i.e.,
  - $V_{ij}(\alpha, \beta) = 0 \Leftrightarrow \alpha = \beta$
  - $V_{ij}(\alpha, \beta) = V_{ij}(\beta, \alpha) \geq 0$
  - $V_{ij}(\alpha, \beta) \leq V_{ij}(\alpha, \gamma) + V_{ij}(\gamma, \beta)$
- In each iteration,
  - pixels with label α keep their label
  - pixels with a different label $\bar{\alpha}$ either keep it or switch it to α

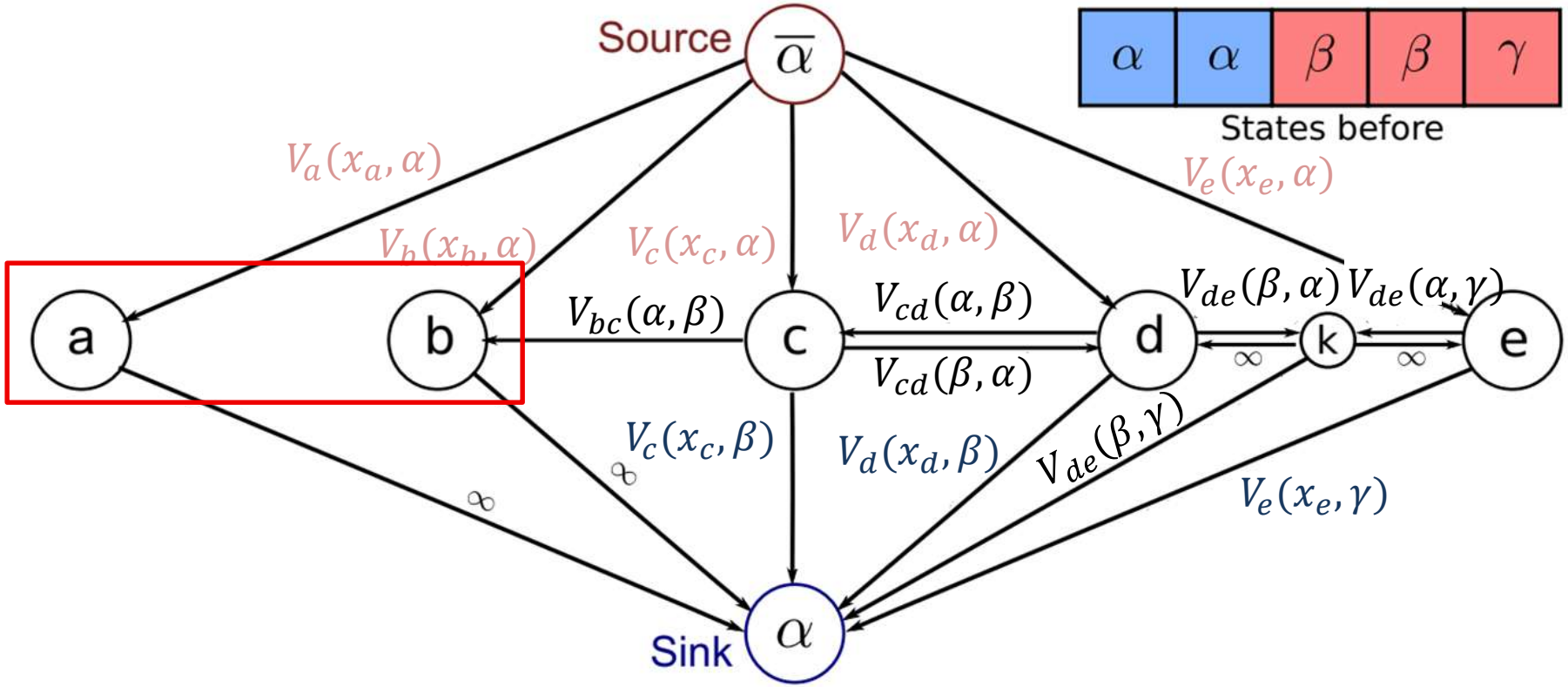
Graph Structure for Alpha Expansion



- Capacities from source / to sink according to unary costs
  - Pixels with label α must keep it

Illustration adapted from [Prince 2012]

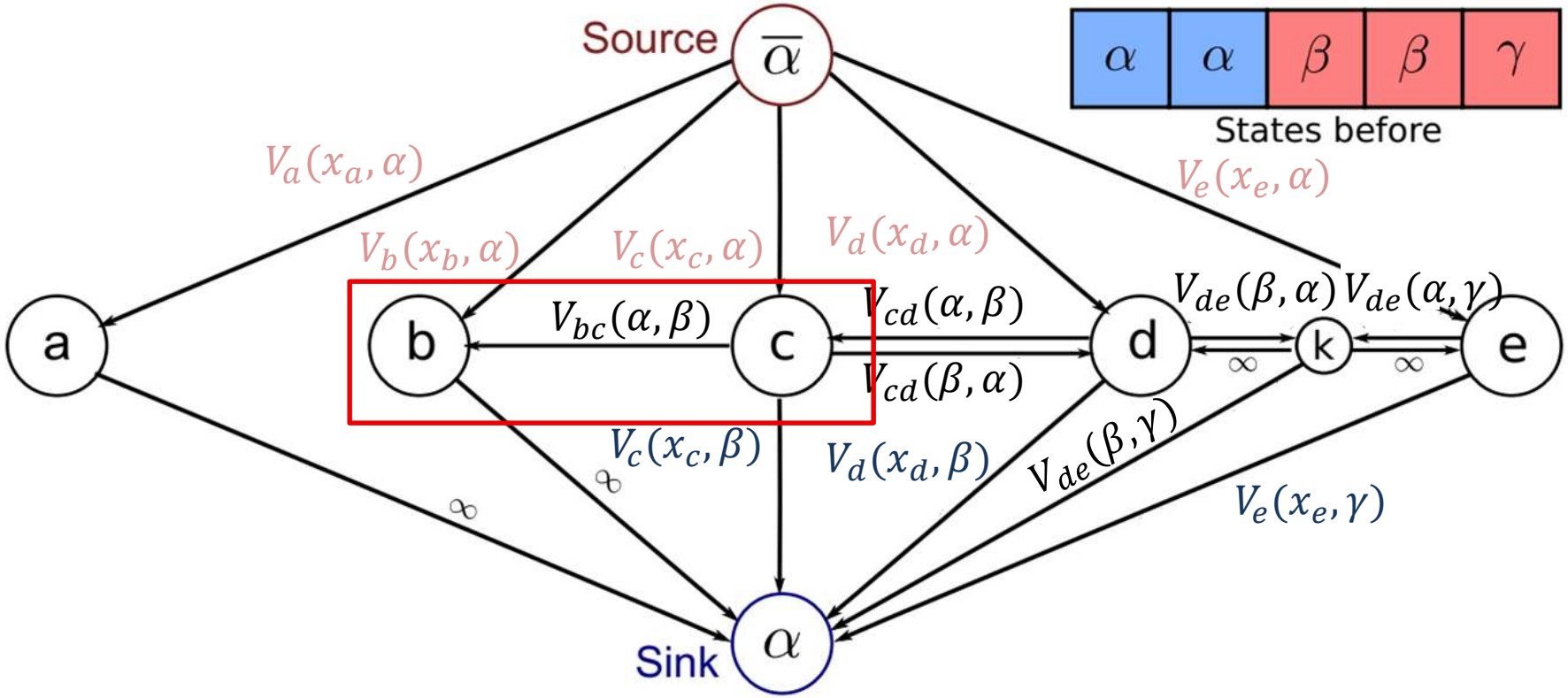
Graph Structure for Alpha Expansion



- If two neighboring pixels had label α, no need to add an edge between them

Illustration adapted from [Prince 2012]

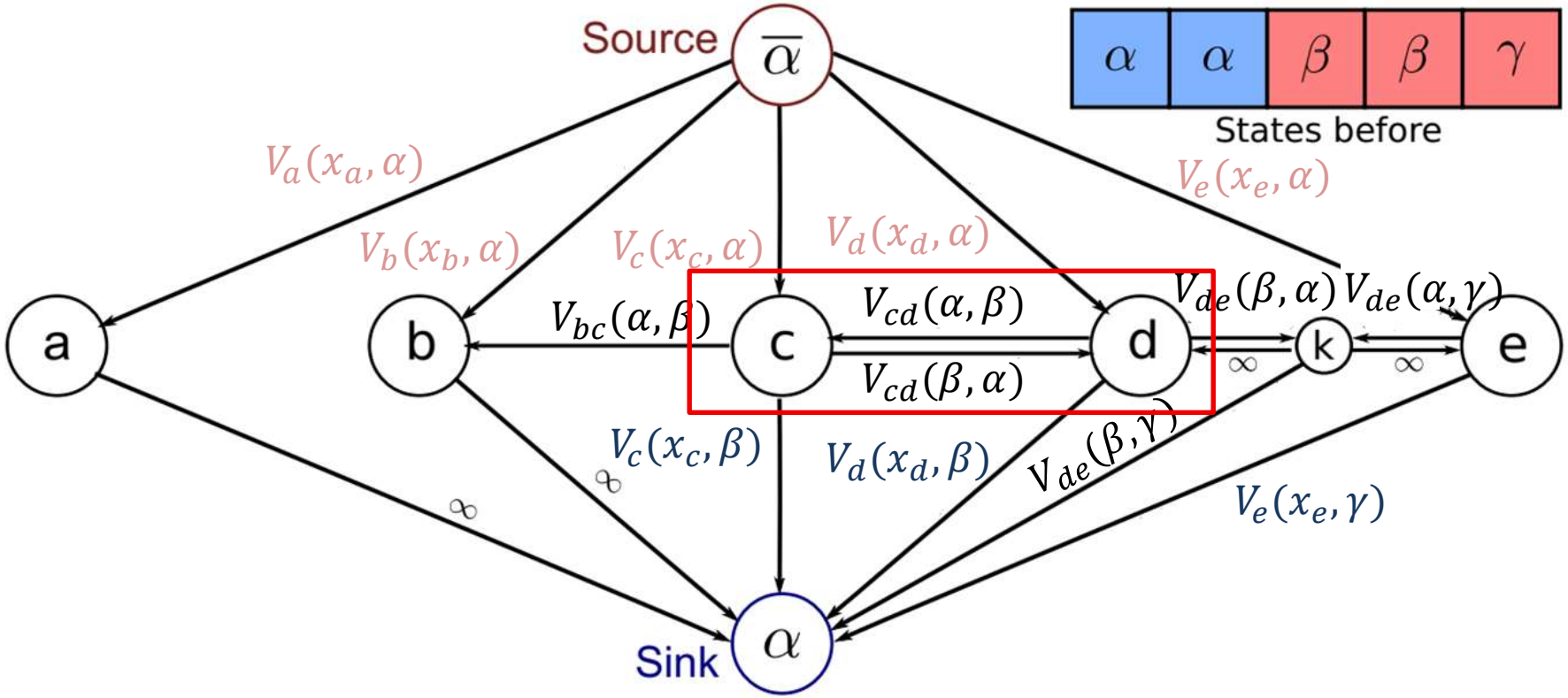
Graph Structure for Alpha Expansion



- If one of two neighboring pixels had label α, require one edge between them

Illustration adapted from [Prince 2012]

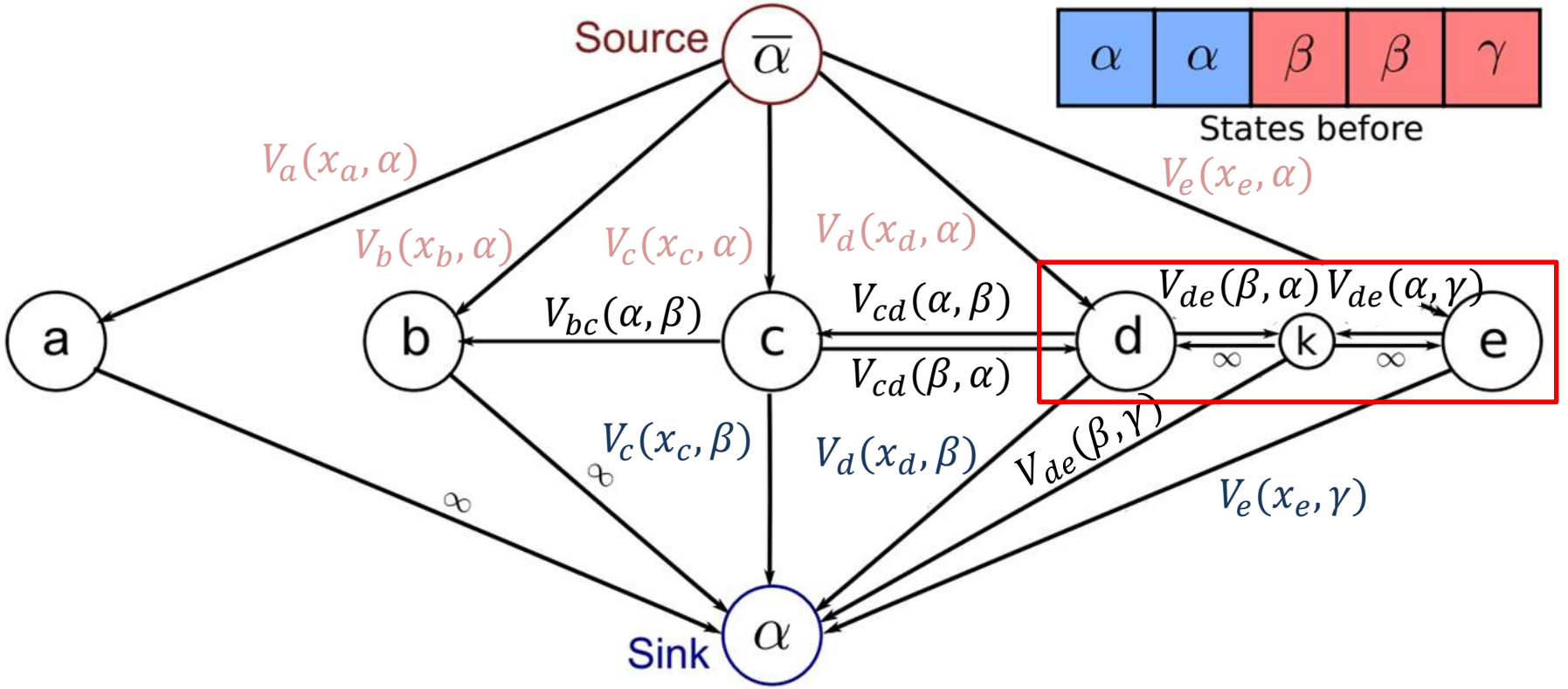
Graph Structure for Alpha Expansion



- If two neighboring pixels had the same label other than α, require two edges between them

Illustration adapted from [Prince 2012]

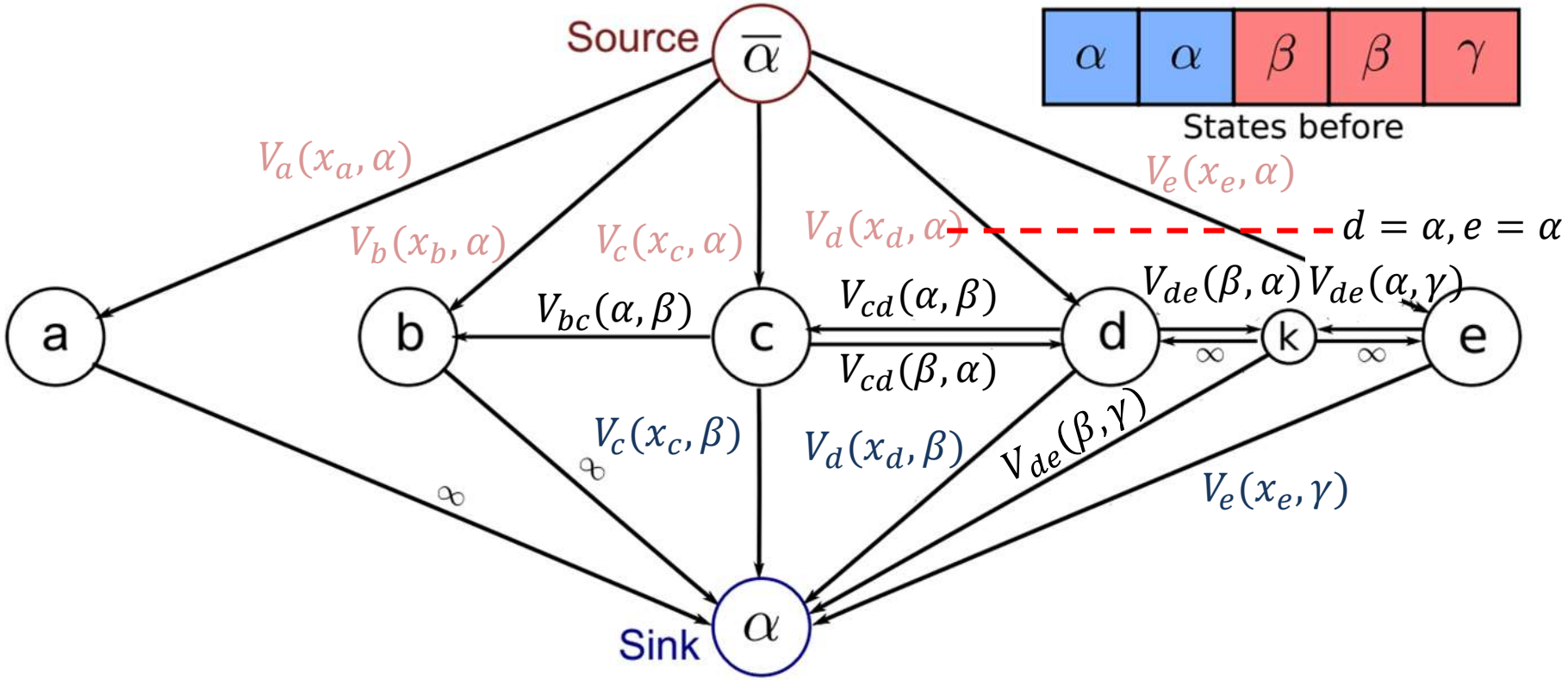
Graph Structure for Alpha Expansion



- If two neighboring pixels had different labels other than α, require a new vertex between them

Illustration adapted from [Prince 2012]

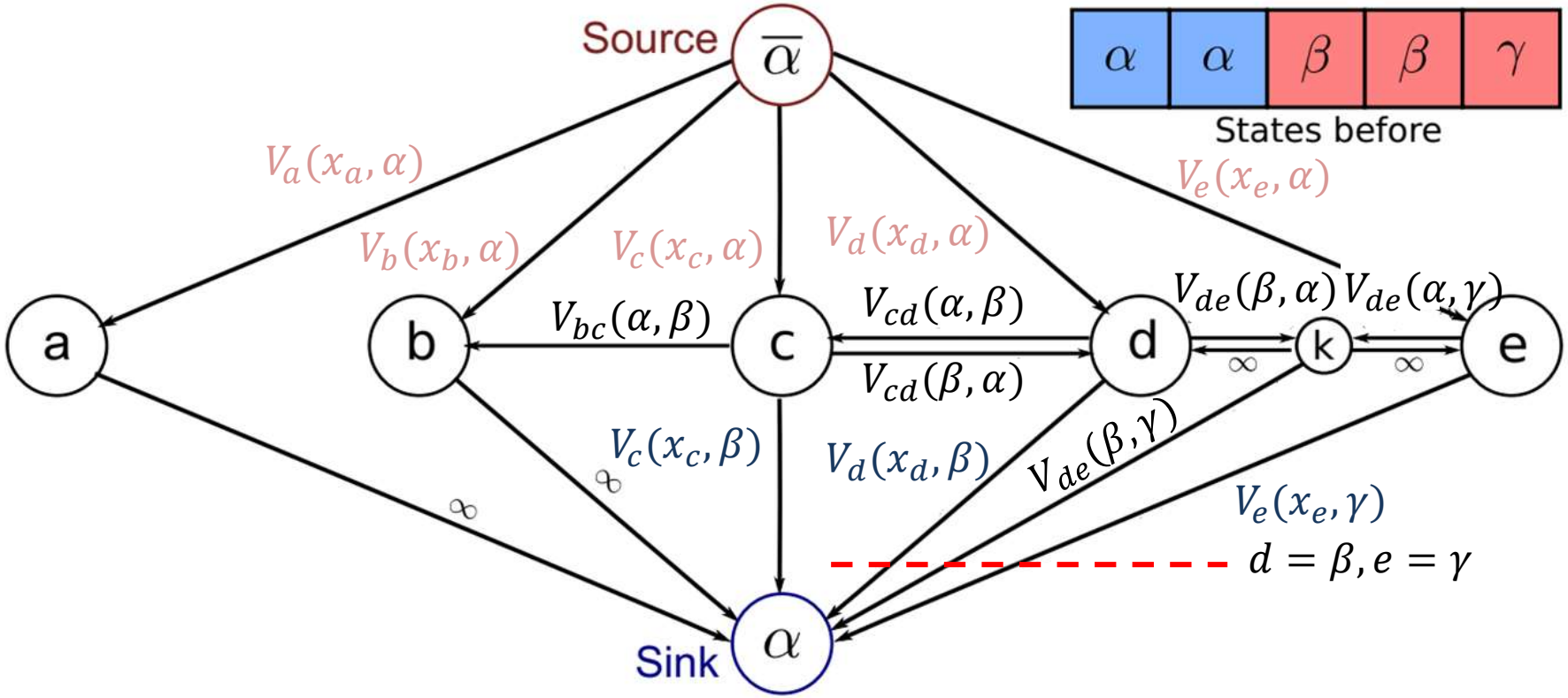
Graph Structure for Alpha Expansion



- If two neighboring pixels had different labels other than α, require a new vertex between them

Illustration adapted from [Prince 2012]

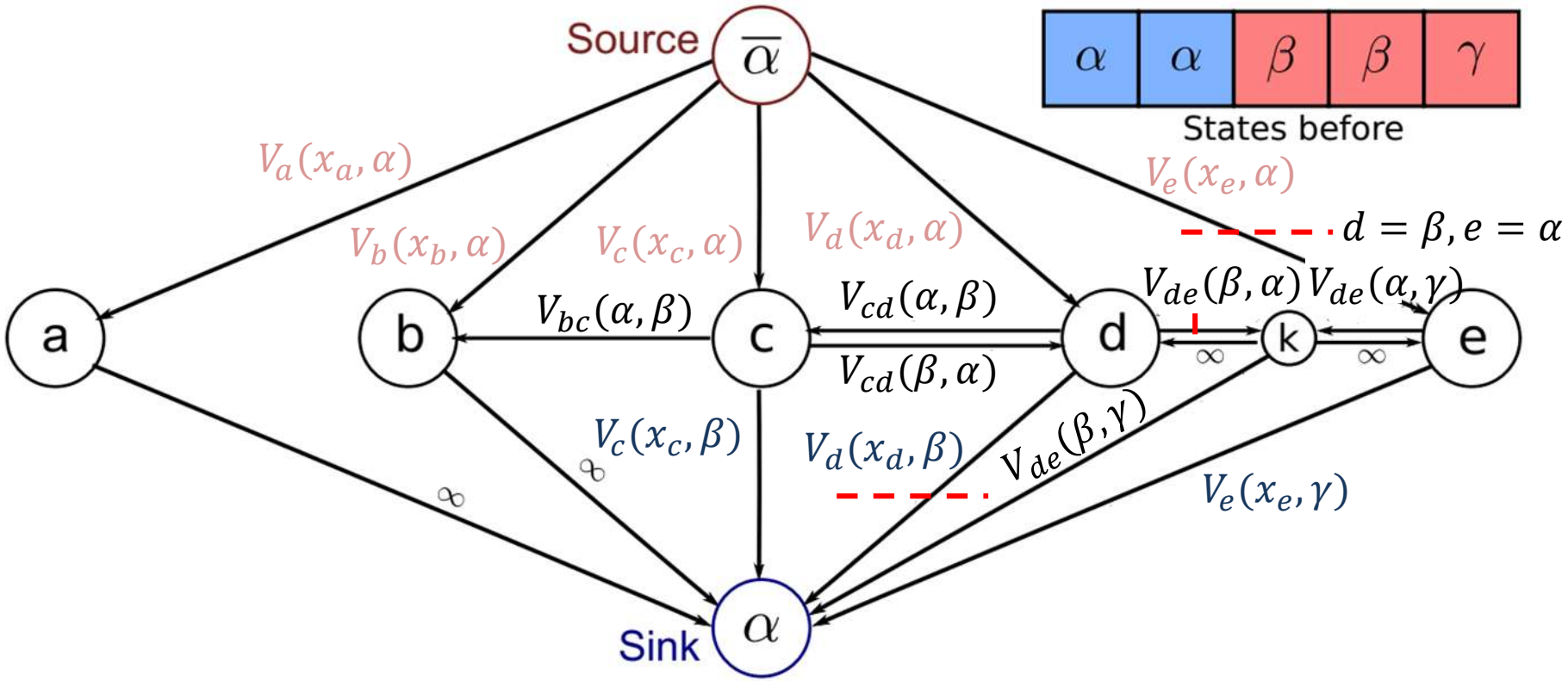
Graph Structure for Alpha Expansion



- If two neighboring pixels had different labels other than α, require a new vertex between them
  - Triangle inequality crucial for this case!

Illustration adapted from [Prince 2012]

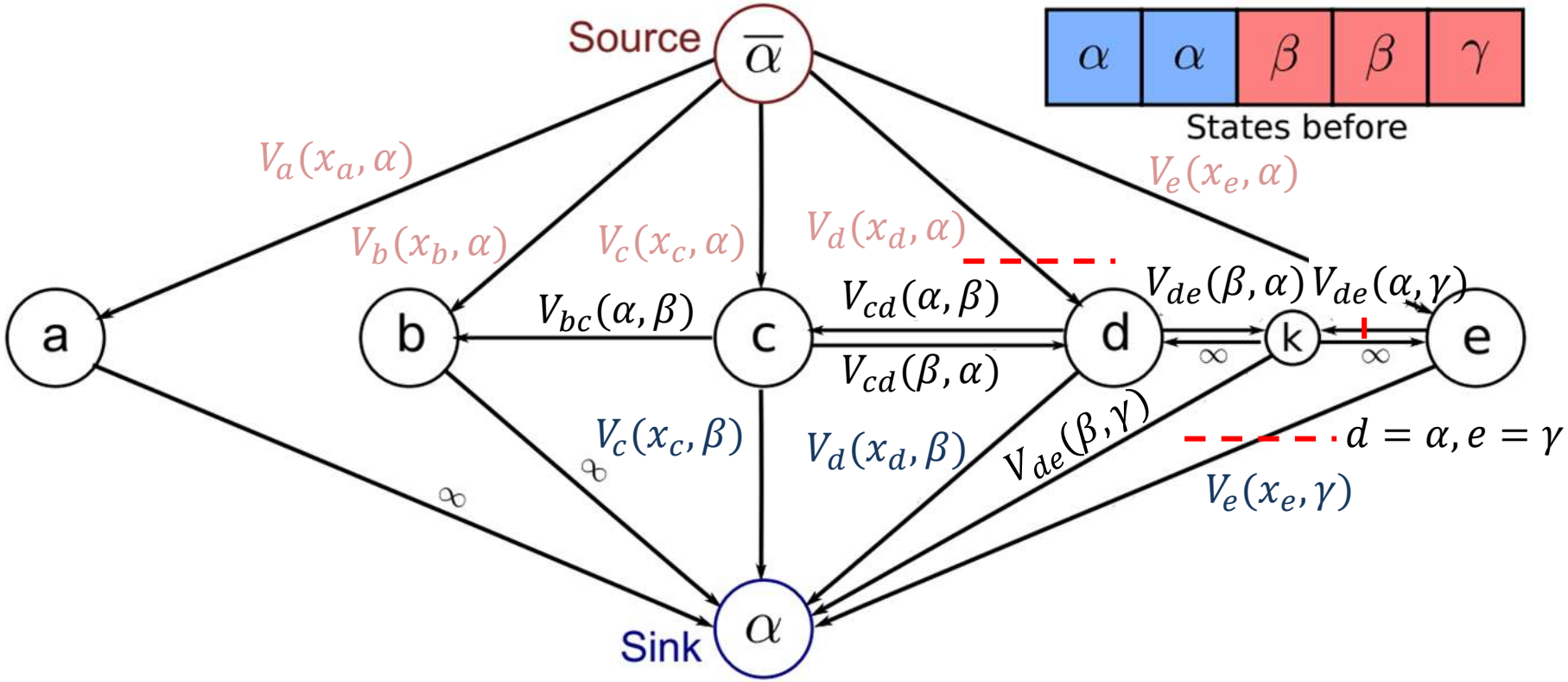
Graph Structure for Alpha Expansion



- If two neighboring pixels had different labels other than α, require a new vertex between them

Illustration adapted from [Prince 2012]

Graph Structure for Alpha Expansion



- If two neighboring pixels had different labels other than α, require a new vertex between them

Illustration adapted from [Prince 2012]

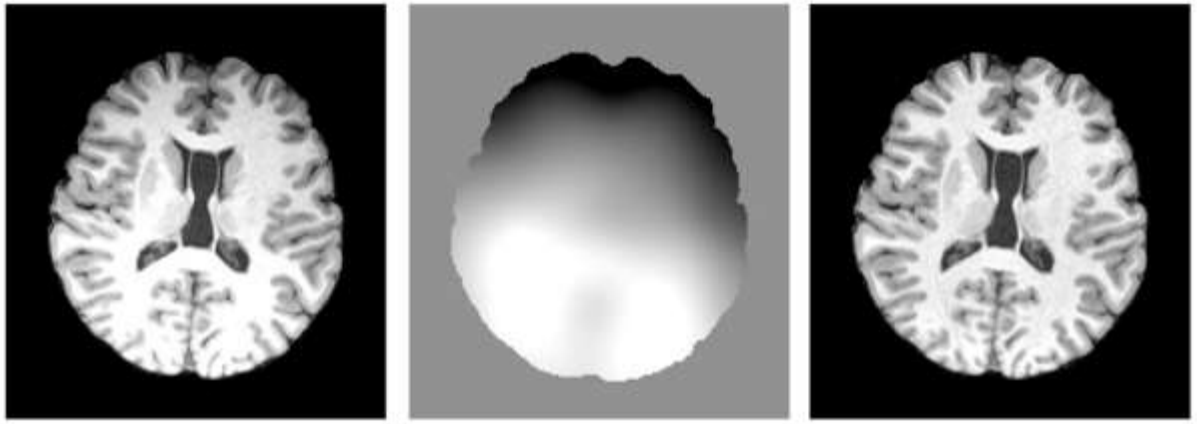
Summary

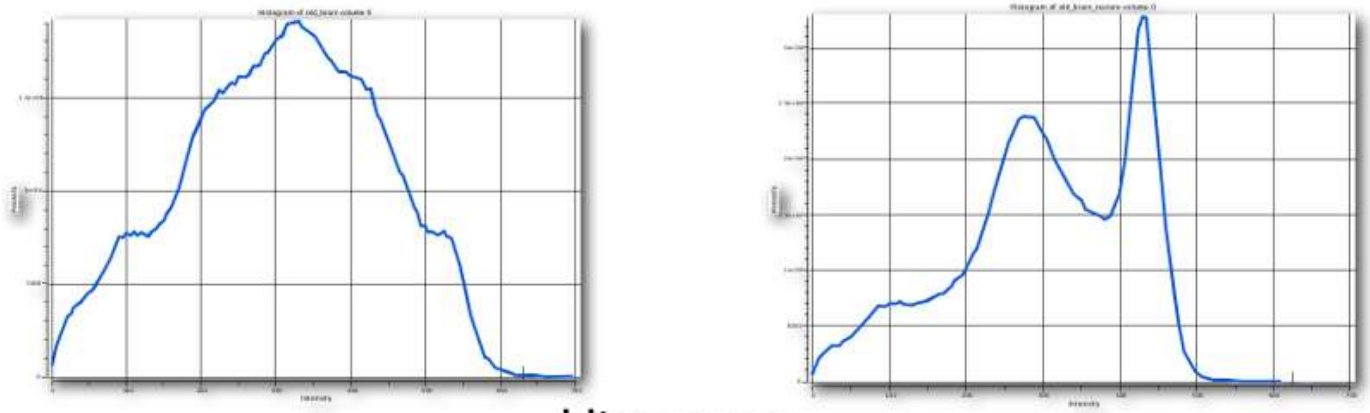
- We saw how **MAP inference in MRFs** can be mapped to a min-cut problem
  - Provides an exact and efficient solution for binary segmentation
  - Can be generalized (via alpha expansion) to multiple labels, but becomes approximative

5.6 Eliminating Bias Fields

Motivation: Bias Fields

- **Bias fields** distort the histogram, can prevent Gaussian Mixture Model from working correctly





Histograms

Images from Oxford brain fMRI lab

Modeling Bias Fields

- **Bias Fields** are due to inhomogeneities in the magnetic field, and/or to the positioning of receiver coils
  - **They are multiplicative**, so we will work with log-intensities $y_i = \ln x_i$ to turn bias into a linear offset
    - Assuming log-Gaussian distribution, accounting for bias b_i:

$$p(y_i|z_{ik}=1, b_i) = \frac{1}{\sqrt{2\pi}\sigma_k} \mathrm{e}^{-\frac{(y_i-\mu_k-b_i)^2}{2\sigma_k^2}}$$

  - **They are slowly varying:** We model the prior probability

$$p(b_S) = \frac{1}{(2\pi)^{\frac{n}{2}}|\Sigma_\mathrm{b}|^{\frac{1}{2}}} \mathrm{e}^{-\frac{1}{2}b_S^T\Sigma_b^{-1}b_S}$$

    as a zero-mean Gaussian with a covariance matrix Σ_b that imposes smoothness (strong and long-range spatial covariation)

Estimating the Bias Field

- Update bias field estimate by setting

$$\nabla_{b_S}(\ln p(y_S|z_S, b_S) + \ln p(b_S)) = \mathbf{0}$$

  - *Reminder:* $p(y_i|z_{ik}=1, b_i) = \frac{1}{\sqrt{2\pi}\sigma_k} \mathrm{e}^{-\frac{(y_i-\mu_k-b_i)^2}{2\sigma_k^2}}$
  - Computation of $\nabla_{b_S} \ln p(y_S|z_S, b_S)$ is almost the same as for $\frac{\partial \ln p}{\partial \mu_k}$ in Chapter 5.3:

$$\frac{\partial}{\partial b_i} \ln p(y_S|z_S, b_S) = \sum_{k=1}^{K} \rho_{ik} \frac{y_i - \mu_k - b_i}{\sigma_k^2} = \underbrace{\sum_{k=1}^{K} \rho_{ik} \frac{y_i - \mu_k}{\sigma_k^2}}_{\text{Define a "residual" vector } r_S \text{ whose } i\text{th coefficient this is}} - b_i \underbrace{\sum_{k=1}^{K} \frac{\rho_{ik}}{\sigma_k^2}}_{\text{Define a diagonal "inverse covariance" } \Sigma_m^{-1} \text{ whose } i\text{th element this is}}$$

Enforcing Smoothness of Bias Field

- Using our new definitions, we obtain

$$r_S - \Sigma_m^{-1} b_S + \nabla_{b_S} \ln p(b_S) = \mathbf{0}$$

- *Reminder:* $p(b_S) = \frac{1}{(2\pi)^{\frac{n}{2}} |\Sigma_b|^{\frac{1}{2}}} \mathrm{e}^{-\frac{1}{2} b_S^T \Sigma_b^{-1} b_S}$
- Taking gradient of log: $\nabla_{b_S} \ln p(b_s) = -\Sigma_b^{-1} b_S$
  leads to the overall equation

$$r_S - \Sigma_m^{-1} b_S - \Sigma_b^{-1} b_S = 0$$

  and yields the following update rule for the bias vector:

$$b_S = H r_S \text{ with } H = \left[\Sigma_m^{-1} + \Sigma_b^{-1}\right]^{-1}$$

Making the Update Rule Practicable

- **Problem:** Computation of $H = \left[\Sigma_m^{-1} + \Sigma_b^{-1}\right]^{-1}$ which is needed to update $b_S = Hr_S$ is infeasible
- **Heuristic solution:** Replace *H* with an efficient to compute matrix that "does the right thing" qualitatively
  - Intuitively, it's clear that *H* should act as a low-pass filter on r_S, so pick an efficient one (e.g., Gauss). We will call it *F*
    - Bandwidth reflects expected smoothness of bias field
  - [Wells et al. 1996] propose to account for measurement variance as follows:

$$b_S = \frac{Fr_S}{F\Sigma_m^{-1}\mathbf{1}}$$ ← component-wise division

  - Unclear (and less important in practice) if this formally corresponds to a valid prior distribution

Integrating Bias Fields into HMRF-EM

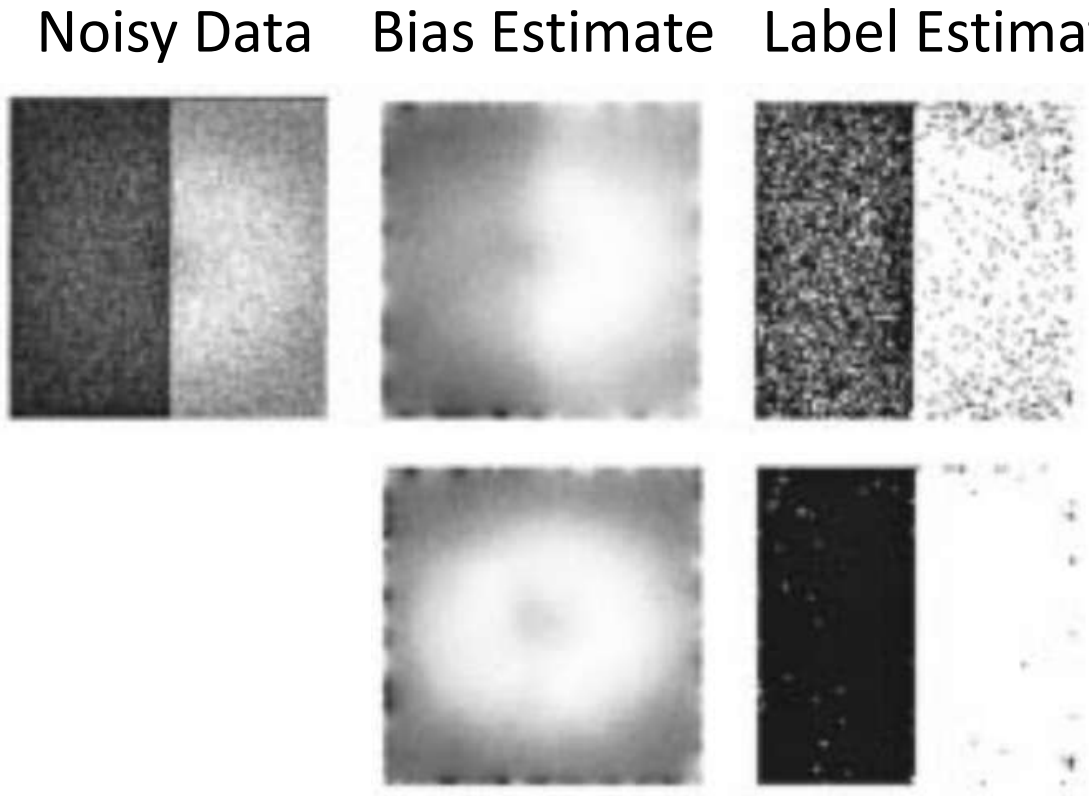
- Bias field estimation has to be iterated as part of the HMRF-EM algorithm:
  - **E-step:**
    1. **New: Estimate bias field (according to rule above)**
    2. Compute MRF-MAP estimate (hard labels)
       - **New: Condition on bias field**
    3. Calculate posterior distribution of labels:

$$\rho_{ik} = p(z_{ik} = 1 | y_i, \boldsymbol{b_i}) = \frac{p(y_i | z_{ik} = 1, \boldsymbol{b_i})\, p(z_{ik} = 1 | N(i))}{p(y_i)}$$

  - **M-step:**
    - **Account for bias** when estimating μ_k and σ_k^2

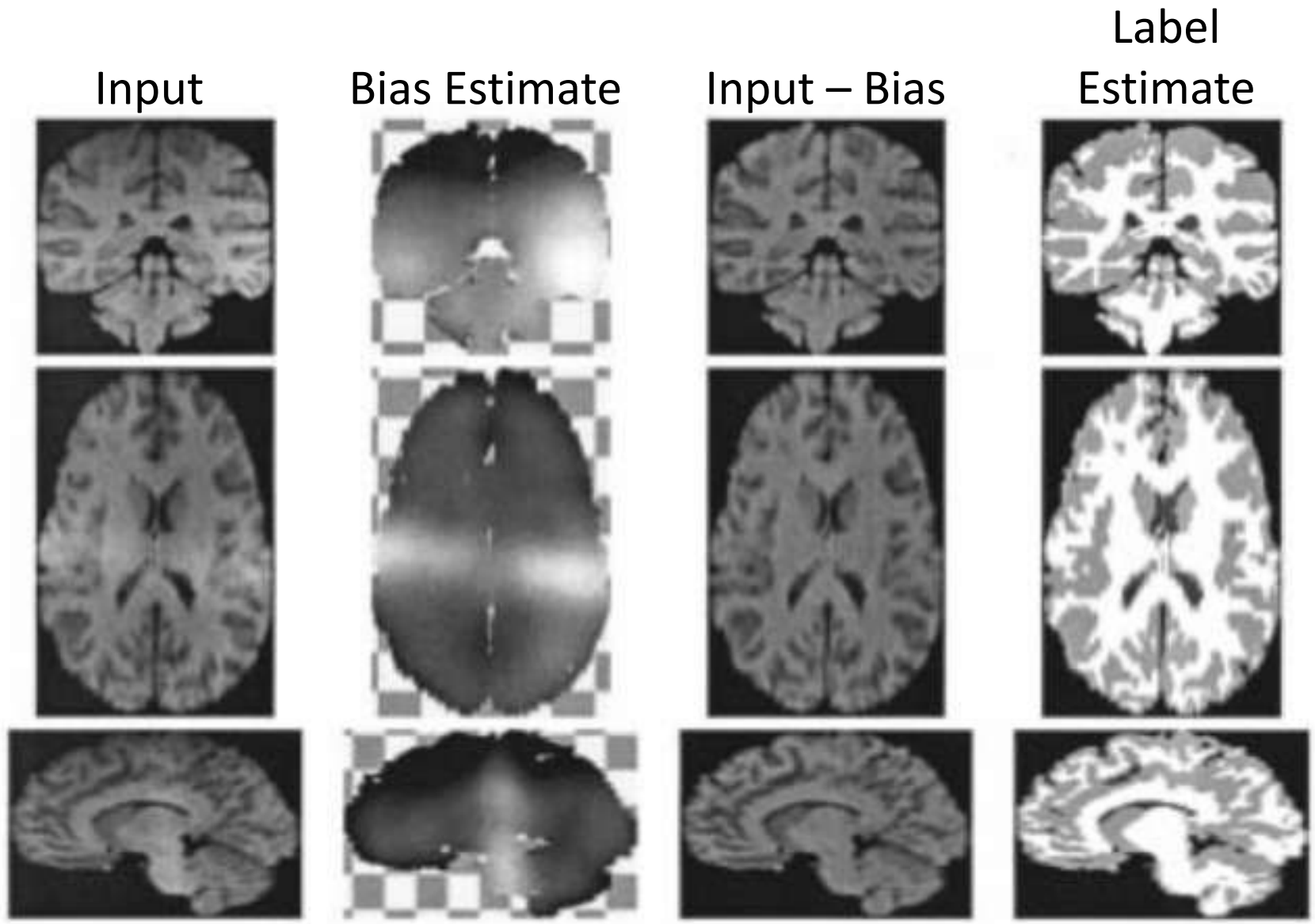
Results on Simulated Data

- In the presence of noise, combining bias field estimation with Markov Random Fields performs much better than bias field removal alone:



Images from Zhang et al. 2001

Results on Real Data



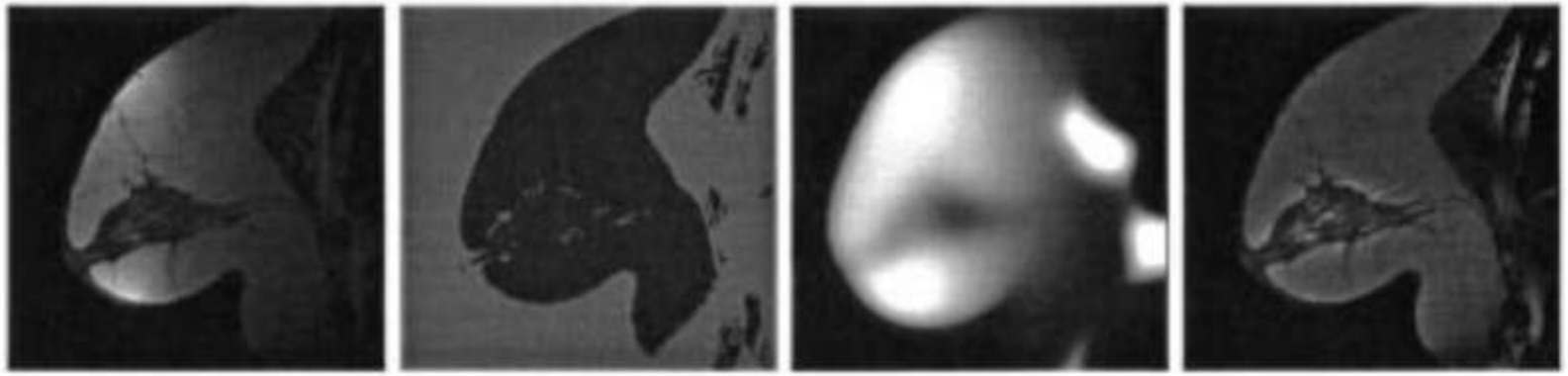
Images from Zhang et al. 2001

Modification for Non-Tissue Classes

- Classes (such as CSF or air) that are poorly described by Gaussians adversely affect bias estimation, which is based on deviations from class means
  - Low-intensity classes are especially problematic, due to logarithm
- [Guillemaud/Brady 1997] propose to model such classes with a uniform distribution instead
  - Removes their impact on bias estimation
    - Partial derivative of uniform distribution with respect to bias parameter vanishes
  - Low-pass filter extrapolates bias estimates into regions where no reliable estimate can be found

Impact of Modification

- Result with Gaussian classes only:



- Result with uniform non-tissue class:

Input Data Classification Bias Estimate Input - Bias

Images from [Guillemaud/Brady 1997]

Summary: Tissue Classification

- A well-established method for **tissue classification** combines:
  - **Gaussian mixture model** for tissue classes
  - **Uniform model** for non-tissue class
  - Hidden **Markov Random Field** to model dependencies between labels in neighboring pixels
  - Explicit modeling of **bias fields**
    - Work with logarithm of the signal, since bias fields are multiplicative
    - Heuristic “prior” (smooth residuals)

5.7 Convolutional Neural Networks

Motivation: Limitations of Markov Random Fields

- In **Markov Random Field** based image segmentation, pixel labels only depend on local intensities and the labels of direct neighbors
  - Distinguishing between locally similar brain structures requires time-consuming registration to an atlas
  - Inference requires time-consuming iterative algorithms, i.e., EM algorithm, iterated conditional modes, alpha expansion
  - FreeSurfer: Approx. 7h/brain volume, 4h with parallel processing

Main Idea: CNNs for Image Segmentation

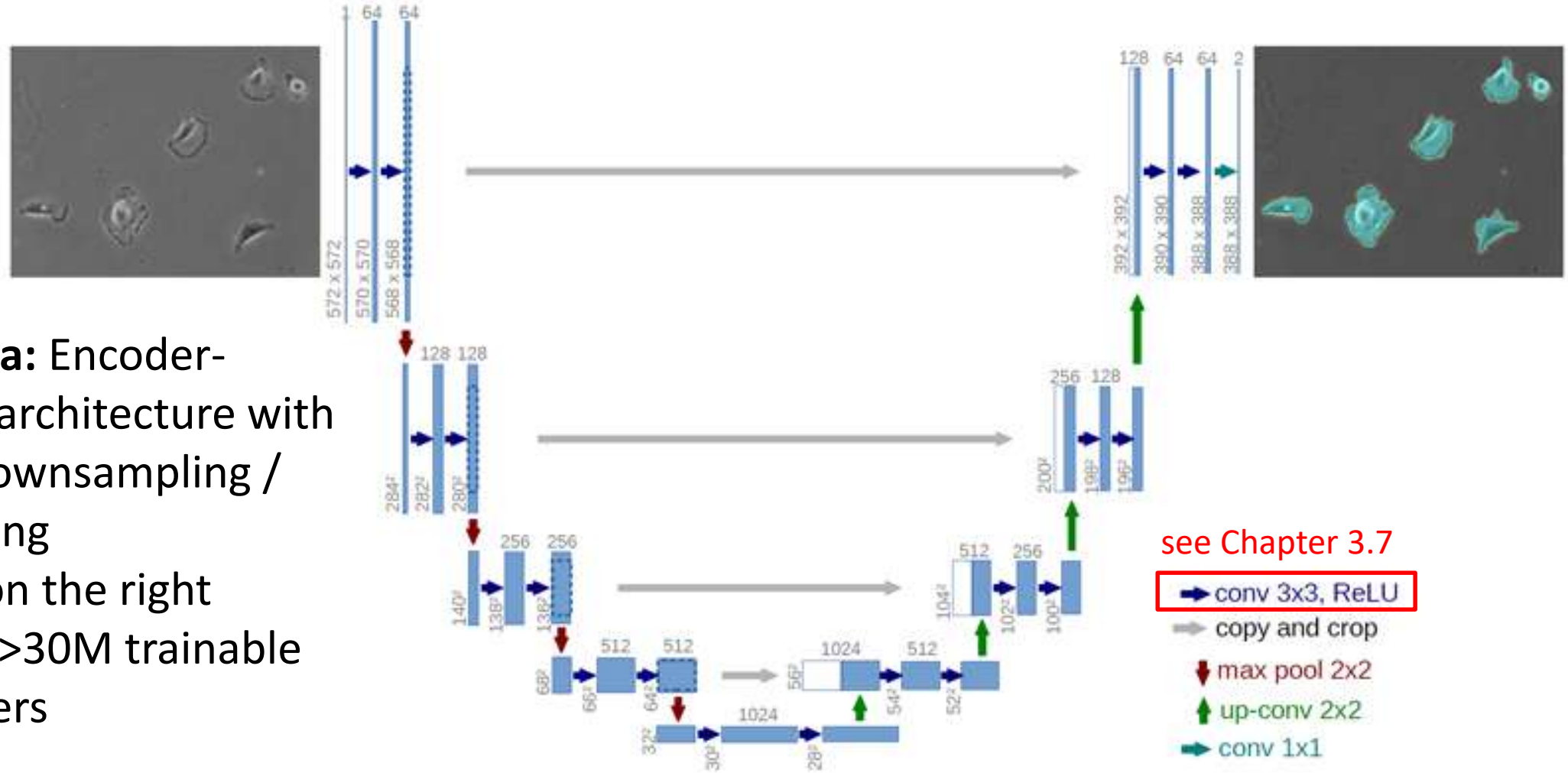
- **Convolutional neural networks** perform segmentation by applying a fixed sequence of **feed-forward** operations
  - Hierarchical convolution filters, followed by pixel-wise nonlinearities, usually at multiple spatial resolutions
    - **Fully convolutional** architectures can process images of arbitrary size
  - Same architecture for many different segmentation tasks
    - Gradient-based **end-to-end learning** of parameters (filter weights)
  - No need for registration / atlas construction, makes it easier to deal with anatomical variability, potential to increase quality
  - Training is costly, but inference is fast
    - FastSurfer: 1 minute (GPU) or 14 minutes (CPU)

Overview of this Section

- We will focus on two specific CNN architectures for image segmentation
  - **U-Net** [Ronneberger et al. 2015] is extremely popular for medical image segmentation (Nov 2023: >73000 citations)
  - **FastSurfer** [Henschel et al. 2020] provides a specific solution for brain segmentation
- As before, we will focus on segmentation process (“inference”) itself, we will not go into much detail on training

Overview: U-Net Architecture

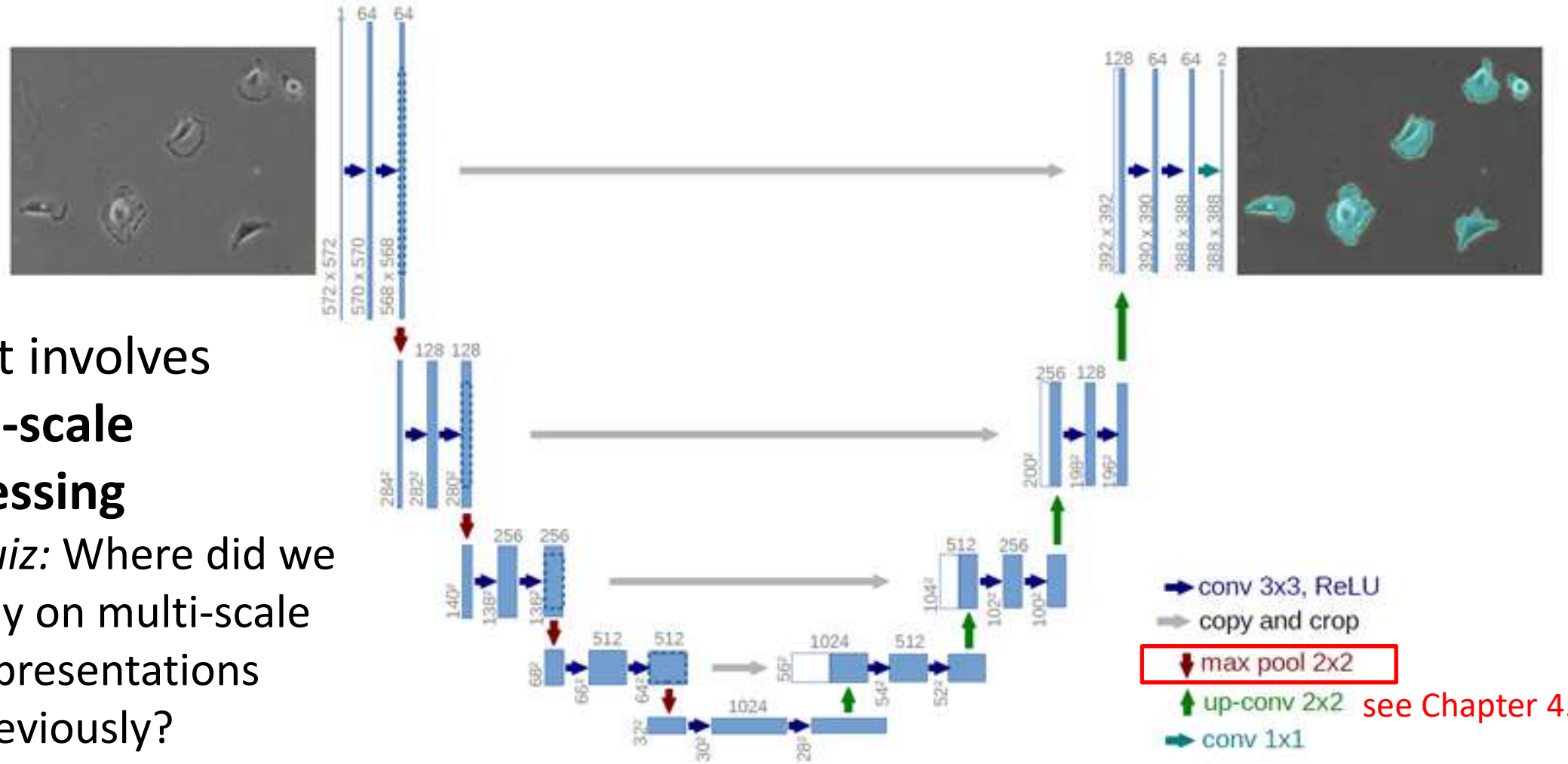
- **Main idea:** Encoder-decoder architecture with spatial downsampling / upsampling
- Version on the right contains >30M trainable parameters



U-Net architecture by Ronneberger et al. (2015)

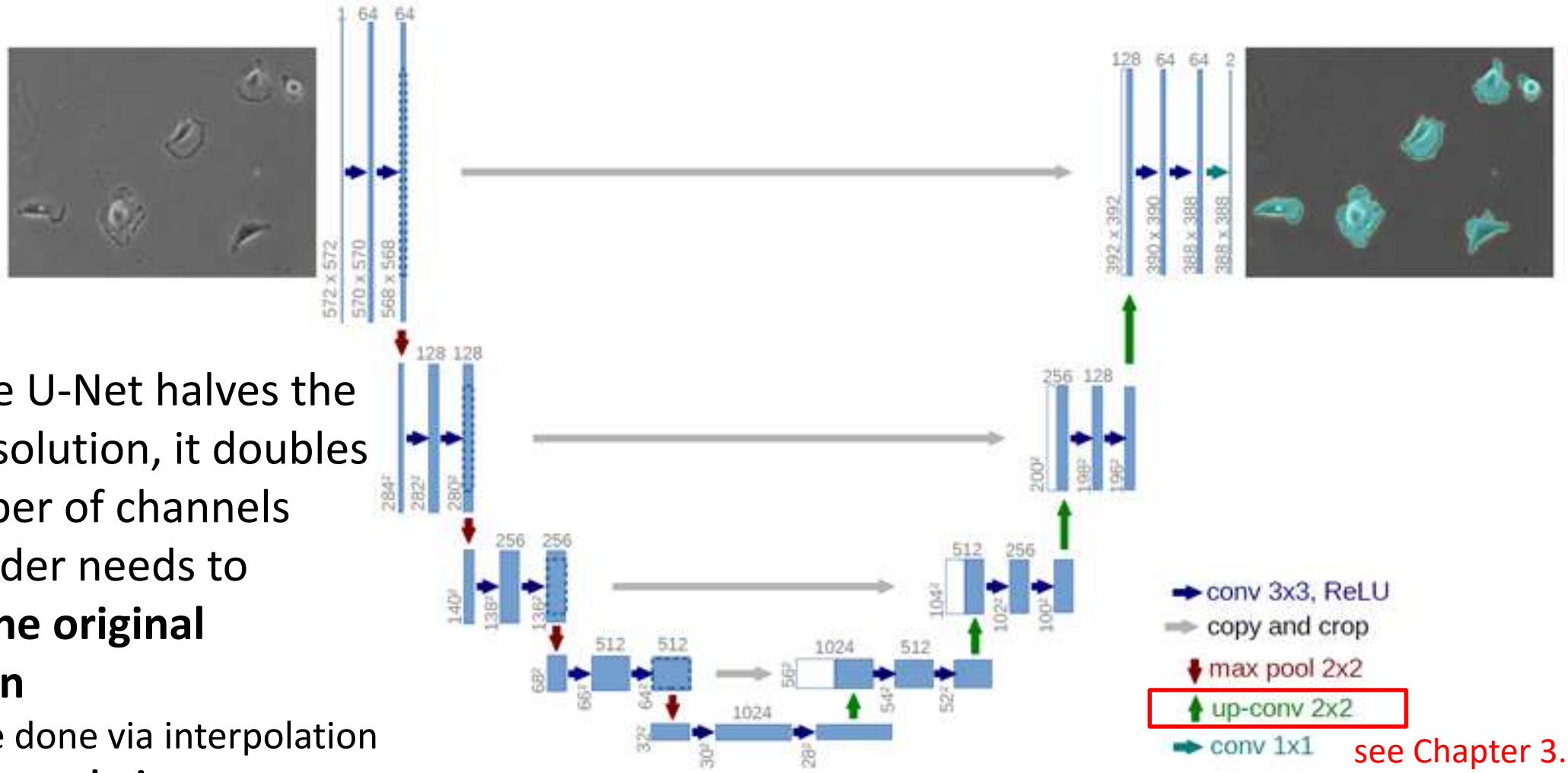
Overview: U-Net Architecture

- U-Net involves **multi-scale processing**
  - *Quiz:* Where did we rely on multi-scale representations previously?



U-Net architecture by Ronneberger et al. (2015)

Overview: U-Net Architecture



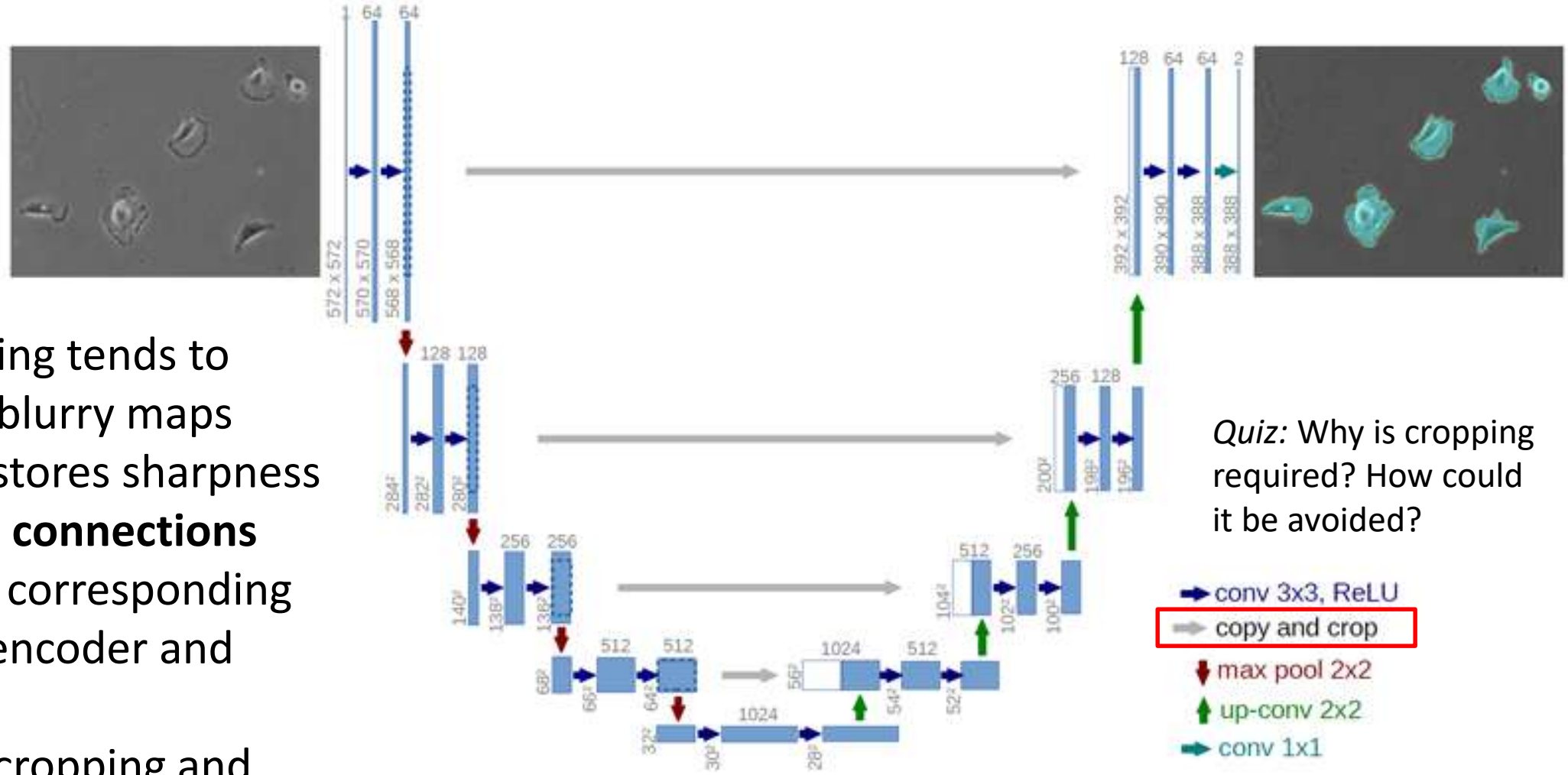
- When the U-Net halves the image resolution, it doubles the number of channels
- The decoder needs to **restore the original resolution**
  - Can be done via interpolation or **up convolution = transposed convolution**

U-Net architecture by Ronneberger et al. (2015)

see Chapter 3.7

U-Net: Skip Connections

- Upsampling tends to produce blurry maps
- U-Net restores sharpness with **skip connections** between corresponding parts of encoder and decoder
- Involves cropping and concatenation



Quiz: Why is cropping required? How could it be avoided?

U-Net architecture by [Ronneberger et al. 2015]

U-Net: Padding and Overlap-tile

- The original U-Net uses convolutions without padding
  - *Consequence*: Segmentation result is smaller than input patch
  - Input image is padded with mirror boundary condition
- Large images (exceeding GPU memory) can be processed piecewise
  - Patches overlap so that outputs tile the image

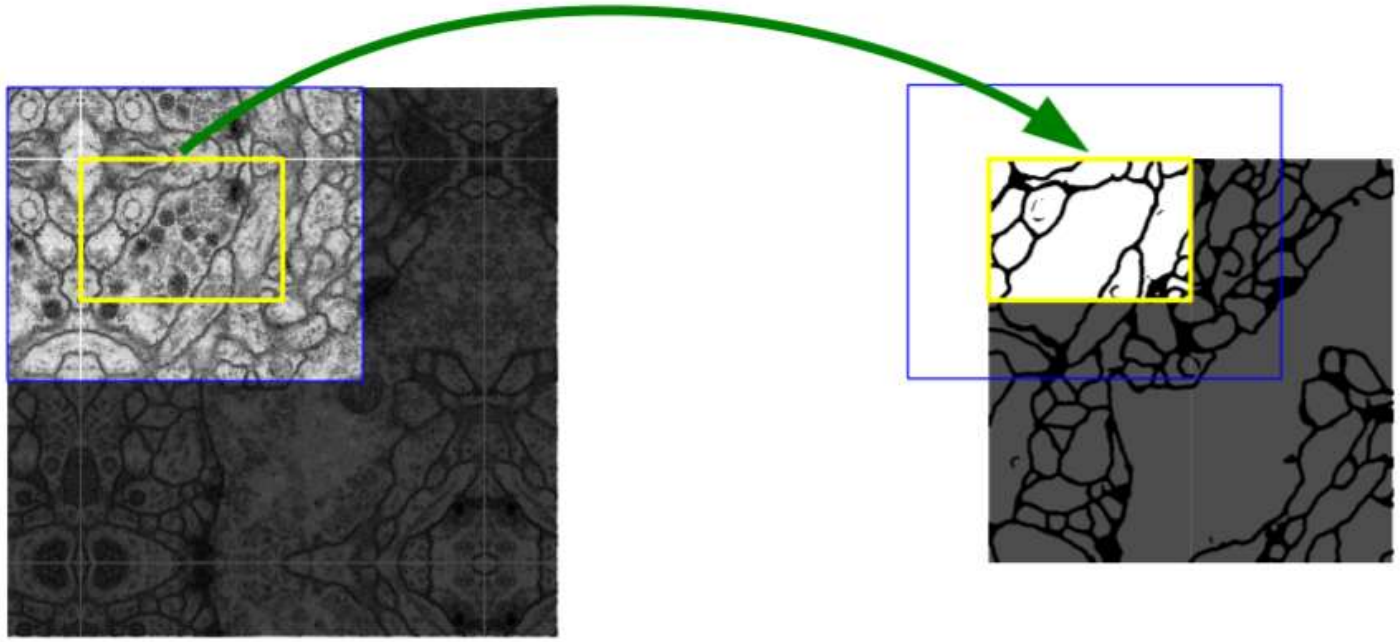


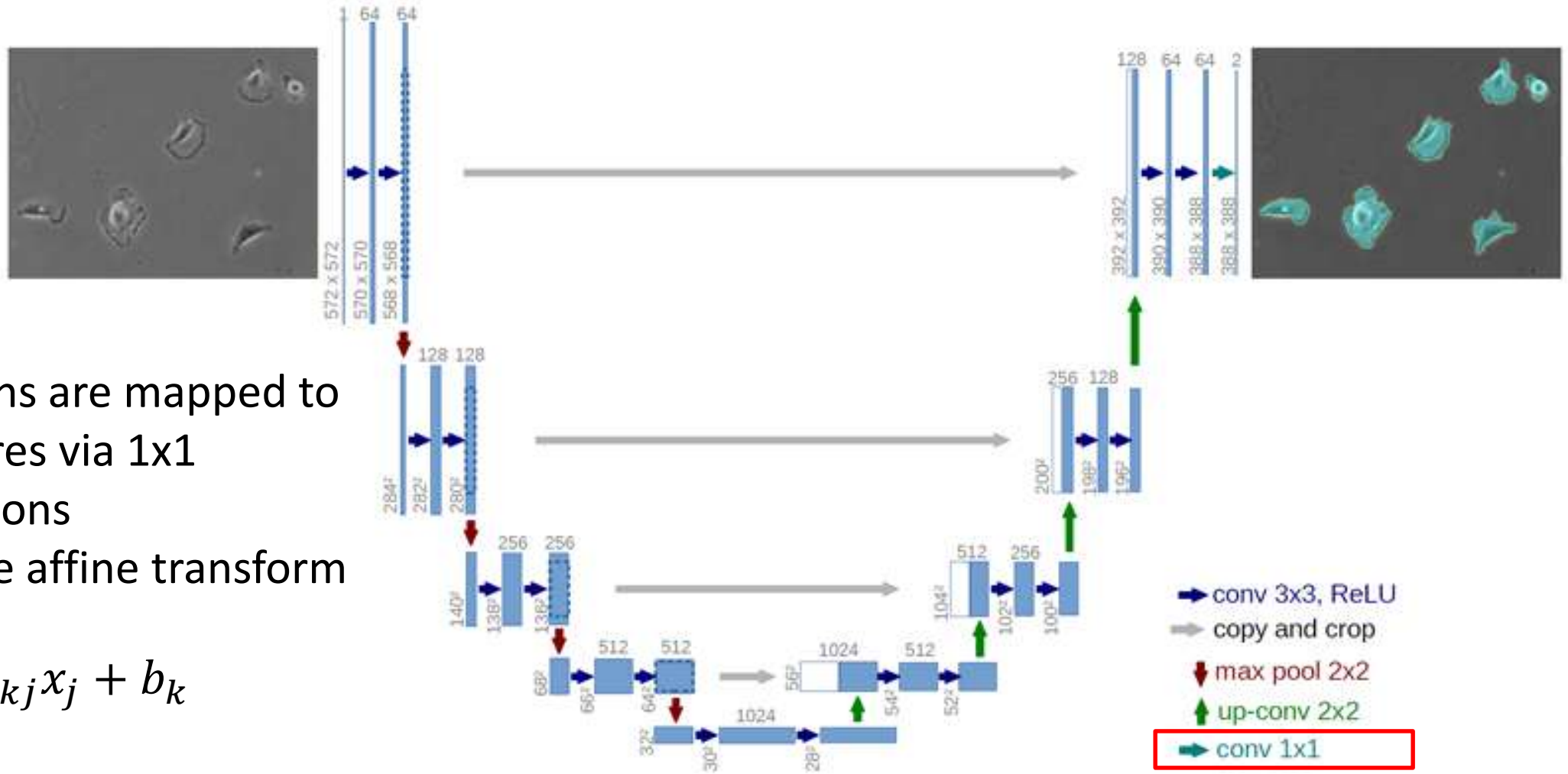
Figure from [Ronneberger et al. 2015]

Overview: U-Net Architecture

- Activations are mapped to class scores via 1x1 convolutions
- Pixel-wise affine transform

$$\sum_{j=1}^{m} w_{kj} x_j + b_k$$

  - Activations x_j
  - Weights/biases w_{kj}/b_k



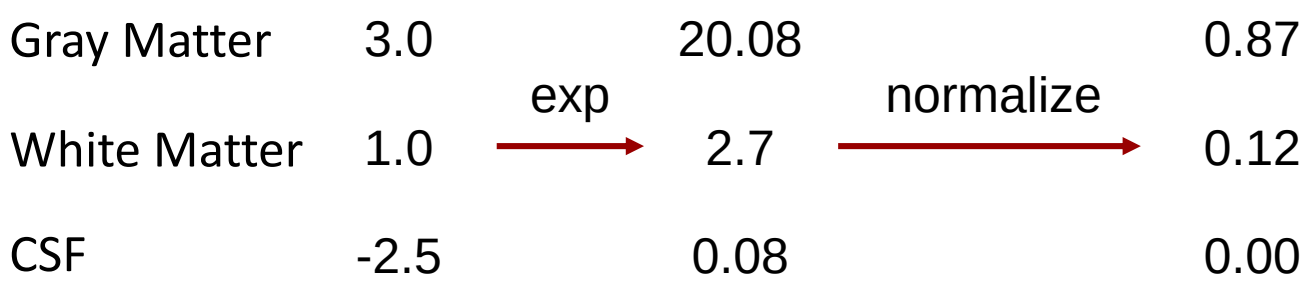
U-Net architecture by Ronneberger et al. (2015)

Probabilistic Predictions via Softmax

Softmax converts raw class scores s_j into non-negative and normalized class probabilities:

$$\frac{e^{s_k}}{\sum_j e^{s_j}}$$

Example:

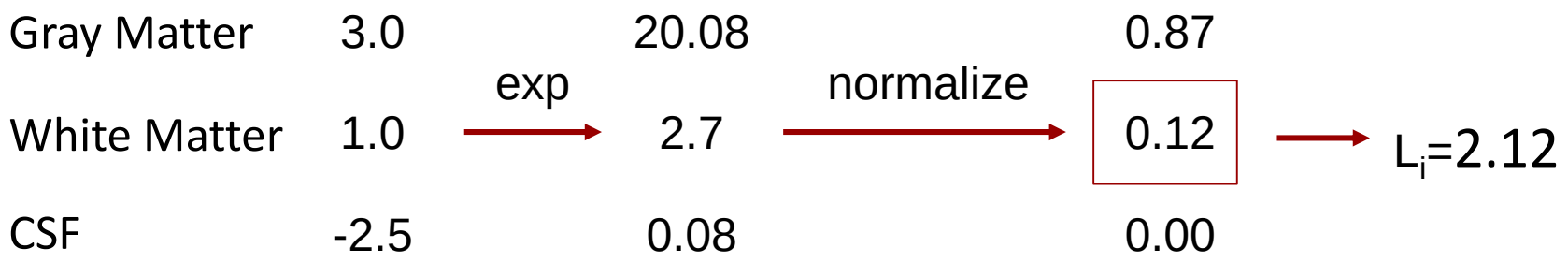


U-Net Training: Cross-entropy Loss

Using **cross-entropy** between the softmax output and a hard labeling as a loss amounts to minimizing the negative log-likelihood of the true class y_i:

$$L_i = -ln\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

Example, assuming the voxel is labeled as "white matter":



U-Net Training: Weighted Loss

- Taking a *weighted* average of per-pixel loss (e.g., with inverse class frequency) can help to properly reproduce minority classes
- U-Net paper applied additional weights w_0 to emphasize boundaries

$$w(x) = w_c(x) + w_0 \exp\left(-\frac{\left(d_1(x) + d_2(x)\right)^2}{2\sigma^2}\right)$$

  - w_c is used to balance class frequencies, w_0 for boundary emphasis
  - d_1/d_2 are distances to nearest and second nearest cells

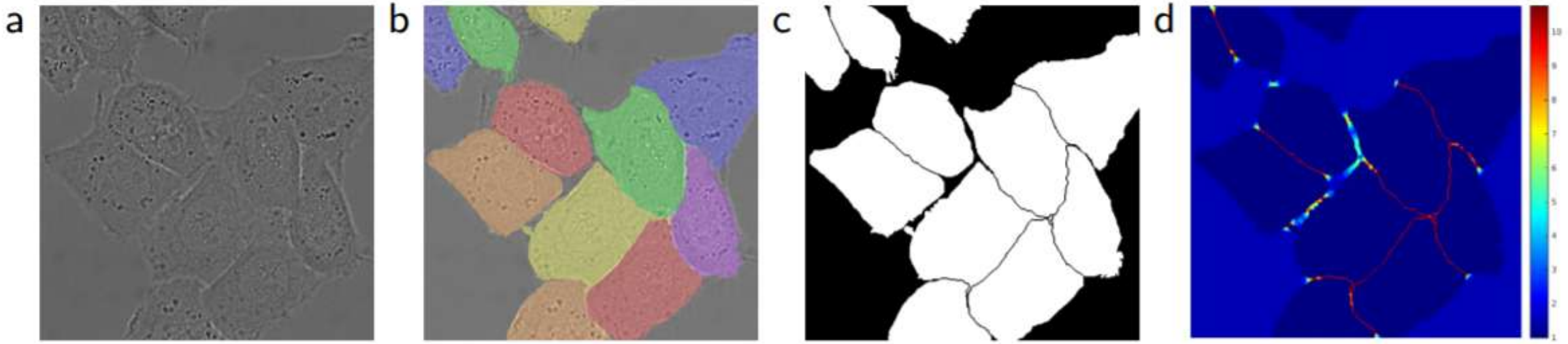


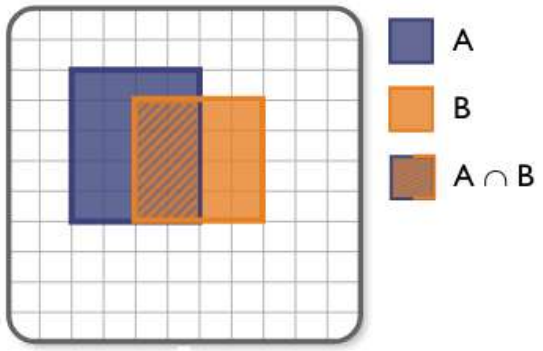
Image from Ronneberger et al.

U-Net Training: Dice Loss

- Based on the Dice score, [Milletari et al. 2016] proposed a differentiable **Dice loss** for training segmentation networks

$$\mathrm{DL}(a, b) = 1 - \frac{2\sum_i a_i b_i + \epsilon}{\sum_i a_i^2 + \sum_i b_i^2 + \epsilon}$$

  - Sums are taken over all pixels i, small $\epsilon > 0$ for regularization
  - Addresses class imbalance, but optimization can be unstable
    - Possible to combine it with cross-entropy by adding both up

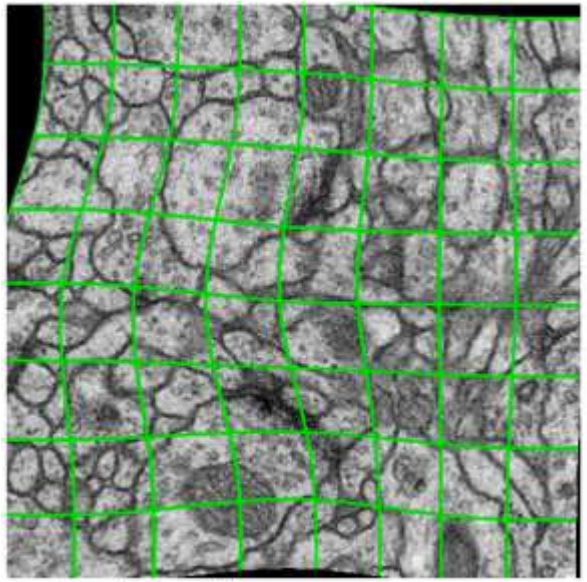


$$DSC(A,B) = \frac{2 \left|A \cap B\right|}{\left|A\right| + \left|B\right|}$$

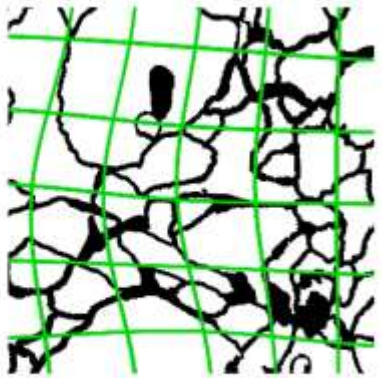
Image from Reinke et al. 2021

U-Net Training: Augmentation

- Obtaining **enough training data** is a major practical challenge
  - Per-pixel annotations are time consuming, expert time is expensive
- Common to **augment** the available data by applying random transformations
  - Rotations, scaling, Gaussian noise, Gaussian blur, brightness, contrast, …
- Augmentations that affect pixel positions need to be applied to image *and* labels!
  - U-Net augments cell microscopy images with **random elastic deformations**



resulting deformed image

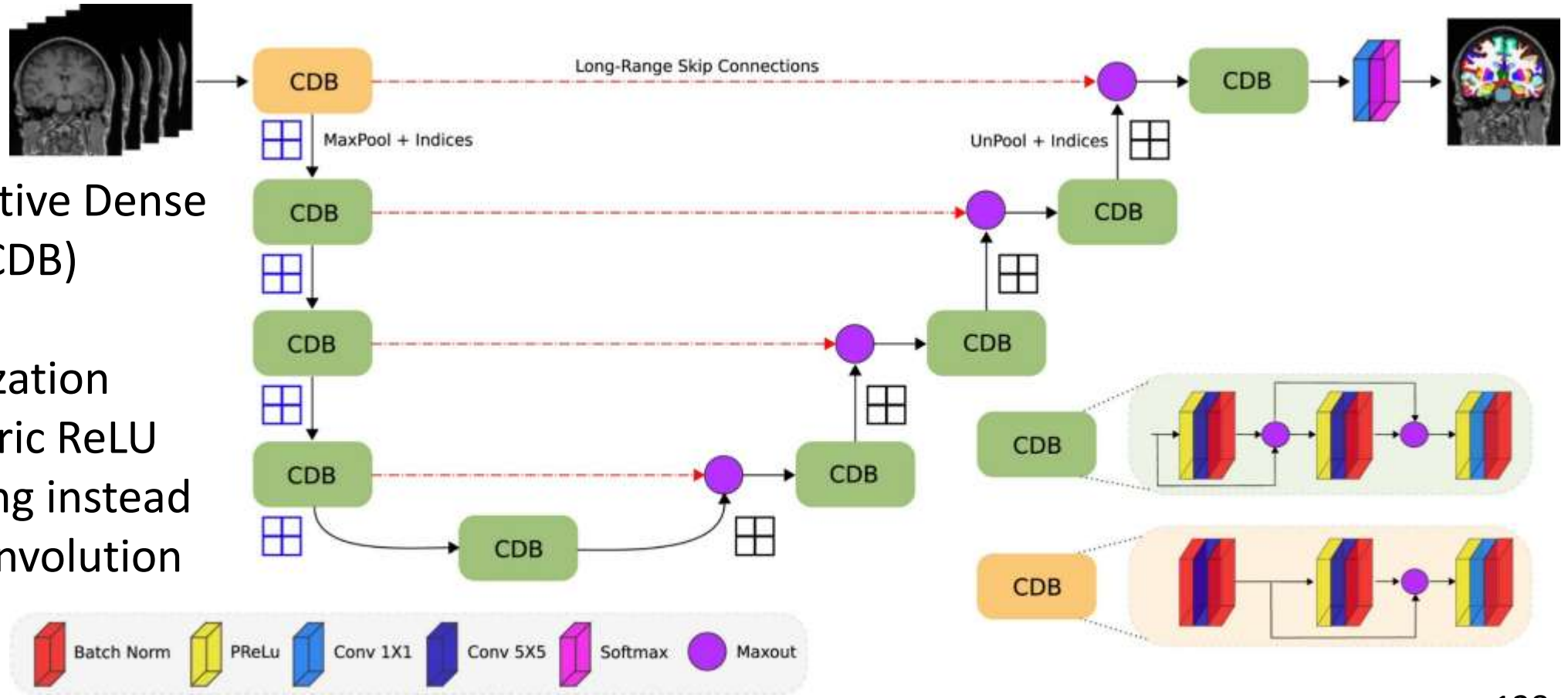


correspondingly deformed manual labels

Figure from https://lmb.informatik.uni-freiburg.de/people/ronneber/u-net/

FastSurfer

- The FastSurfer CNN architecture [Henschel et al., 2020] is similar to the U-Net, but uses some additional concepts

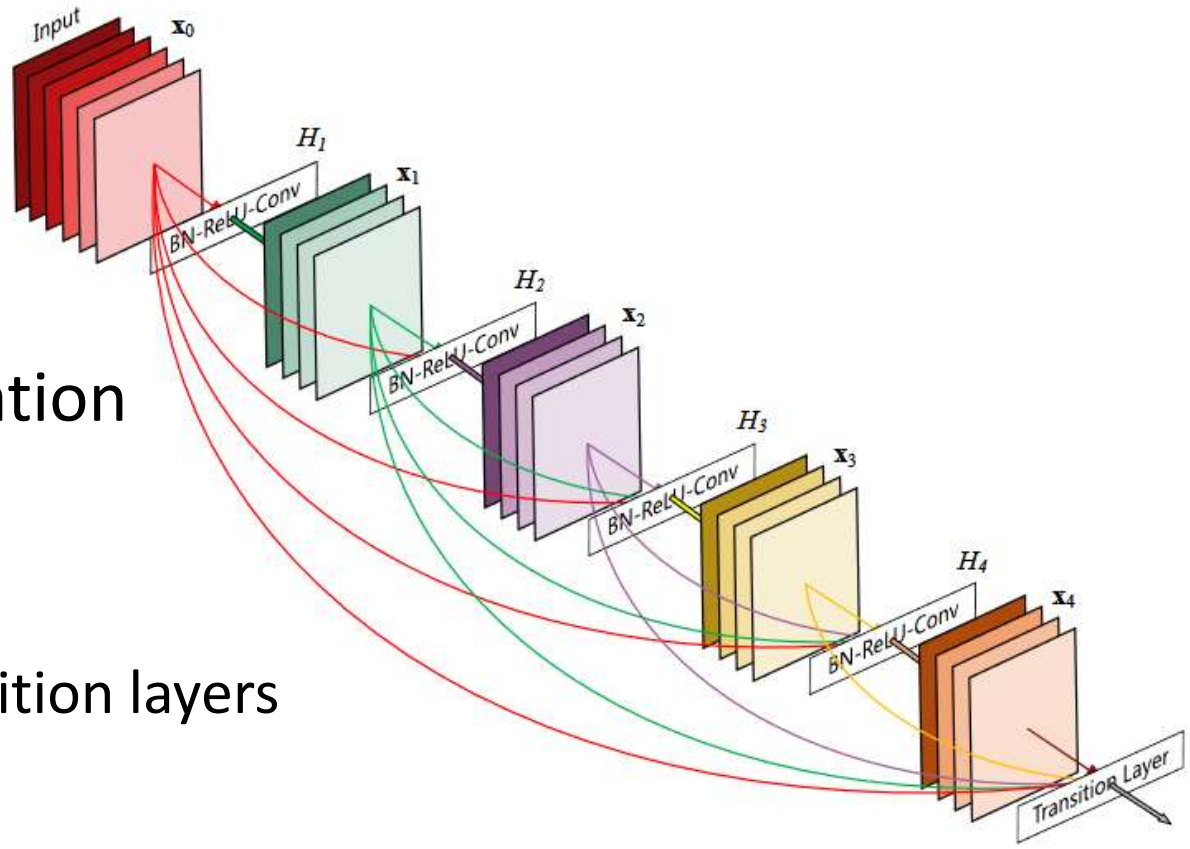


- Competitive Dense Blocks (CDB)
- Batch Normalization
- Parametric ReLU
- Unpooling instead of up-convolution

Figure from [Henschel et al. 2020]

DenseNet

- **DenseNets** [Huang et al., 2017] connect each layer to *all* previous ones
  - *Idea:* Encourage feature re-use, layers should focus on extracting new useful information instead of having to also preserve existing information
  - Dimensions of activation maps need to remain the same
    - Use of DenseNet blocks with transition layers in between



Competitive Dense Blocks

- DenseNet **concatenates** all previous layers, which increases the number of parameters in subsequent kernels
- FastSurfer increases efficiency by replacing concatenation with **maxout**
  - Convolution layer outputs activation volume of the same dimensions as its input, maxout takes the pointwise maximum of both
  - Can be understood as “competition” between old and new features
  - Skip connections between encoder and decoder are implemented in the same way

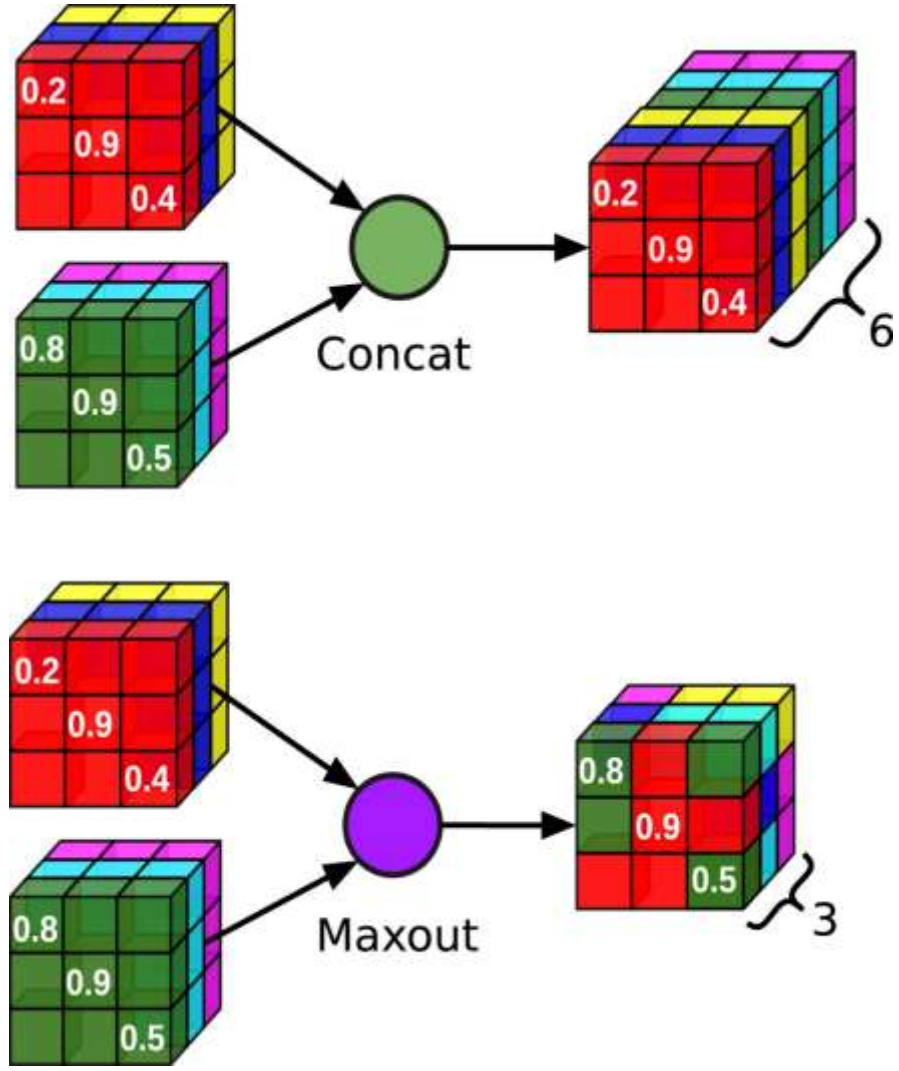


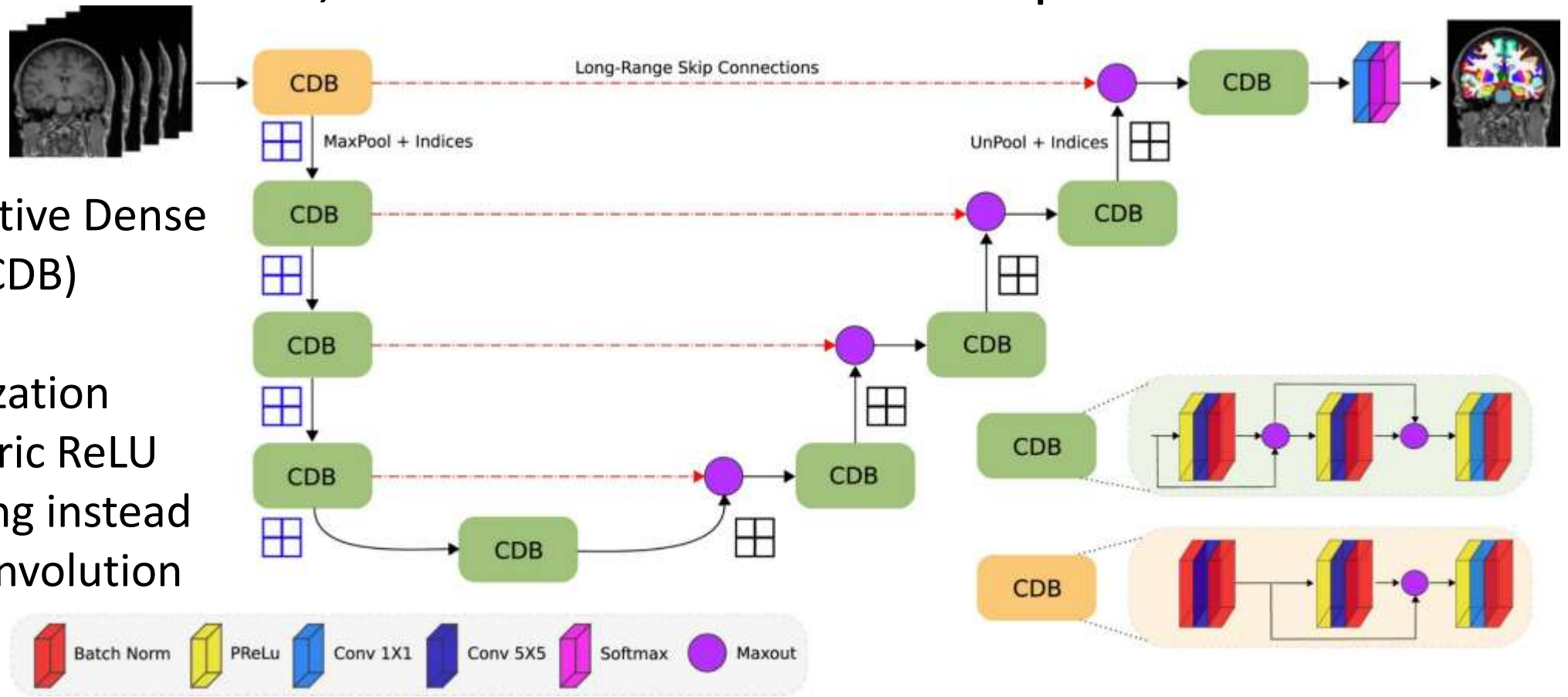
Figure from [Henschel et al. 2020]

Batch Normalization

- Training updates all layers simultaneously, based on their current input
  - *But:* Adjusting weights for earlier layers shifts the inputs of later ones
- *Idea*: Re-parametrize activations $\mathbf{H}$ to zero mean/unit variance
  - During training, $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ are computed for each mini batch
  - At test time, we apply $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$ from a running average during training
- To preserve capacity, we shift and scale with learnable parameters $\boldsymbol{\beta}$,$\boldsymbol{\gamma}$
  - *Benefit*: Modified network is easier to train. Mean and std.-dev. depend directly on $\boldsymbol{\beta}$,$\boldsymbol{\gamma}$, not on a complex interdependence between the layers
- **Batch normalization** usually makes training faster and more stable
  - No consensus on whether BN should come before or after activation
- FastSurfer also uses it to standardize the input to the first layer

Recap: FastSurfer

- The FastSurfer CNN architecture [Henschel et al., 2020] is similar to the U-Net, but uses additional concepts:



- Competitive Dense Blocks (CDB)
- Batch Normalization
- Parametric ReLU
- Unpooling instead of up-convolution

Figure from [Henschel et al. 2020]

Segmenting 3D Images

- The architectures we discussed operate on 2D images
  - *Quiz*: How could we apply them to 3D MR images?
- 3D CNNs are straightforward, but often impractical
  - Large number of parameters, high memory demands
  - Deeper 2D CNNs often work better than shallower 3D CNNs
- FastSurfer combines 2D CNNs with two strategies:
  1. **Using 3D context:** Provide 3 additional slices on both sides, only segment the central one
  2. **View aggregation:** Segment after slicing along all three axes. Obtain final segmentation as weighted average of the three results
     - *Note*: Left/right hemispheres are not reliably distinguished in sagittal slices. Corresponding left/right classes are merged, weight of sagittal views is reduced by one half in the final aggregate.

FastSurfer: Evaluation Results

- The FastSurfer CNN distinguishes between 78 labels
- In a comparison on 20 manually labeled subjects, the combination of competitive dense blocks (CDB) and spatial information (SPI) yielded the most accurate results

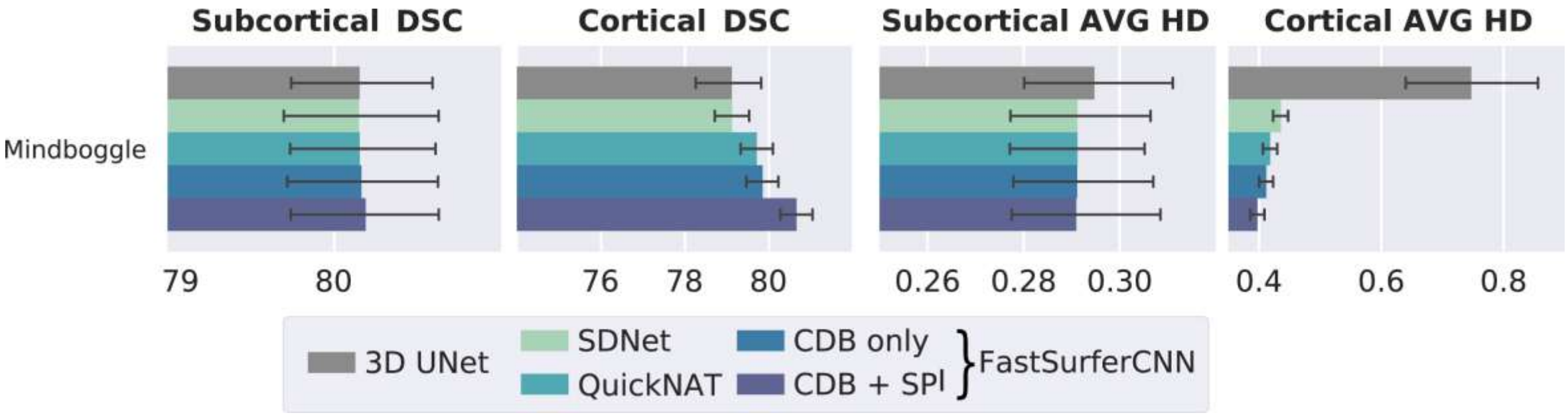


Figure from [Henschel et al. 2020]

Summary: CNN-based Segmentation

- **Convolutional Neural Networks** provide state-of-the-art results for many segmentation tasks
  - Training is costly, but inference often much cheaper than alternatives
- **U-Net architecture** has proven to be widely applicable when trained with sufficiently many examples
  - **Data augmentation** helpful to increase robustness
- **FastSurfer** CNN architecture specialized for brain segmentation
  - 3D context and view aggregation for 3D segmentation

Cortical Surfaces and Cortical Thickness

- FreeSurfer / FastSurfer provide pipelines to reconstruct the **inner (gray/white)** and **outer (pial)** **cortical surfaces**
  - Enables measurements of cortical thickness
  - Can be combined with segmentation of cortical regions
  - Can be registered between subjects

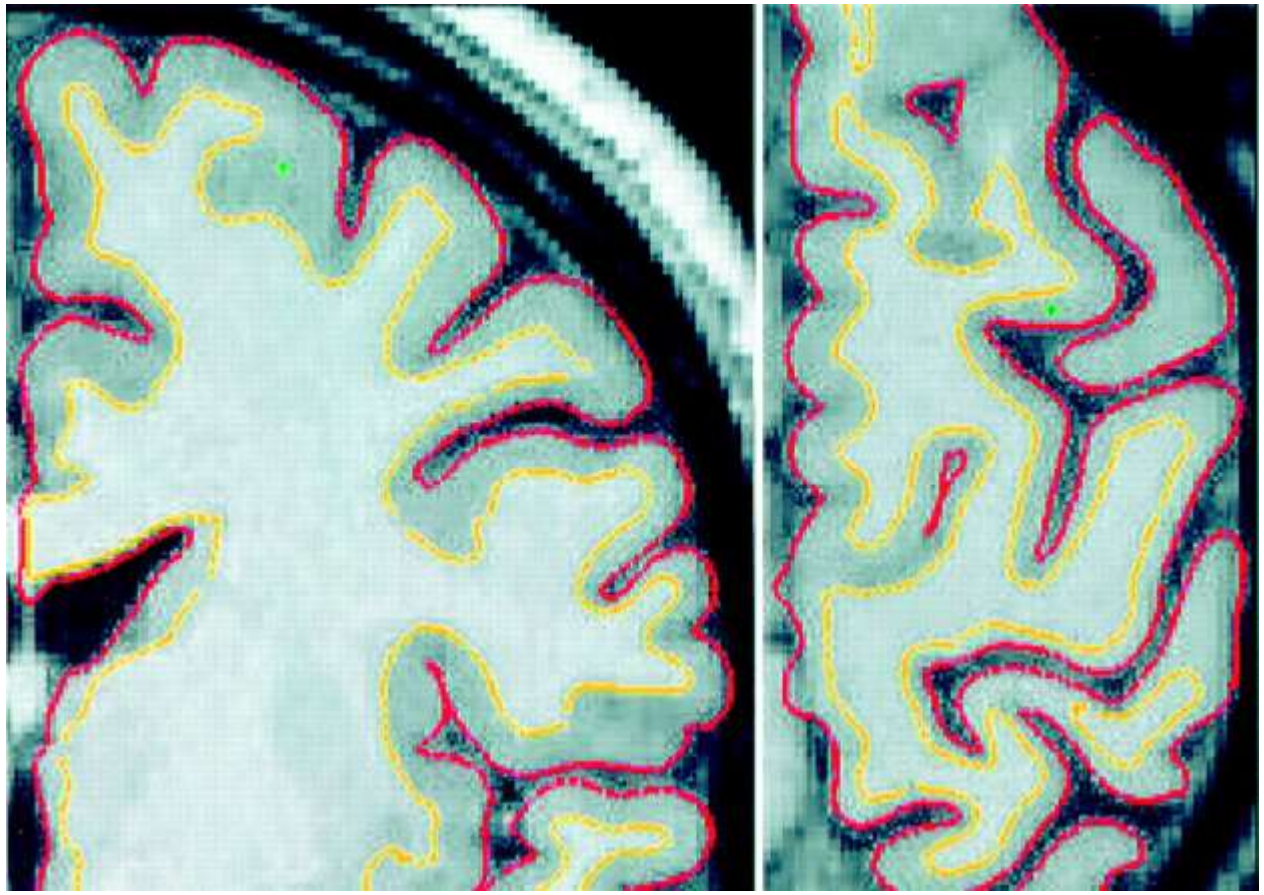
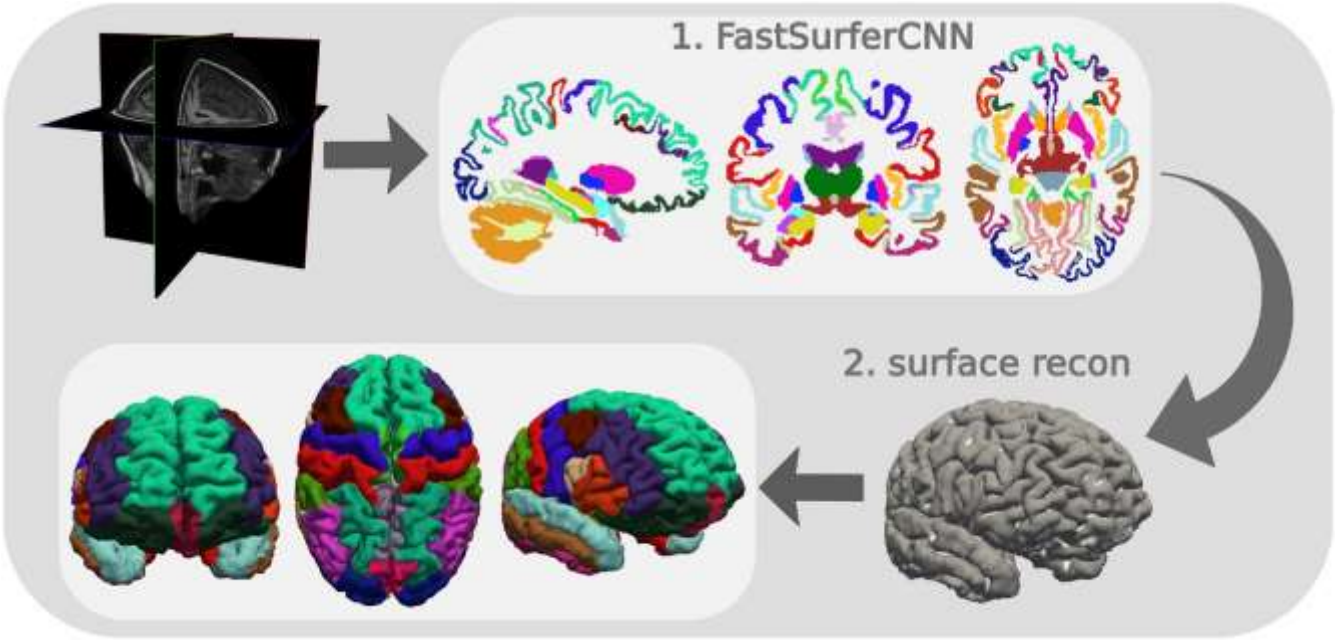
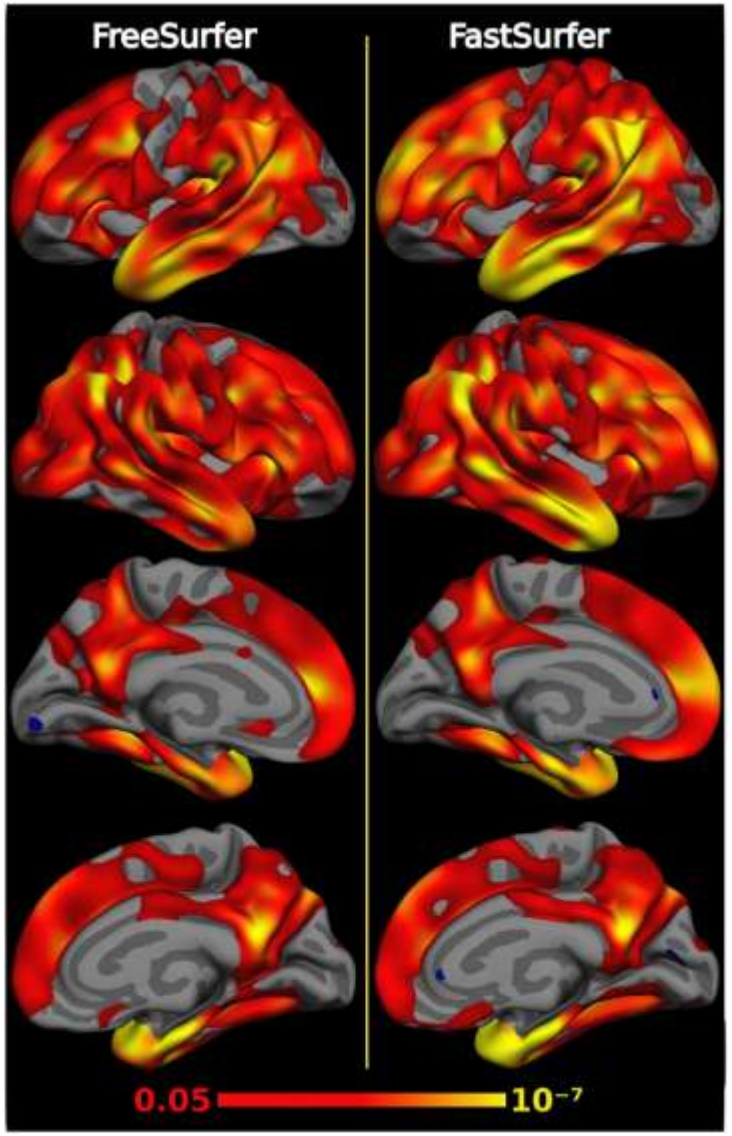


Figure from [Fischl et al. 2000]

Group Comparisons of Cortical Thickness





- When comparing cortical thickness of patients with Alzheimer's disease to healthy controls, FreeSurfer and FastSurfer suggest similar patterns
- FastSurfer analysis slightly more sensitive

Figures from [Henschel et al. 2020]

Further Reading: Markov Random Fields

- Simon J.D. Prince: *Computer Vision: Models, Learning, and Inference.* Cambridge University Press, 2012
- Y. Zhang et al.: *Segmentation of Brain MR Images Through a Hidden Markov Random Field Model and the Expectation-Maximization Algorithm.* IEEE Trans. Medical Imaging 20(1):45-57, 2001
- W.M. Wells, III et al.: *Adaptive Segmentation of MRI Data.* IEEE Trans. Medical Imaging 15(4):429-442, 1996

Further Reading: BET, FreeSurfer, U-Net, FastSurfer

- S. Smith: *Fast Robust Automated Brain Extraction.* Human Brain Mapping 17:143-155, 2002
- B. Fischl et al.: *Measuring the thickness of the human cerebral cortex from magnetic resonance images.* PNAS 97:11050-11055, 2000
- B. Fischl et al.: *Whole Brain Segmentation: Automated Labeling of Neuroanatomical Structures in the Human Brain.* Neuron 33:341-355, 2002
- O. Ronneberger et al.: *U-net: Convolutional networks for biomedical image segmentation.* MICCAI 2015
- L. Henschel et al.: *FastSurfer - A fast and accurate deep learning based neuroimaging pipeline.* NeuroImage 219:117012, 2020

Exam-like Questions

1. What happens during the M step of the EM algorithm when using a Gaussian Mixture Model for image segmentation?
2. State an advantage of combining Gaussian Mixture Models with Markov Random Fields for image segmentation.
3. Describe the U-Net CNN architecture for image segmentation. Mention one difference between the FastSurfer CNN and the standard U-Net.